

强化学习面试课程 Roadmap

这套笔记面向 AI（Artificial Intelligence，人工智能）算法工程师的强化学习入门与面试准备。目标不是把强化学习研究方向穷尽，而是系统掌握面试中最常被考察的概念、公式、算法区别、训练技巧和项目表达方式。

0.1 全局符号和缩写说明

阅读约定：每个章节还会在开头重复列出本章会用到的核心符号。一般来说，大写字母表示随机变量、函数或集合，小写字母表示一次采样到的具体取值；例如 S_t 是时刻 t 的状态随机变量， s 是某个具体状态。

注意符号会按上下文复用：例如 A 可以表示 action space（动作空间），而 $A_{\pi}(s,a)$ 可以表示 advantage function（优势函数）； $P(s'|s,a)$ 是 transition probability， $P(i)$ 则是采样概率。

本套笔记统一使用大写 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ 表示价值函数；有些教材会写成小写 $v_{\pi}(s)$ 、 $q_{\pi}(s,a)$ ，含义相同，只是记号风格不同。

符号/缩写	English	中文	含义
AI	Artificial Intelligence	人工智能	本课程面向 AI 算法工程师的面试准备。
RL	Reinforcement Learning	强化学习	agent 通过交互学习最大化长期 reward 的范式。
MDP	Markov Decision Process	马尔可夫决策过程	RL 最常见的数学建模。
DP	Dynamic Programming	动态规划	已知环境模型时用 Bellman backup 求解 MDP 的方法。
MC	Monte Carlo	蒙特卡洛	用完整 episode 的真实 return 估计价值。
TD	Temporal Difference	时序差分	用一步 reward 加下一状态估计值做 bootstrap 更新。
epsilon-greedy	Epsilon-greedy Policy	epsilon-greedy 策略 / epsilon 贪心策略	以 ϵ 概率随机探索，否则选择当前价值最大的动作。
behavior policy	Behavior Policy	行为策略	实际和环境交互、产生训练样本的策略。
target policy	Target Policy	目标策略	算法真正想评估或优化的策略。
on-policy	On-policy Learning	同策略学习	behavior policy 和 target policy 是同一个策略。
off-policy	Off-policy Learning	异策略学习	用 behavior policy 的数据学习另一个 target policy。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	使用实际下一动作更新的 TD control 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
TRPO	Trust Region Policy Optimization	信赖域策略优化	通过 KL 约束限制策略更新幅度。
PPO	Proximal Policy Optimization	近端策略优化	用 clipping objective 稳定策略更新。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 actor-critic 算法。

符号/缩写	English	中文	含义
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 actor-critic 算法，通常用异策略数据训练。
TD3	Twin Delayed DDPG	双延迟 DDPG	改进 DDPG 稳定性的连续控制算法。
A2C / A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic	优势 Actor-Critic / 异步优势 Actor-Critic	actor-critic 的同步和异步版本。
GAE	Generalized Advantage Estimation	广义优势估计	用 λ 平衡 advantage 估计的 bias 和 variance。
UCB	Upper Confidence Bound	置信上界	bandit 中基于不确定性 bonus 的探索方法。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好信号优化模型行为。
RM	Reward Model	奖励模型	RLHF 中根据偏好数据学习出的奖励模型。
S / S_t	State / State at time t	状态空间 / 时刻 t 的状态	S 常表示状态空间， S_t 表示状态随机变量。
A / A_t	Action / Action at time t	动作空间 / 时刻 t 的动作	A 常表示动作空间， A_t 表示动作随机变量。
R / R_{t+l}	Reward	奖励	R_{t+l} 表示执行 A_t 后收到的即时奖励。
G_t	Return	回报 / 累计折扣奖励	G_t 表示从时刻 t 开始的累计收益； G 是常用记号，不建议理解成固定英文缩写。
$P(s' s, a)$	Transition Probability	状态转移概率	在状态 s 执行动作 a 后转移到 s' 的概率。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的期望 return。
$Q_\pi(s, a)$	Action-value Function	动作价值函数	策略 π 下，在 s 先执行 a 的期望 return； Q 可理解为 action quality。
$A_\pi(s, a)$	Advantage Function	优势函数	动作 a 相对状态平均价值的优势；这里的 A 不是 action space。
π	Policy	策略	从 state 到 action 或 action distribution 的映射。
γ	Discount Factor	折扣因子	控制未来 reward 的折扣强度。
θ / w	Parameters / Weights	参数 / 权重	神经网络或函数逼近器的可学习参数。
$J(\theta)$	Objective Function	优化目标函数	policy optimization 中要最大化的目标。
$L(\theta)$	Loss Function	损失函数	训练神经网络时要最小化的目标。
$H(\pi)$	Entropy	熵	衡量策略随机性，常用于鼓励探索。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个概率分布差异。

0.2 学习目标

学完后你应该能做到：

- 清楚解释强化学习和监督学习的区别。
- 熟练写出 MDP、return、value function、Bellman equation。

- 区分 DP、Monte Carlo、TD、SARSA、Q-learning。
- 解释 DQN 为什么需要 replay buffer 和 target network。
- 推导 policy gradient 的核心公式。
- 讲清 Actor-Critic、GAE、PPO、SAC 的动机。
- 理解 offline RL、imitation learning、RLHF 的基础问题。
- 准备一套可以在面试中讲清楚的 RL 项目。

0.3 全课程主线

这套课按“问题建模 -> 精确求解 -> 采样估计 -> 函数逼近 -> 策略优化 -> 进阶约束”的顺序组织。每章不是孤立算法，而是在回答上一个阶段留下的问题：

章节	主要问题	逻辑位置
第 1 章	RL 问题如何写成 MDP、return、value 和 Bellman 方程	建立统一语言
第 2 章	如果环境模型已知，怎样用 DP 精确做 evaluation 和 improvement	Bellman 方程的精确求解原型
第 3 章	如果模型未知，怎样用采样做 prediction 和 control	从 DP 过渡到 model-free tabular RL
第 4 章	学 Q 或 policy 时，数据从哪里来，如何平衡探索和利用	解释采样数据质量
第 5 章	表格法放不下真实状态空间时，怎样用函数逼近	进入 deep RL 的必要前提
第 6 章	Q-learning 加神经网络为什么会不稳，DQN 如何稳定它	value-based deep RL
第 7 章	不想通过 argmax Q 选动作时，如何直接优化 policy	policy-based RL
第 8 章	如何同时利用 policy gradient 和 value learning	actor-critic 框架
第 9 章	policy 更新太大导致崩溃时，如何限制更新幅度	现代 on-policy 稳定训练
第 10 章	连续动作下无法枚举动作时，如何做 off-policy actor-critic	连续控制
第 11 章	交互、模型、离线数据、人类反馈带来哪些新约束	高级 RL 与 LLM 对齐接口
第 12 章	面试中如何横向比较、推公式、讲项目	收束成可表达知识体系

0.4 推荐学习节奏

建议周期：6 到 8 周。

建议投入：每周 8 到 10 小时。

每章建议按这个顺序学习：

1. 先读核心概念。
2. 手写关键公式。
3. 用自己的话讲一遍算法直觉。
4. 回答本章面试高频问题。
5. 至少实现或阅读一个最小代码版本。
6. 独立完成章末练习，再点击展开参考答案核对推理过程。

0.5 推导阅读方式

RL 的公式不要直接背结论，建议按下面链路推：

1. 先写目标：agent 要最大化长期 return，而不是单步 reward。
2. 再拆递归：把 G_t 写成 $R_{t+1} + \gamma G_{t+1}$ ，自然得到 Bellman 思路。
3. 再区分期望和采样：Bellman 方程是期望形式，TD、SARSA、Q-learning 是用一次 transition 近似这个期望。
4. 再看函数逼近：DQN 把 tabular update 变成对 Bellman target 的监督回归。
5. 再看策略优化：policy gradient 用 log-derivative trick 处理“采样分布本身依赖参数”的问题。
6. 最后看稳定化：baseline、critic、GAE、target network、replay、PPO clipping、SAC entropy 都是在处理方差、偏差、分布漂移或探索。

面试里如果一时忘公式，可以从这条链路往回推：目标是什么，target 是什么，target 为什么可靠，更新过大或估计不准会出什么问题。

0.6 新名词阅读约定

本套笔记后续按一个硬规则写：一个新名词第一次进入推导前，必须先说明它的英文、中文、解决什么问题，以及它和当前公式的关系。

读到算法段落时，建议先检查四件事：

1. 当前数据是谁采出来的，也就是 behavior policy。
2. 当前算法想学谁，也就是 target policy。
3. 当前 target 用的是完整 return、实际下一动作，还是 greedy max。
4. 因此它是 on-policy 还是 off-policy。

0.7 例子阅读方式

每章都补了“本章例子”或“典型例子”。读例子时不要只记场景名，要把它映射回四件事：

1. 状态是什么。
2. 动作是什么。
3. 奖励怎么定义。
4. 算法为什么适合这个场景。

例如 CartPole 不是“一个小游戏”，而是：

元素	例子
状态	小车位置、速度、杆角度、角速度。
动作	向左推或向右推。
奖励	杆保持不倒时每步 +1。
算法	离散动作、低维状态时可用 DQN 或 PPO。

面试时讲例子的目的不是展示你见过环境，而是证明你能把业务问题建模成 MDP，再解释为什么某个算法匹配这个 MDP。

0.8 章节目录

章节	文件	主题
第 1 章	01-rl-fundamentals-mdp-bellman	RL 基本设定、MDP、价值函数、Bellman 方程
第 2 章	02-dynamic-programming	Policy Evaluation、Policy Iteration、Value Iteration

章节	文件	主题
第 3 章	03-monte-carlo-td-learning	Monte Carlo、TD、SARSA、Q-learning
第 4 章	04-exploration-bandit	探索与利用、Multi-Armed Bandit、UCB、Thompson Sampling
第 5 章	05-function-approximation-deadly-triad	函数逼近、稳定性、Deadly Triad
第 6 章	06-dqn-and-variants	DQN、Double DQN、Dueling DQN、Prioritized Replay
第 7 章	07-policy-gradient	REINFORCE、Policy Gradient、baseline、advantage
第 8 章	08-actor-critic-gae	Actor-Critic、A2C、A3C、GAE
第 9 章	09-ppo-trpo	TRPO、PPO、KL 约束、clipping objective
第 10 章	10-continuous-control-ddpg-td3-sac	DDPG、TD3、SAC、连续动作控制
第 11 章	11-model-based-offline-imitation-rlhf	Model-based RL、Offline RL、Imitation Learning、RLHF
第 12 章	12-interview-cheatsheet-projects	面试速查、横向对比、项目准备

0.9 面试复习优先级

如果时间有限，优先掌握：

1. MDP、Bellman 方程、V/Q 函数。
2. MC vs TD，SARSA vs Q-learning。
3. on-policy vs off-policy。
4. DQN 的 replay buffer、target network、Double DQN。
5. Policy Gradient 推导，baseline 为什么不引入 bias。
6. Actor-Critic 和 GAE。
7. PPO clipping objective。
8. SAC 的 entropy regularization。
9. offline RL 为什么难。
10. RLHF 的 PPO/RM/Preference data 基本流程。

0.10 推荐资料

- Sutton & Barto, Reinforcement Learning: An Introduction
- David Silver Reinforcement Learning Course
- OpenAI Spinning Up
- CleanRL single-file implementations
- Stable-Baselines3 文档和示例

0.11 面试表达模板

学习每个算法时，都尝试用这个模板总结：

算法名:
解决的问题:
核心思想:
目标函数或更新公式:
on-policy/off-policy:
value-based/policy-based/actor-critic:
稳定训练技巧:
优点:
缺点:
常见面试追问:

第 1 章：强化学习基本设定、MDP（Markov Decision Process，马尔可夫决策过程）与 Bellman 方程

1.1 本章符号说明

本章第一次出现公式前，先把核心符号列清楚：

本章统一写大写 $V_\pi(s)$ 和 $Q_\pi(s,a)$ ；如果你在 Sutton & Barto 等教材里看到小写 $v_\pi(s)$ 、 $q_\pi(s,a)$ ，它们表示同类 state-value/action-value function。

符号	English	中文	含义
S / S_t	State / State at time t	状态空间 / 时刻 t 的状态	S 是状态空间， S_t 是时刻 t 的状态随机变量。
s	State realization	具体状态	小写 s 表示某一个具体 state，不是 value function。
A / A_t	Action / Action at time t	动作空间 / 时刻 t 的动作	A 是动作空间， A_t 是时刻 t 的动作随机变量。
a	Action realization	具体动作	小写 a 表示某一个具体 action，不是 action-value function。
R / R_{t+1}	Reward	奖励	R_{t+1} 是执行 A_t 后得到的即时奖励。
G_t	Return	回报 / 累计折扣奖励	G_t 表示从时刻 t 开始的长期累计收益； G 是常用记号，不建议理解成固定英文缩写。
$P(s' s, a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$\pi(a s)$	Policy	策略	在状态 s 选择动作 a 的概率。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的期望 return。
$Q_\pi(s, a)$	Action-value Function	动作价值函数	策略 π 下，在 s 先执行 a 的期望 return； Q 可理解为 action quality。
γ	Discount Factor	折扣因子	控制未来 reward 权重的系数。
α	Learning Rate / Step Size	学习率 / 步长	控制一次 TD 或 Q-learning 更新吸收多少新误差信号，通常满足 $0 < \alpha \leq 1$ 。
RL	Reinforcement Learning	强化学习	本课程主题，学习长期决策策略。
POMDP	Partially Observable Markov Decision Process	部分可观测马尔可夫决策过程	观测不满足完整 Markov property 时的建模。
TD	Temporal Difference	时序差分	后续用 Bellman target 做采样更新的方法。
DQN / PPO / A2C / SAC	Deep Q-Network / Proximal Policy Optimization / Advantage Actor-Critic / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 优势 Actor-Critic / 软 Actor-Critic	后续章节会学习的典型 deep RL 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法名。

1.2 本章目标

本章是强化学习的地基。你需要掌握：

- 强化学习和监督学习的区别。
- agent、environment、state、action、reward、policy 的含义。
- return 和 discount factor。
- MDP 五元组和 Markov property。
- $V(s)$ 、 $Q(s,a)$ 的定义和关系。
- Bellman expectation equation 与 Bellman optimality equation。

1.3 本章主线

本章先把 RL 问题压成一个标准数学对象：MDP。顺序是 agent 和 environment 交互，得到 trajectory；trajectory 产生 return；return 的期望定义出 $V_\pi(s)$ 和 $Q_\pi(s,a)$ ；value function 的自治关系就是 Bellman equation。后续所有算法基本都在近似求解或优化这些 Bellman 关系。

1.4 强化学习问题设定

1.4.1 强化学习在学什么

强化学习研究的是：一个 agent 通过和 environment 交互，学习一个 policy，使得长期累计 reward 最大。

一次交互可以写成：

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

含义：

- S_t ：State at time t ，时刻 t 的状态。
- A_t ：Action at time t ，时刻 t 的动作。
- R_{t+1} ：Reward after action，执行 A_t 后得到的奖励。
- S_{t+1} ：Next state，环境转移到的下一个状态。

监督学习通常学习 $x \rightarrow y$ 的映射，而强化学习学习的是 sequential decision making。当前动作不仅影响当前 reward，也会影响未来会遇到的状态分布。

面试表达：

RL 的核心难点在于 delayed reward 和 sequential decision making。Agent 的动作会影响未来状态分布，因此数据不是固定的 i.i.d. 样本，目标也不是单步预测正确，而是长期累计收益最大。

1.4.2 Return

强化学习优化的是累计回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

也就是：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

这里 G_t 表示 Return，也就是“从时刻 t 开始往后看的累计回报”。 G 是强化学习教材中的常用记号，不建议把它理解成某个固定英文缩写。

γ 是 discount factor，通常在 $[0, 1]$ 。

γ 的作用：

- $\gamma = 0$ 时, agent 只关心眼前 reward。
- γ 接近 1 时, agent 更重视长期 reward。
- 在 infinite horizon 下, 折扣可以让 return 更容易有界。
- 工程上, γ 也控制短视和长远之间的权衡。

面试追问: 为什么需要 discount factor?

可以回答:

一方面是数学上让无限时域的累计 reward 更容易收敛, 另一方面表达了对未来 reward 的偏好衰减或不确定性。 γ 越大越重视长期收益, 但训练可能更难、更不稳定。

1.4.3 Policy

Policy 描述 agent 如何根据 state 选择 action。

确定性策略:

$$a = \pi(s)$$

随机策略:

$$\pi(a | s) = P(A_t = a | S_t = s)$$

强化学习最终通常是在寻找最优策略:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[G_t]$$

常见算法分类:

- value-based: 先学习价值函数, 再通过价值函数导出策略, 例如 Q-learning、DQN。
- policy-based: 直接参数化策略并优化, 例如 REINFORCE、PPO。
- actor-critic: 同时学习策略 actor 和价值 critic, 例如 A2C、SAC。

1.5 MDP 建模

1.5.1 MDP

MDP 是 Markov Decision Process, 强化学习最常见的数学建模。

MDP 五元组:

$$(S, A, P, R, \gamma)$$

分别表示:

- S : State Space, 状态空间。
- A : Action Space, 动作空间。
- $P(s' | s, a)$: Transition Probability, 状态转移概率。
- $R(s, a)$ 或 $R(s, a, s')$: Reward Function, 奖励函数。
- γ : Discount Factor, 折扣因子。

MDP 的核心是假设 Markov property:

$$\begin{aligned} P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) \\ = P(S_{t+1} | S_t, A_t) \end{aligned}$$

也就是说，当前 state 已经包含了预测未来所需的全部历史信息。

面试表达：

MDP 假设 state 是历史信息的 sufficient statistic。给定当前 state 和 action 后，未来状态转移与更早的历史无关。如果观测不满足这个条件，例如只能看到局部信息，则更准确的模型是 POMDP。

1.6 Value Function 与 Bellman 方程

1.6.1 Value Function

状态价值函数 (State-value Function, 记作 V , V 来自 value)：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

含义：从状态 s 出发，之后一直按照策略 π 行动，期望累计 return 是多少。

动作价值函数 (Action-value Function, 记作 Q , Q 可理解为 action quality)：

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

含义：在状态 s 先执行动作 a ，之后按照策略 π 行动，期望累计 return 是多少。

两者关系：

$$V_{\pi}(s) = \sum_a \pi(a | s) Q_{\pi}(s, a)$$

最优情况下：

$$V^*(s) = \max_a Q^*(s, a)$$

最优动作：

$$a^* = \arg \max_a Q^*(s, a)$$

为什么 Q 更适合直接决策？

因为 $Q(s, a)$ 已经评价了具体 action 的长期价值，可以直接用 $\arg \max_a Q(s, a)$ 选动作。而 $V(s)$ 只评价状态好坏，如果不知道环境转移模型，单独的 $V(s)$ 不一定能直接告诉你该选哪个动作。

1.6.2 Bellman 方程

Return 可以递归拆开：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

于是价值函数也可以递归表示：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

展开得到 Bellman expectation equation：

$$\begin{aligned} V_{\pi}(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_{\pi}(s')] \end{aligned}$$

对于动作价值函数：

$$\begin{aligned} Q_{\pi}(s, a) &= \sum_{s'} P(s' | s, a) \\ &[R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q_{\pi}(s', a')] \end{aligned}$$

最优 Bellman 方程：

$$\begin{aligned} V^*(s) &= \max_a \sum_{s'} P(s' | s, a) \\ &[R(s, a, s') + \gamma V^*(s')] \end{aligned}$$

$$\begin{aligned} Q^*(s, a) &= \sum_{s'} P(s' | s, a) \\ &[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')] \end{aligned}$$

这个式子非常重要。Q-learning 和 DQN 都是在用采样和函数逼近来逼近它。

1.6.3 Bellman 推导过程：不要直接背公式

Bellman 方程可以按四步讲。

第一步，从 return 的定义出发：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

把第一项单独拿出来，后面的部分正好是从 $t+1$ 开始的 return：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

第二步，对“从状态 s 出发”的长期回报取期望：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t=s]$$

把递归式代进去：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t=s]$$

第三步，把 G_{t+1} 换成下一状态的价值。因为到达 S_{t+1} 后，后续仍按策略 π 行动：

$$\mathbb{E}_{\pi}[G_{t+1} | S_{t+1}=s'] = V_{\pi}(s')$$

于是得到：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t=s]$$

第四步，把期望展开成“先按策略选动作，再按环境转移到下一个状态”：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

这就是 Bellman expectation equation。

从 expectation 到 optimality 只改一个地方：不再对动作按当前策略求平均，而是假设在每个状态都选最优动作：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

面试表达：

Bellman 方程本质是价值函数的递归自洽关系。当前状态的价值等于即时 reward 加上下一个状态价值的折扣期望。Expectation version 是评估固定策略，optimality version 是每一步都对动作取最大值。

1.6.4 从 Bellman 到算法的推理链

不同 RL 算法可以看成在回答同一个问题：怎么逼近 Bellman backup?

方法	推理方式	需要环境模型吗	用什么近似 Bellman target
DP	直接对 $P(s' s,a)$ 求和	需要	完整期望 backup
MC	采样完整 episode	不需要	实际 return G_t
TD	采样一步 transition	不需要	$r + \gamma V(s')$
Q-learning	采样一步 transition，并对下一动作取 max	不需要	$r + \gamma \max_a Q(s', a')$
DQN	用神经网络拟合 Q-learning target	不需要	$r + \gamma \max_a Q(s', a'; \theta)$

所以不要把这些算法看成完全无关的技巧。它们都围绕 Bellman target，只是期望怎么估、target 怎么构造、是否用函数逼近不同。

1.6.5 TD Target 与 TD Error

Q-learning 的更新式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

其中：

$$\begin{aligned} TD \text{ target} &= r + \gamma \max_{a'} Q(s', a') \\ TD \text{ error} &= TD \text{ target} - Q(s, a) \end{aligned}$$

直觉：

- 旧估计是 $Q(s, a)$ 。
- 新观测告诉我们，一步之后的目标应该是 $r + \gamma \max_{a'} Q(s', a')$ 。
- 两者差值就是 TD error。
- 用学习率 α 做一个增量更新。

1.7 本章例子：把问题建模成 MDP

1.7.1 例子 1: GridWorld 迷宫

一个机器人在网格里走，从起点走到终点。

MDP 元素	对应例子
State	机器人当前格子坐标，例如 (2,3)。
Action	上、下、左、右。
Transition	执行动作后到达相邻格子，也可能因为墙或噪声停在原地。
Reward	到终点 +10，撞墙 -1，每走一步 -0.1。
Policy	在每个格子选择哪个方向走。

MDP 元素	对应例子
Value	从某个格子出发，长期来看能拿到多少 reward。

这个例子的价值在于直观：离终点近、不容易撞墙的状态价值高；容易掉进惩罚区域的状态价值低。

1.7.2 例子 2：推荐系统

用户打开 App，系统要选择展示什么内容。

MDP 元素	对应例子
State	用户画像、最近点击、当前场景、时间、设备。
Action	推荐某个视频、商品或文章。
Reward	点击、停留、购买、负反馈等综合信号。
Transition	用户看完内容后兴趣状态变化。
Policy	给不同用户和场景选择不同推荐内容。

如果只看单次点击，这更像 bandit；如果考虑长期留存、疲劳、兴趣迁移，就更像完整 RL。

面试表达：

我会先问这个问题有没有长期影响。如果动作只影响当前 reward，可以先用 bandit；如果动作会改变后续状态，例如用户兴趣或库存状态，就需要 MDP/RL 视角。

1.8 本章面试高频问题

1.8.1 RL 和监督学习有什么区别？

监督学习有固定标签，通常假设样本 i.i.d.，目标是拟合输入到标签的映射。RL 没有逐步标准答案，只有 reward，且 reward 可能延迟出现。Agent 的动作会改变未来状态分布，因此需要处理 exploration-exploitation 和长期收益最大化。

1.8.2 什么是 MDP？

MDP 是强化学习的标准建模，由状态空间、动作空间、转移概率、奖励函数和折扣因子组成。它的核心假设是 Markov property，即未来只依赖当前状态和动作，而不依赖完整历史。

1.8.3 V 和 Q 有什么区别？

$V_{\pi}(s)$ 衡量状态 s 在策略 π 下的长期价值。 $Q_{\pi}(s,a)$ 衡量在状态 s 先执行动作 a 后，再跟随策略 π 的长期价值。 Q 多了动作维度，因此更适合直接用来选动作。

1.8.4 Bellman expectation equation 和 Bellman optimality equation 区别？

Bellman expectation equation 用来评估一个给定策略 π 的价值。Bellman optimality equation 描述最优价值函数，会在动作上取最大值，对应寻找最优策略。

1.8.5 Bellman 方程的核心直觉是什么？

当前价值等于即时奖励加上折扣后的未来价值。它把长期决策问题拆成一步 reward 和下一状态 value 的递归组合。

1.9 本章练习

1. 用自己的话解释为什么强化学习的数据不是固定 i.i.d.。
2. 手写 $V_{\pi}(s)$ 、 $Q_{\pi}(s,a)$ 的定义，以及 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ 的关系。
3. 从 $G_t = R_{t+1} + \gamma G_{t+1}$ 推导 Bellman expectation equation。

4. 解释为什么 $Q^*(s,a)$ 的 Bellman optimality equation 中有 $\max_a$ 。

▼ 参考答案 (点击展开)

1. 为什么 RL 数据不是固定 i.i.d.: 相邻 transition 来自同一条 trajectory, 时间上有相关性; 当前 policy 决定会访问哪些 state 和 action, 而 policy 又会随训练更新; agent 的 action 还会改变后续 state。因此数据分布会被策略和交互过程共同改变, 不是预先固定、相互独立且同分布的样本集。
2. V 、 Q 的定义和关系: $V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$; $Q_\pi(s,a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ 。二者满足 $V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s,a)$; 反过来, $Q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t = a]$ 。
3. Bellman expectation 推导: 从 $G_t = R_{t+1} + \gamma G_{t+1}$ 出发, 对条件 $S_t = s$ 取期望: $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$ 。根据下一状态价值的定义, 用 $V_\pi(S_{t+1})$ 替换对 G_{t+1} 的条件期望, 得到 $V_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s]$ 。
4. 为什么有 $\max_a$: $Q(s,a)$ 假设当前先执行 a , 从下一状态开始继续采用最优策略。因此到达 s' 后应选择价值最大的下一动作, 即 $\max_a Q(s',a')$; 它表达的是“最优延续”, 不是环境随机性。

第 2 章：DP（Dynamic Programming，动态规划）、Policy Evaluation 与 Value Iteration

2.1 本章符号说明

符号/缩写	English	中文	含义
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
$P(s' s, a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$R(s, a, s')$	Reward Function	奖励函数	从 s 执行 a 到 s' 时的奖励。
π / π_{new}	Policy / New Policy	策略 / 新策略	当前被评估或改进的策略。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s, a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s, a) 的长期价值。
$V_k(s)$	Value Estimate at iteration k	第 k 次迭代的价值估计	value iteration / policy evaluation 中的中间估计。
$V(s) / \pi(s)$	Optimal Value / Policy	最优价值 / 最优策略	Bellman optimality backup 的收敛结果，以及由它导出的最优动作规则。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
GPI	Generalized Policy Iteration	广义策略迭代	policy evaluation 和 policy improvement 交替推进的通用框架。
MDP	Markov Decision Process	马尔可夫决策过程	本章 DP 要求解的对象。
RL	Reinforcement Learning	强化学习	DP 是 RL 的理论原型之一。
TD	Temporal Difference	时序差分	后续用采样近似 Bellman backup 的方法。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 Q-learning 的算法。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	第 3 章会讲的采样式 TD control 算法。

2.2 本章目标

本章学习在已知环境模型 P 和 R 的情况下，如何通过动态规划求解 MDP。

你需要掌握：

- Policy Evaluation。
- Policy Improvement。
- Policy Iteration。
- Value Iteration。
- Generalized Policy Iteration。
- DP 方法的局限性。

2.3 本章主线

本章回答一个理想化问题：如果 $P(s' | s, a)$ 和 $R(s, a, s')$ 都已知，Bellman equation 能不能直接求解？答案是可以。Policy Evaluation 负责“评估固定策略”，Policy Improvement 负责“按价值改进策略”，两者交替就是 GPI。第 3 章会把这里的精确 Bellman backup 换成采样 backup。

2.4 动态规划适用前提

动态规划类方法通常假设环境模型已知：

$$P(s' | s, a), R(s, a, s')$$

也就是说，agent 知道每个动作会以什么概率转移到什么状态，并得到什么 reward。

这在真实问题中经常不成立，但 DP 仍然重要，因为它给出了强化学习算法的理论原型。后面的 TD、Q-learning、DQN 都可以看成是在没有完整模型时，用采样近似 Bellman backup。

2.5 Policy Iteration 路线：先评估再改进

2.5.1 Policy Evaluation

Policy Evaluation 的目标：给定一个策略 π ，计算它的价值函数 $V_\pi(s)$ 。

Bellman expectation equation：

$$\begin{aligned} V_\pi(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_\pi(s')] \end{aligned}$$

迭代更新：

$$\begin{aligned} V_{k+1}(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

直觉：

1. 初始化所有状态价值。
2. 使用 Bellman expectation backup 更新每个状态。
3. 重复直到价值函数收敛。

伪代码：

```
initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = sum_a pi(a|s) sum_s' P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold
```

2.5.2 Policy Improvement

有了 $V_\pi(s)$ 之后，可以通过贪心方式改进策略：

$$\begin{aligned} \pi_{new}(s) &= \arg \max_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_\pi(s')] \end{aligned}$$

直觉：

如果在某个状态下，选择动作 a 后的期望回报比当前策略更好，就应该把策略改成选择 a 。

Policy Improvement Theorem：

如果对所有状态都有：

$$Q_{\pi}(s, \pi_{new}(s)) \geq V_{\pi}(s)$$

那么新策略不差于旧策略：

$$V_{\pi_{new}}(s) \geq V_{\pi}(s)$$

2.5.3 Policy Iteration

Policy Iteration 在两个步骤之间交替：

1. Policy Evaluation：评估当前策略。
2. Policy Improvement：根据价值函数改进策略。

流程：

```
initialize pi randomly
repeat:
    V_pi = policy_evaluation(pi)
    pi_new = greedy_policy(V_pi)
    if pi_new == pi:
        stop
    pi = pi_new
```

它最终会收敛到最优策略 π^* 。

面试表达：

Policy Iteration 是 evaluation 和 improvement 的交替过程。先精确或近似评估当前策略，再对价值函数做贪心改进。这个过程可以看成广义策略迭代的一种具体形式。

2.6 Value Iteration 路线：直接做最优 backup

2.6.1 Value Iteration

Value Iteration 直接使用 Bellman optimality backup：

$$\begin{aligned} V_{k+1}(s) \\ &= \max_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

和 Policy Iteration 的区别：

- Policy Iteration 每轮先完整或近似评估一个策略，再改进策略。
- Value Iteration 把 evaluation 和 improvement 合在一次 backup 中。
- Value Iteration 每次更新都直接对动作取 max。

最后从 V^* 导出策略：

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')]$$

伪代码：

```
initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = max_a sum_{s'} P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold
derive pi from V
```

2.7 GPI 框架与 DP 局限

2.7.1 Generalized Policy Iteration

Generalized Policy Iteration，简称 GPI，指的是 policy evaluation 和 policy improvement 相互作用的一大类方法。

核心思想：

- evaluation 让 value function 更接近当前策略的真实价值。
- improvement 让 policy 根据当前 value function 变得更贪心。
- 两者交替推进，最终趋向最优策略和最优价值函数。

很多 RL 算法都可以放进这个框架：

- Policy Iteration：完整 evaluation + 贪心 improvement。
- Value Iteration：每一步都做截断 evaluation + improvement。
- SARSA：用同一套采样策略同时做 evaluation 和 improvement；第 3 章会把这类方法称为 on-policy。
- Q-learning：用采样逼近 optimal Bellman backup。
- Actor-Critic：critic 做 evaluation，actor 做 improvement。

2.7.2 DP 的局限性

动态规划方法有两个主要限制：

1. 需要已知完整环境模型 P 和 R 。
2. 状态空间和动作空间大时计算成本很高。

这也是为什么后续要学习 model-free RL：

- Monte Carlo 不需要环境模型，但要等 episode 结束。
- TD 不需要模型，也不需要等 episode 结束。
- Deep RL 用神经网络处理大规模状态空间。

2.8 本章例子：小型 GridWorld 上的 DP

假设一个 3x3 GridWorld：

- 右下角是终点，reward 为 +10。
- 撞墙留在原地，reward 为 -1。
- 普通移动 reward 为 -0.1。

- 每个动作都是确定性的。

2.8.1 Policy Evaluation 例子

如果当前策略是在每个格子随机选择上下左右，那么 policy evaluation 做的是：

固定这个随机策略，计算每个格子的长期价值。

直觉结果：

- 靠近终点的格子价值更高。
- 靠近墙角但远离终点的格子价值较低。
- 如果随机策略经常撞墙，撞墙附近的价值会被拉低。

这时算法只是在回答：“如果我一直按这个随机策略走，每个格子平均值多少钱？”

2.8.2 Policy Improvement 例子

有了 $V_{\pi}(s)$ 后，在某个格子比较四个动作：

向上：下一格价值 1.2
 向下：下一格价值 4.8
 向左：撞墙，价值 -1.0
 向右：下一格价值 3.5

policy improvement 会选择“向下”，因为它的一步 reward 加未来价值最大。

2.8.3 Value Iteration 例子

value iteration 不等完整 policy evaluation 收敛，而是每轮直接做：

每个格子都看四个动作，选 Bellman backup 最大的那个。

所以它更像“边估值，边贪心改策略”。在小 GridWorld 上，这会逐渐形成指向终点的箭头场。

面试表达：

DP 的例子一定要强调“已知模型”。GridWorld 里我知道每个动作会到哪个格子，所以能对下一状态做 Bellman backup。到了真实环境里不知道转移模型，就要转向 MC/TD。

2.9 本章面试高频问题

2.9.1 Policy Evaluation 是什么？

给定策略 π ，根据 Bellman expectation equation 计算该策略下每个状态的价值 $V_{\pi}(s)$ 。

2.9.2 Policy Iteration 和 Value Iteration 的区别？

Policy Iteration 交替进行策略评估和策略改进。Value Iteration 直接使用 Bellman optimality backup，把评估和改进合并在一次 max backup 中。

2.9.3 为什么 DP 方法在真实问题中受限？

因为它通常要求环境模型已知，即知道完整的状态转移概率和奖励函数。而真实环境中模型往往未知，且状态动作空间可能极大。

2.9.4 Value Iteration 为什么能得到最优策略？

它反复应用 Bellman optimality operator。在折扣 MDP 中，该 operator 是 contraction mapping，因此价值函数会收敛到唯一的最优价值函数 V^* ，再由 V^* 导出最优策略。

2.9.5 GPI 是什么？

GPI 是 policy evaluation 和 policy improvement 交替推进的通用思想。很多强化学习算法都可以看成不同形式的 GPI。

2.10 本章练习

1. 手写 Policy Evaluation 的 Bellman update。
2. 对比 Policy Iteration 和 Value Iteration 的伪代码。
3. 解释为什么 Value Iteration 中可以不完整评估一个策略。
4. 思考：如果状态空间有一亿个状态，表格型 DP 为什么不可行？

▼ 参考答案（点击展开）

1. **Policy Evaluation 更新**： $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$ 。反复对所有状态执行该 Bellman expectation backup，直到变化足够小。
2. **两种算法的伪代码差异**：Policy Iteration 的外层循环是“把当前 policy 评估到近似收敛，再做一次 greedy improvement”；Value Iteration 每轮只做一次 optimality backup： $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R + \gamma V_k(s')]$ ，收敛后再从 V 提取 greedy policy。
3. **为什么不必完整评估**：策略评估和策略改进可以交错进行。Bellman optimality operator 是 contraction；只要持续做 optimality backup，价值估计就会向 V^* 收敛。Value Iteration 等价于把“有限次评估”和“立即改进”合并为一步。
4. **一亿状态为什么不可行**：至少要存储一亿个 value；每次 sweep 还要遍历每个 state、action 和可能的 next state，并要求完整的 P 、 R 模型。内存、计算量和模型获取成本都不可接受，而且无法对未枚举状态泛化，因此需要采样和函数逼近。

第 3 章：MC（Monte Carlo，蒙特卡洛）、TD（Temporal Difference，时序差分）、SARSA 与 Q-learning

3.1 本章符号说明

符号/缩写	English	中文	含义
MC	Monte Carlo	蒙特卡洛	用完整 episode 的真实 return 估计价值。
TD	Temporal Difference	时序差分	用 bootstrap target 在线更新价值估计。
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
MDP	Markov Decision Process	马尔可夫决策过程	强化学习中描述状态、动作、转移、奖励和折扣的数学框架。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	更新公式使用的五元组名称。
$S_t / A_t / R_{t+1}$	State / Action / Reward	状态 / 动作 / 奖励	一步交互中的随机变量。
(s, a, r, s')	Sampled Transition	采样转移	当前状态、实际动作、即时奖励和下一状态的一次具体交互样本。
$\pi(a s)$	Policy	策略	在状态 s 选择动作 a 的概率；下标 π 表示“在该策略下”。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 到 episode 结束的累计收益。
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值估计。
$Q(s, a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值估计。
δ_t	TD Error	时序差分误差	bootstrap target 和当前估计之间的差。
α	Learning Rate	学习率	控制每次更新幅度。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
ϵ	Epsilon	探索概率	epsilon-greedy 中随机探索的概率。
greedy action	Greedy Action	贪心动作	当前估计下 $Q(s, a)$ 最大的动作。
behavior policy	Behavior Policy	行为策略	实际负责采样数据、和环境交互的策略。
target policy	Target Policy	目标策略	算法真正想评估或优化的策略。
on-policy	On-policy Learning	同策略学习	behavior policy 和 target policy 是同一个策略。
off-policy	Off-policy Learning	异策略学习	用 behavior policy 的数据学习另一个 target policy。
RL	Reinforcement Learning	强化学习	本章讨论 model-free RL 的估值和控制方法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	on-policy Monte Carlo policy gradient 算法。

符号/缩写	English	中文	含义
DQN	Deep Q-Network	深度 Q 网络	off-policy value-based deep RL 算法。
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	DDPG 的稳定化改进。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 off-policy actor-critic 算法。
PPO	Proximal Policy Optimization	近端策略优化	on-policy actor-critic 算法。
A2C	Advantage Actor-Critic	优势 Actor-Critic	on-policy actor-critic 算法。

3.2 本章目标

本章进入 model-free RL。你需要掌握：

- Monte Carlo 方法。
- Temporal Difference Learning。
- MC 和 TD 的 bias/variance 区别。
- SARSA。
- Q-learning。
- on-policy 与 off-policy。

3.3 本章主线

第 2 章的 DP 依赖完整环境模型。本章去掉这个假设，只允许 agent 从真实交互中拿到样本。

主线分两步：

1. Prediction：先学会评估一个固定策略的价值。MC 用完整 return 做 target，TD 用一步 reward 加下一状态估计做 bootstrap target。
2. Control：再把价值估计用于改进策略。为了直接比较动作，需要从 $V(s)$ 转到 $Q(s,a)$ 。SARSA 和 Q-learning 不是突然出现的新主题，而是 TD 思想在 action-value control 上的两个版本：SARSA 是 on-policy TD control，Q-learning 是 off-policy TD control。

所以本章的逻辑是：

```
DP exact backup
-> model-free sample backup
-> MC/TD prediction
-> TD control with Q(s,a)
-> SARSA vs Q-learning
```

3.4 Model-free Prediction 的出发点

3.4.1 Model-free 的含义

Model-free 指的是不需要显式知道环境转移模型：

$$P(s' | s, a), R(s, a, s')$$

Agent 只通过和环境交互得到样本：

```
(s, a, r, s')
```

然后用这些样本估计价值函数或优化策略。

3.4.2 统一出发点：价值函数是一个期望

MC 和 TD 都不是凭空来的。它们解决的是同一个问题：价值函数定义里有一个期望，但 model-free 场景下我们没法直接算这个期望。

先看状态价值函数：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

其中 return 是：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

这句话的含义是：

状态 s 的价值，就是在策略 π 下从 s 出发，未来累计折扣 reward 的平均水平。

问题在于这个平均值不能直接看到。我们每次和环境交互，只能看到一条随机轨迹：

$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \dots$

因此从价值定义到算法，核心只有一步：

用样本估计期望。

不同方法的区别在于“用什么样的样本 target”：

方法	可用信息	target 怎么来	代价
DP	知道 $P(s' s,a)$ 和 $R(s,a,s')$	对所有下一状态求精确期望	需要环境模型
MC	不知道模型，但能等 episode 结束	用完整真实 return G_t	等得久，方差大
TD	不知道模型，也不想等结束	用一步 reward 加下一状态估计 $R_{t+1} + \gamma V(S_{t+1})$	有 bootstrap bias

所以 MC 和 TD 可以理解成 DP 的两个 model-free 版本：

- MC：不用模型，也不用下一状态估计，直接等真实 return 出来。
- TD：不用模型，也不等真实 return，用一步采样加当前价值函数估计。

3.5 MC Prediction

3.5.1 Monte Carlo 方法

Monte Carlo 方法用完整 episode 的实际 return 来估计价值。

对于状态价值：

$V(s) \leftarrow \text{average of returns observed after visiting } s$

增量形式：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

其中 G_t 是从时间 t 到 episode 结束的真实累计 return。

特点：

- 不需要环境模型。
- 不需要 bootstrap。
- 必须等 episode 结束才能计算 return。
- 方差通常较大。
- 估计是无偏的，前提是采样足够充分。

First-visit MC：

每个 episode 中，一个状态第一次出现时才用它更新。

Every-visit MC：

每个 episode 中，一个状态每次出现都可以用来更新。

3.5.2 MC 的推理过程

MC 的出发点是价值函数定义：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

这个式子可以拆成一句统计学问题：

我想估计一个随机变量 G_t 在条件 $S_t = s$ 下的均值。

如果我们多次访问同一个状态 s ，就会得到很多个 return 样本：

$$G_t^{(1)}, G_t^{(2)}, \dots, G_t^{(n)}$$

用样本均值估计期望：

$$V_{\pi}(s) \approx 1 / n \sum_{i=1}^n G_t^{(i)}$$

这就是 MC prediction 的核心。它不是先写更新公式，而是先把“价值函数”看成“return 的均值”。

如果每次都重新算平均值，写法是：

$$V_n(s) = 1 / n \sum_{i=1}^n G_t^{(i)}$$

为了在线更新，可以把样本均值写成递推形式：

$$V_n(s) = V_{n-1}(s) + 1 / n [G_t^{(n)} - V_{n-1}(s)]$$

如果不用严格的 $1/n$ ，而用固定学习率 α ，就得到常见写法：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

这里的结构很重要：

```
new estimate = old estimate + step size * (target - old estimate)
```

对 MC 来说：


```
target = G_t
error = G_t - V(s)
```

因此 MC 的完整逻辑是：

1. 从状态 s 出发采样一条 episode。
2. 等 episode 结束后，计算真实 return G_t 。
3. 把 G_t 当作这次对 $V_\pi(s)$ 的观测样本。
4. 用样本平均或增量平均更新 $V(s)$ 。

3.5.2.1 为什么 MC target 是无偏的

$$\mathbb{E}_\pi[G_t | S_t=s] = V_\pi(s)$$

这说明在策略 π 不变、采样充分时， G_t 的期望就是目标价值。MC 用真实 return 做 target，target 本身不依赖当前估计 $V(s)$ ，因此没有 bootstrap bias。

但它的方差可能很大。原因是 G_t 包含整个未来：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

从 $S_t=s$ 开始，后面每一步动作、转移和 reward 都会引入随机性。轨迹越长，return 波动越大。

举个直觉例子：

- 同一个状态 s ，这次 episode 后面走到高奖励区域， G_t 很大。
- 下次 episode 后面因为探索动作走偏， G_t 很小。
- MC 会忠实记录这些完整结果，所以 target 真实，但抖动大。

3.5.2.2 First-visit 和 Every-visit 的区别

如果一个 episode 里同一个状态 s 出现多次：

```
s -> ... -> s -> ... -> terminal
```

First-visit MC 只用第一次访问后的 return 更新。Every-visit MC 每次访问都更新。

面试里一般不需要展开收敛证明，核心记住：

- 两者都在用 return 样本估计期望。
- first-visit 更贴近“每条 episode 给一个独立样本”的直觉。
- every-visit 使用更多样本，但同一 episode 内样本相关性更强。

3.6 TD Prediction

3.6.1 从 MC 过渡到 TD：为什么不想等到 episode 结束

MC 的问题不是思路错，而是工程上有明显限制：

1. 很多任务 episode 很长，等完整 return 才更新太慢。
2. continuing task 没有自然终止， G_t 不容易等出来。
3. 长轨迹 return 方差大，学习曲线抖动。
4. 如果中间某一步已经明显好或坏，MC 也要等结束后才把信号传回前面状态。

TD 的动机就是解决这个问题：

能不能不等完整 G_t ，只走一步就更新？

答案来自 return 的递归分解：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

如果我们知道真实的 G_{t+1} ，那还是要等 episode 结束。但价值函数本身就是对 G_{t+1} 的估计：

$$V_{\pi}(S_{t+1}) = \mathbb{E}_{\pi}[G_{t+1} | S_{t+1}]$$

于是 TD 做了一个替换：

$$G_{t+1} \approx V(S_{t+1})$$

代回去：

$$G_t \approx R_{t+1} + \gamma V(S_{t+1})$$

这就是 TD target 的来源。

3.6.2 TD Learning

Temporal Difference Learning 结合了 Monte Carlo 和 Dynamic Programming 的思想。

TD(0) 更新：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD target：

$$R_{t+1} + \gamma V(S_{t+1})$$

TD error：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

特点：

- 不需要环境模型。
- 不需要等 episode 结束。
- 使用 bootstrap，用当前估计 $V(S_{t+1})$ 更新当前估计。
- 通常方差低于 MC，但可能有 bias。

3.6.3 TD 的推理过程

TD 可以从两个等价角度推出来。

3.6.3.1 角度一：从 Bellman expectation 推出

Bellman expectation equation 是：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

如果有环境模型，DP 可以把这个期望展开成对所有动作和下一状态求和：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

但 model-free 场景下，我们不知道 $P(s'|s,a)$ ，也不知道完整的转移分布。我们只有一次真实采样：

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

于是用这一次样本近似 Bellman 期望里的随机变量：

$$R_{t+1} + \gamma V(S_{t+1})$$

这就是 stochastic approximation，即用随机样本近似期望更新。

3.6.3.2 角度二：从 return 递归推出

return 有递归形式：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

MC 使用真实 G_t ，所以必须等未来全部 reward 出来。TD 不等 G_{t+1} ，而是用当前估计替代：

$$G_{t+1} \approx V(S_{t+1})$$

所以 TD target 变成：

$$R_{t+1} + \gamma V(S_{t+1})$$

这一步叫 bootstrap，因为 target 里用了模型自己当前的估计。

3.6.3.3 TD error 的含义

有了 target 后，再用“target 减当前估计”构造误差：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

然后按误差方向更新：

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

如果 $\delta_t > 0$ ，说明当前 $V(S_t)$ 低估了，因为一步 reward 加下一状态价值比当前估计更高。

如果 $\delta_t < 0$ ，说明当前 $V(S_t)$ 高估了。

3.6.3.4 TD 为什么方差低但有 bias

TD target 只包含一步真实 reward：

$$R_{t+1}$$

后面的长期未来不再用真实随机轨迹，而用估计值：

$$V(S_{t+1})$$

因此 TD target 少受后续随机动作和随机转移影响，方差通常低于 MC。

但 $V(S_{t+1})$ 可能本身不准。target 依赖当前估计，所以会引入 bootstrap bias。

一句话：

MC 的 target 真实但晚、噪声大；TD 的 target 来得早、噪声小，但里面混入了当前价值函数的估计误差。

3.7 Prediction 方法对比与例子

3.7.1 MC vs TD

维度	Monte Carlo	TD
是否需要模型	不需要	不需要
是否需要完整 episode	需要	不需要
是否 bootstrap	否	是
目标	真实 return G_t	$r + \gamma V(s')$
方差	较高	较低
偏差	较低	有 bootstrap bias
适用场景	episodic task	episodic 或 continuing task
更新时机	episode 结束后	每一步之后
信息传播	等完整 return 再回传	reward 可以一步步向前传播

面试表达：

MC 用完整轨迹的真实 return 做 target，不 bootstrap，所以 bias 小但 variance 大。TD 用一步 reward 加下一状态的估计值做 target，能在线更新，variance 小，但因为 target 依赖当前估计，所以会引入 bias。

3.7.2 一个最小数值例子

假设当前估计：

$$V(s)=5, V(s')=8, \gamma=0.9, \alpha=0.1$$

这一步从 s 到 s' ，拿到 reward：

$$R_{t+1}=2$$

TD target 是：

$$2+0.9 \times 8=9.2$$

TD error 是：

$$\delta=9.2-5=4.2$$

更新后：

$$V(s) \leftarrow 5+0.1 \times 4.2=5.42$$

这里没有等 episode 结束。TD 只用了一步真实 reward 和下一状态当前估计。

如果用 MC，则必须等整条 episode 结束。假设最后算出来真实 return 是：

$$G_t=12$$

MC 更新是：

$$V(s) \leftarrow 5+0.1 \times (12-5)=5.7$$

两者差异就在 target:

- MC target 是完整真实 return: 12。
- TD target 是一步 bootstrap: 9.2。

这个例子说明: TD 不一定比 MC 更准, 它只是更早、更低方差地更新。

3.7.3 本章例子: 什么时候用 MC, 什么时候用 TD

3.7.3.1 例子 1: Blackjack 适合解释 MC

Blackjack 是一个 episodic task:

- 一局牌开始。
- 玩家要牌或停牌。
- 最后赢、输或平局。
- episode 很短, 结束明确。

这时 MC 很自然。比如玩家在某个状态 s 做决策后, 等这一局结束就能知道真实结果:

```
赢了: G_t = +1
输了: G_t = -1
平局: G_t = 0
```

多玩很多局后, 访问同一个状态得到很多 return 样本, 用平均值估计这个状态价值。

面试讲法:

Blackjack 适合 MC, 因为 episode 短、终止清晰、完整 return 容易拿到。MC 不需要知道牌堆转移概率, 只要反复采样完整对局。

3.7.3.2 例子 2: 在线广告更适合解释 TD

假设系统每次给用户推荐一个广告, 用户状态会持续变化, 没有明确“游戏结束”。

如果用 MC, 可能要等用户一整天甚至一周的长期收益才更新, 这太慢。

TD 可以每一步更新:

```
当前状态: 用户刚打开 App
动作: 展示广告 A
即时 reward: 点击 +1, 没有点击 0
下一状态: 用户继续浏览或离开
TD target = 即时 reward + 下一状态价值估计
```

这说明 TD 的优势:

- 不需要等完整长期结果。
- 每次交互后都能更新。
- 适合 continuing task 或长 episode 任务。

3.8 从 Prediction 到 TD Control

3.8.1 从 Prediction 到 Control: 为什么接 SARSA 和 Q-learning

到这里为止, MC 和 TD 主要解决的是 prediction:

给定一个策略 π ，估计它的价值。

但强化学习最终要做 control：

找到一个更好的策略。

两者的区别是：

任务	学什么	策略是否固定	典型问题
V prediction	$V_{\pi}(s)$	固定	当前策略下，状态 s 值多少钱
Q prediction	$Q_{\pi}(s,a)$	固定	当前策略下，在 s 做 a 值多少钱
control	$Q(s,a)$ 并改进 π	不固定	在 s 应该选哪个动作

为什么 control 常从 $Q(s,a)$ 入手？

如果已知环境模型，可以用 $V(s)$ 加上 $P(s'/s,a)$ 和 $R(s,a,s')$ 做一步 lookahead：

$$\sum_s P(s'/s,a) [R(s,a,s') + \gamma V(s')]$$

但 model-free 场景下，模型未知，agent 不能直接对所有下一状态求和。此时更自然的是直接学习每个动作的长期价值：

$$Q(s,a)$$

有了 $Q(s,a)$ ，策略改进就很直接：

choose action with larger $Q(s, a)$

所以 SARSA 和 Q-learning 的位置应该理解为：

```
TD prediction learns  $V_{\pi}(s)$ 
TD control learns  $Q(s,a)$  and improves policy
SARSA and Q-learning are two TD control algorithms
```

它们都沿用 TD 的更新骨架：

```
new estimate = old estimate + alpha * (TD target - old estimate)
```

区别只在 TD target 怎么定义：

- SARSA：target 跟随实际行为策略采样到的下一个动作。
- Q-learning：target 假设下一步执行 greedy action。

3.8.2 先定义：greedy、epsilon-greedy、on-policy、off-policy

下面马上要讲 SARSA 和 Q-learning。这里先把几个词定义清楚，否则后面的“因此是 on-policy / off-policy”没有意义。

3.8.2.1 Greedy action / greedy policy

Greedy action 指当前 $Q(s,a)$ 估计下价值最大的动作：

$$a_{\text{greedy}} = \arg \max_a Q(s,a)$$

Greedy policy 指每个状态都选择当前看起来最好的动作：

```
choose argmax_a Q(s, a)
```

它的问题是：如果早期 Q 估计错了，agent 可能一直选错动作，永远不探索其他动作。

3.8.2.2 Epsilon-greedy behavior policy

Epsilon-greedy 是一种带探索的行为策略。 ϵ 表示随机探索的概率：

```
with probability epsilon:
    choose a random action
otherwise:
    choose argmax_a Q(s, a)
```

例如 $\epsilon=0.1$ ：

- 10% 概率随机选动作，用来探索。
- 90% 概率选当前 $Q(s,a)$ 最大的动作，用来利用当前知识。

所以 epsilon-greedy 不是一个新算法，而是一种“采样数据时怎么选动作”的策略。

3.8.2.3 Behavior policy 和 target policy

在 control 问题里要分清两个策略：

名词	中文	含义
behavior policy	行为策略	实际和环境交互、产生样本的策略。
target policy	目标策略	算法真正想评估或优化的策略。

如果 agent 用 epsilon-greedy 和环境交互，那么 behavior policy 就是 epsilon-greedy。

如果算法更新时假设“未来都选 Q 最大的动作”，那么 target policy 就是 greedy policy。

3.8.2.4 On-policy 和 off-policy

On-policy，中文可以叫同策略学习：

用当前实际采样的 behavior policy，同时作为学习目标。

也就是：

$$\pi_{behavior} = \pi_{target}$$

如果 behavior policy 是 epsilon-greedy，on-policy 算法学到的就是“我继续按这套 epsilon-greedy 策略行动时，长期价值是多少”。

Off-policy，中文可以叫异策略学习：

用一个 behavior policy 采样数据，但学习另一个 target policy。

也就是：

$$\pi_{behavior} \neq \pi_{target}$$

例如：用 epsilon-greedy 采样，保持探索；但更新 target 时假设下一步走 greedy action。这个就是 off-policy。

一句话判断：

判断问题	结论
target 用实际采样到的下一个动作 A_{t+1} 吗?	通常是 on-policy。
target 跳过实际动作，直接用 $\max_a Q(s',a)$ 吗?	通常是 off-policy。

3.9 TD Control 算法

3.9.1 TD Control 1: SARSA

现在再看 SARSA。SARSA 是 on-policy TD control 方法。

这里的 on-policy 含义是：SARSA 用当前行为策略采样数据，也学习这个当前行为策略本身的价值。如果行为策略是 epsilon-greedy，那么 SARSA 学的就是“继续按这套 epsilon-greedy 策略行动时，长期回报是多少”。

SARSA 名字来自一次更新用到的五元组：

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$$

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

关键点：

- A_{t+1} 是 behavior policy 在下一状态真实采样出来的动作。
- 如果 behavior policy 是 epsilon-greedy，那么 A_{t+1} 可能是 greedy action，也可能是随机探索动作。
- SARSA 的 target 使用这个真实采样的 A_{t+1} ，所以它学习的 target policy 就是当前 behavior policy。
- 因此 SARSA 是 on-policy：采样用谁，学习目标就是谁。

直觉：

SARSA 学的是“我按照当前这套带探索的策略行动，长期回报会是多少”。

3.9.2 SARSA 的 target 为什么是 $Q(s',a')$

对 action-value 来说，Bellman expectation 可以理解成：

$$Q_{\pi}(s,a) = \mathbb{E}[R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1})]$$

SARSA 在采样时真的执行了下一步动作 A_{t+1} 。如果当前 behavior policy 是 epsilon-greedy，那么 A_{t+1} 可能是 greedy action，也可能是随机探索动作。

所以 SARSA 的 target 是：

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

它把“探索可能造成的后果”也学进去。这就是为什么在 Cliff Walking 里 SARSA 更保守：它知道自己未来还可能因为 ϵ 探索而掉下去。

3.9.3 TD Control 2: Q-learning

Q-learning 是 off-policy TD control 方法。

这里的 off-policy 含义是：Q-learning 可以用 epsilon-greedy 这种 behavior policy 去探索和采样，但它更新时学习的是 greedy target policy，也就是“未来都选当前 Q 最大动作”的策略。

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

关键点：

- 行为时可以用 epsilon-greedy 探索。
- 更新 target 时直接用 $\max_a Q(s', a)$ 。
- 学习目标是 greedy policy 的价值，而不完全等于行为策略。
- 因此 Q-learning 是 off-policy：采样策略可以带探索，但学习目标是 greedy policy。

直觉：

Q-learning 学的是“如果未来都选当前估计下最优的动作，长期回报会是多少”。

3.9.4 Q-learning 的 target 为什么用 max

Q-learning 对应的是 Bellman optimality 思路：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a')]$$

model-free 场景下，把期望用一次 transition 近似，就得到 target：

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a')$$

这里要分清两件事：

- 行为时：可以用 epsilon-greedy 收集数据，保留探索。
- 学习时：target 假设未来执行 greedy action。

所以 Q-learning 是 off-policy。它不是在评估“实际带探索的行为策略”，而是在用探索数据学习一个 greedy target policy 的价值。

面试里可以这样推：

SARSA 从 Bellman expectation 来，下一动作就是行为策略实际采样的 a' ；Q-learning 从 Bellman optimality 来，下一动作取当前 Q 最大的动作。因此 SARSA on-policy，Q-learning off-policy。

3.9.5 SARSA vs Q-learning

维度	SARSA	Q-learning
策略类型	on-policy	off-policy
target	$r + \gamma Q(s', a')$	$r + \gamma \max_a Q(s', a)$
是否考虑探索动作影响	是	否
风格	更保守	更贪心
学习目标	当前行为策略	最优 greedy 策略

3.9.6 本章例子：Cliff Walking 解释 SARSA 和 Q-learning

Cliff Walking 中，起点到终点的最短路径贴着悬崖走，但 epsilon-greedy 探索可能让 agent 随机走错一步掉下去。

SARSA 学当前带探索行为策略的价值，所以它会把“未来仍可能探索并掉下悬崖”的风险计入 target：

```
r + gamma * Q(s', actual a')
```

因此 SARSA 往往学到更保守、更远离悬崖的安全路径。

Q-learning 学 greedy target policy 的价值。虽然行为时仍用 epsilon-greedy 探索，但更新 target 假设下一步会选择当前最优动作：

```
r + gamma * max_a Q(s', a)
```

因此 Q-learning 更容易学到贴近悬崖的最短路径。

这个例子对应的逻辑是：

- SARSA 会考虑 epsilon-greedy 探索可能掉下悬崖，因此倾向选择更安全路径。
- Q-learning 更新时假设未来总能选最优动作，因此更倾向学习贴近悬崖的最短路径。

面试讲法：

SARSA 和 Q-learning 都是 TD control。SARSA 的 target 来自行为策略实际采样到的下一动作，所以是 on-policy；Q-learning 的 target 来自 greedy max，所以是 off-policy。Cliff Walking 中 SARSA 更保守，Q-learning 更贪心。

3.9.7 On-policy vs Off-policy 总结

前面已经先定义过一次，这里只做总结。

On-policy：

学习和评估的是当前实际采样的行为策略。

$$\pi_{behavior} = \pi_{target}$$

例如：

- SARSA
- REINFORCE
- A2C
- PPO

Off-policy：

用一种行为策略收集数据，但学习另一种目标策略。

$$\pi_{behavior} \neq \pi_{target}$$

例如：

- Q-learning
- DQN
- DDPG
- TD3
- SAC

off-policy 的优势：

- 可以复用历史数据。
- sample efficiency 通常更高。
- 可以从 replay buffer 学习。

off-policy 的难点:

- 分布不匹配。
- 重要性采样或 value correction 可能带来高方差。
- 结合函数逼近和 bootstrap 时可能不稳定。

3.10 本章面试高频问题

3.10.1 MC 和 TD 的区别?

MC 用完整 episode 的真实 return 更新, 不 bootstrap, bias 小但 variance 大。TD 用 $r + \gamma V(s')$ 这种一步 bootstrap target 更新, 可以在线学习, variance 小但有 bias。

3.10.2 SARSA 和 Q-learning 的区别?

SARSA 是 on-policy, target 使用实际采样到的下一个动作 a' 。Q-learning 是 off-policy, target 使用 $\max_a Q(s', a)$, 学习 greedy target policy。

3.10.3 SARSA/Q-learning 和 TD 有什么关系?

SARSA 和 Q-learning 都是 TD control。TD prediction 更新 $V(s)$, 用于评估固定策略; SARSA/Q-learning 更新 $Q(s, a)$, 用于边估值边改进策略。两者都使用一步 bootstrap target, 只是 SARSA 用实际采样的 a' , Q-learning 用 greedy max。

3.10.4 为什么 Q-learning 是 off-policy?

因为采样数据可以来自 epsilon-greedy 行为策略, 但更新目标是假设下一步选择 greedy action 的价值, 即 target policy 和 behavior policy 不同。

3.10.5 TD error 是什么?

TD error 是当前价值估计和 bootstrap target 之间的差:

$$\delta = r + \gamma V(s') - V(s)$$

它衡量当前估计需要被修正的方向和大小。

3.10.6 为什么 TD 可以不等 episode 结束?

因为 TD target 只需要一步 reward 和下一状态的当前价值估计, 不需要完整轨迹 return。

3.11 本章练习

1. 手写 SARSA 和 Q-learning 的更新公式。
2. 解释为什么 SARSA 在 cliff walking 中更保守。
3. 写一个 tabular Q-learning 的伪代码。
4. 用 bias/variance 角度比较 MC 和 TD。

▼ 参考答案 (点击展开)

1. **更新公式:** SARSA 使用 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$; Q-learning 使用 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$ 。终止状态的下一状态价值按 0 处理。
2. **Cliff Walking 中更保守的原因:** SARSA 的 target 使用 epsilon-greedy policy 实际可能执行的 A_{t+1} , 所以会把“沿悬崖走时探索动作可能掉下去”的风险计入价值。Q-learning 的 target 假设下一步总选 greedy action, 学习的是不含探索失误的最优路径, 因此更贴近悬崖。
3. **Tabular Q-learning 伪代码:**

```

initialize Q(s, a)
for each episode:
    initialize state s
    while s is not terminal:
        choose a from an epsilon-greedy behavior policy based on Q
        execute a, observe reward r and next state s'
        y = r                                     if s' is terminal
            r + gamma * max_a' Q(s', a') otherwise
        Q(s, a) = Q(s, a) + alpha * (y - Q(s, a))
        s = s'

```

4. **Bias/variance** 对比: MC 用完整真实 return, 给定 on-policy 采样时 target 通常无 bootstrap bias, 但一条轨迹中许多随机 reward 会累积, variance 高。TD 用 $R_{t+1} + \gamma V(S_{t+1})$, 依赖当前不准确的估计而有 bootstrap bias, 但只引入一步随机性, variance 较低, 并且能在线更新。

第 4 章：探索与利用、Bandit、UCB (Upper Confidence Bound, 置信上界) 与 Thompson Sampling

4.1 本章符号说明

符号/缩写	English	中文	含义
UCB	Upper Confidence Bound	置信上界	用价值估计加不确定性 bonus 做探索。
K	Number of Arms	拉杆数量 / 动作数量	K-armed bandit 中候选动作的数量。
A_t	Action at time t	时刻 t 的动作	bandit 或 RL 中当前选择的动作。
R_t	Reward at time t	时刻 t 的奖励	本轮选择动作后获得的奖励。
$Q_t(a)$	Action-value Estimate	动作价值估计	到时刻 t 为止动作 a 的平均 reward 估计。
$N_t(a)$	Action Count	动作选择次数	到时刻 t 为止动作 a 被选过多少次。
ϵ	Exploration Probability	探索概率	epsilon-greedy 中随机选择动作的概率。
$P(a s)$	Action Sampling Probability	动作采样概率	softmax exploration 在状态 s 选择动作 a 的概率。
μ_a / μ^*	Mean Reward / Best Mean Reward	平均奖励 / 最优平均奖励	动作 a 或最优动作的期望奖励。
$Regret(T)$	Cumulative Regret	累计后悔值	到时间 T 为止因为没有总选最优动作损失的收益。
τ	Temperature	温度参数	softmax exploration 中控制分布平滑程度。
c	Exploration Coefficient	探索系数	UCB 中控制不确定性 bonus 强度的正数。
θ_a	Bernoulli Success Probability	Bernoulli 成功概率	Thompson Sampling 例子中动作 a 的未知真实点击率。
α_a / β_a	Beta Posterior Parameters	Beta 后验参数	分别累计动作 a 的成功和失败证据；不是学习率或熵系数。
RL	Reinforcement Learning	强化学习	探索与利用是 RL 的核心问题之一。
DQN	Deep Q-Network	深度 Q 网络	常用 epsilon-greedy 做探索的 value-based 算法。

4.2 本章目标

本章讨论强化学习中的 exploration-exploitation tradeoff。你需要掌握：

- 为什么需要探索，以及 exploration / exploitation 的矛盾。
- 基础采样策略：epsilon-greedy 和 softmax exploration。
- Bandit 为什么是研究探索的最小模型。
- Bandit 里的评价指标：regret。
- 利用不确定性的 bandit 采样策略：UCB 和 Thompson Sampling。
- 这些探索方法在完整 RL 里如何落到 behavior policy、NoisyNet 等机制上。

4.3 本章主线

第 3 章默认 agent 能持续拿到交互样本，但样本本身由策略决定。策略如果只选当前最优动作，可能永远发现不了更好动作；如果一直随机试，又会牺牲收益。

本章结构是：

1. 先定义 exploration-exploitation tradeoff。
2. 再讲只依赖当前价值估计的基础采样策略：epsilon-greedy 和 softmax。
3. 接着用 bandit 把探索问题单独抽出来，并引入 regret。
4. 然后讲 bandit 中利用不确定性的采样策略：UCB 和 Thompson Sampling。
5. 最后回到完整 RL，说明探索机制如何成为 behavior policy 或参数空间探索。

4.4 本章新增概念说明

先把本章会反复出现的探索名词说明清楚：

名词	中文	先记住什么
exploration	探索	尝试还不确定的动作，目的是发现潜在更优选择。
exploitation	利用	选择当前估计最好的动作，目的是获得已知收益。
epsilon-greedy	epsilon-greedy 策略	以 ϵ 概率随机探索，否则选择当前 Q 最大的动作。
greedy action	贪心动作	当前价值估计下最好的动作，即 $\arg \max_a Q(s,a)$ 。
softmax exploration	softmax 探索	按价值大小分配采样概率，价值越高越容易被选中。
temperature	温度参数	控制 softmax 分布平滑程度：温度高更随机，温度低更贪心。
bandit / arm	多臂老虎机 / 拉杆	只考虑动作和即时 reward、暂时不考虑状态转移的探索问题。
regret	后悔值	因为没总选最优动作而损失的累计收益。
UCB	置信上界	当前平均收益加不确定性 bonus，偏向探索不确定动作。
Thompson Sampling	Thompson 采样	从每个动作的后验分布采样，用概率匹配方式探索。
posterior	后验分布	看到历史数据后，对某个动作真实收益的概率估计。
NoisyNet	噪声网络	在网络参数中加入噪声，让探索发生在参数或策略层面。

4.5 探索问题的基本概念

4.5.1 探索与利用

强化学习中，agent 面临两个目标：

- exploitation：选择当前看起来最好的动作，最大化已知收益。
- exploration：尝试不确定的动作，发现潜在更好的选择。

如果只 exploitation，可能永远错过更优动作。

如果只 exploration，则无法稳定获得高收益。

面试表达：

Exploration-exploitation tradeoff 是 RL 的核心问题之一。Agent 需要在利用当前价值估计和探索未知动作之间平衡。探索不足会导致局部最优，探索过多会损害短期收益和训练稳定性。

这一节只是定义问题：agent 到底为什么不能一直选当前最优动作。下面的 epsilon-greedy、softmax、UCB、Thompson Sampling 才是不同的动作采样策略。

4.6 基础采样策略：只依赖当前价值估计

这一组方法只看当前 action-value estimate，不显式建模“这个估计有多不确定”。它们适合作为 RL 里的基础 behavior policy。

4.6.1 Epsilon-greedy

epsilon-greedy 是最常见的探索策略。

```
with probability epsilon:
    choose random action
otherwise:
    choose argmax_a Q(s, a)
```

优点：

- 简单。
- 容易实现。
- DQN 等算法常用。

缺点：

- 随机探索没有方向。
- 对所有非 greedy 动作一视同仁。
- 在大动作空间中效率较低。

常见工程做法：

```
epsilon starts high and decays over time
```

例如从 1.0 逐渐衰减到 0.05。

4.6.2 Softmax Exploration

softmax exploration 按照 action-value 的 softmax 概率采样动作：

$$P(a | s) = \exp(Q(s, a) / \tau) / \sum_b \exp(Q(s, b) / \tau)$$

其中 b 只是遍历全部候选动作的求和下标， $P(a|s)$ 是最终采样动作 a 的概率， τ 是 temperature：

- τ 大：分布更平，探索更多。
- τ 小：分布更尖，接近 greedy。

相比 epsilon-greedy，softmax 会更偏向价值较高的动作，而不是均匀随机探索。

缺点：

- 对 Q 值尺度敏感。
- 如果 Q 值估计不准，采样分布也会被误导。

4.7 Bandit 问题：把探索单独抽出来研究

Bandit 是研究 exploration-exploitation 的最小模型。先在 bandit 中理解“动作采样策略如何权衡收益和不确定性”，再回到完整 RL。

4.7.1 Multi-Armed Bandit

Bandit 是最简单的 sequential decision 问题：

- 只有动作，没有状态转移。
- 每个 action 对应一个未知 reward distribution。
- 目标是在不断试验中最大化累计 reward。

K-armed bandit：

```
choose action A_t in {1, ..., K}
receive reward R_t
```

它可以看成没有状态或只有单一状态的 RL 问题。

Bandit 在推荐系统、广告投放、A/B testing 中非常常见。

4.7.2 Bandit 和完整 RL 的关系

本章先讲 bandit，不是因为 bandit 比 RL 更重要，而是因为它把“探索”从复杂的状态转移里单独抽出来。

维度	Bandit	完整 RL
状态	通常没有状态，或只有单一状态	有状态 s
动作影响	主要影响当前 reward	同时影响当前 reward 和未来状态
学习目标	找到平均 reward 高的动作	找到长期 return 高的策略
探索难点	试哪个 arm	在什么状态试什么动作

所以 bandit 是完整 RL 的一个局部视角：先把“动作选择的不确定性”讲清楚，再回到有状态转移的长期决策问题。

4.7.3 Regret

Regret 衡量由于没有总是选择最优动作而损失的收益。

如果最优动作的期望奖励是 μ^* ，第 t 步选择动作的期望奖励是 μ_{a_t} ，则累计 regret：

$$Regret(T) = \sum_{t=1}^T (\mu^* - \mu_{a_t})$$

目标不是只最大化 reward，也可以表述为最小化 regret。

4.8 Bandit 采样策略：利用不确定性做探索

这一组策略不只是看当前平均收益，还会显式使用“不确定性”。UCB 用 confidence bonus 表达不确定性，Thompson Sampling 用 posterior sampling 表达不确定性。

4.8.1 UCB

Upper Confidence Bound，简称 UCB。

核心思想：

乐观面对不确定性。对不确定的动作给一个探索 bonus。

常见 UCB 形式：

$$A_t = \arg \max_a [Q_t(a) + c \sqrt{(\ln t / N_t(a))}]$$

其中：

- $Q_t(a)$: 动作 a 当前平均 reward 估计。
- $N_t(a)$: 动作 a 被选择次数。
- t : 总时间步。
- c : 控制探索强度。

解释：

- 如果动作被选得少, $N_t(a)$ 小, bonus 大。
- 如果动作被选得多, 不确定性下降, bonus 变小。
- UCB 会系统性探索不确定动作, 而不是随机探索。

4.8.2 Thompson Sampling

Thompson Sampling 是一种 Bayesian exploration 方法。

流程：

1. 为每个动作维护 reward 分布参数的 posterior。
2. 每轮从每个动作的 posterior 中采样一个可能的参数。
3. 选择采样后看起来最优的动作。
4. 根据实际 reward 更新 posterior。

4.8.2.1 具体例子：Beta-Bernoulli 广告点击

假设有 3 个广告候选, reward 是二值点击：

```
click = 1
no click = 0
```

每个广告的真实点击率未知, 记作 θ_a 。因为点击是 Bernoulli variable, 所以可以给每个广告维护一个 Beta posterior：

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

初始化时, 如果没有任何历史数据, 可以用均匀先验：

$$\alpha_a=1, \beta_a=1$$

含义是：一开始不知道广告好坏, 所有点击率都可能。

如果广告 a 被展示后：

- 用户点击了: $\alpha_a \leftarrow \alpha_a + 1$
- 用户没点击: $\beta_a \leftarrow \beta_a + 1$

因为 Beta 是 Bernoulli 的 conjugate prior, 所以这个更新可以理解为：

```
alpha = 1 + click_count
beta  = 1 + no_click_count
```

某一轮开始时, 假设 posterior 是：

广告	历史点击 / 未点击	Posterior	后验均值
A	7 / 3	Beta(8, 4)	0.67
B	2 / 2	Beta(3, 3)	0.50
C	1 / 5	Beta(2, 6)	0.25

如果只看均值，会选择 A。但 Thompson Sampling 不直接选均值，而是每轮从每个 posterior 里采样一个可能点击率：

```
sample theta_A ~ Beta(8, 4)  -> 0.62
sample theta_B ~ Beta(3, 3)  -> 0.78
sample theta_C ~ Beta(2, 6)  -> 0.20
```

这一轮采样结果里 B 最大，所以展示 B。

如果展示 B 后用户点击了：

```
Beta(3, 3) -> Beta(4, 3)
```

如果用户没点击：

```
Beta(3, 3) -> Beta(3, 4)
```

这个例子里，B 的后验均值低于 A，但因为 B 的历史样本少，posterior 更宽，偶尔会采样出很高的点击率，因此仍有机会被探索。A 的样本多，posterior 更窄，如果它确实好，就会更稳定地被选中。

伪代码：

```
for each arm a:
    alpha[a] = 1
    beta[a] = 1

for each round:
    for each arm a:
        theta[a] = sample_beta(alpha[a], beta[a])

    a_t = argmax_a theta[a]
    r_t = show(a_t)

    if r_t == 1:
        alpha[a_t] += 1
    else:
        beta[a_t] += 1
```

面试表达：

Thompson Sampling 不是固定比例随机探索，而是“按不确定性自然探索”。每个动作维护一个收益分布，每轮从分布采样后选最大。数据少的动作 posterior 宽，偶尔会采样出高值，从而获得探索机会；数据多且表现好的动作 posterior 窄，会稳定获得利用机会。

直觉：

一个动作的不确定性越大，它在采样中偶尔成为最优的概率越大，因此会自然获得探索机会。

优点：

- 探索非常自然。
- 在 bandit 问题中表现强。
- 常用于推荐、广告、实验分流。

缺点：

- 需要构造合适的概率模型。
- 在复杂深度 RL 中实现和建模成本更高。

4.9 完整 RL 中的探索落地

回到第 3 章的 SARSA 和 Q-learning，探索通常体现在 behavior policy 上。例如 Q-learning 可以用 epsilon-greedy 收集数据，再用 greedy target 做更新：

```
behavior policy: epsilon-greedy
target update:   max_a Q(s', a)
```

这说明探索机制和学习目标是两件事：

- 探索机制决定 agent 看到什么数据。
- 学习目标决定 value 或 policy 往哪里更新。

后面的 DQN、PPO、SAC 也都要处理探索，只是方式不同：

算法类型	常见探索方式
DQN	epsilon-greedy、NoisyNet
Policy Gradient / PPO	stochastic policy、entropy bonus
SAC	maximum entropy objective

4.9.1 NoisyNet 与参数空间探索

深度 RL 中还可以在参数空间探索。例如 NoisyNet 在网络参数中加入可学习噪声。

直觉：

- epsilon-greedy 是 action-level 随机。
- 参数空间噪声会让整个策略在一段时间内保持一致的探索模式。

这在需要 temporally coherent exploration 的任务中更合理。

4.10 本章例子：广告推荐里的探索

4.10.1 例子 1：Epsilon-greedy

假设有 3 个广告候选：

广告	当前点击率估计
A	4.0%
B	3.5%
C	1.0%

如果完全 exploitation，系统总是展示 A。但 C 的估计可能只是因为曝光少，不代表真实差。

epsilon-greedy 的做法：

- 95% 概率展示当前最优广告 A。
- 5% 概率随机探索 A/B/C。

这能避免过早把 B 或 C 判死。

4.10.2 例子 2: UCB

UCB 会给曝光少、不确定性高的广告更高 bonus。

直觉：

`score = 当前平均收益 + 不确定性奖励`

如果广告 C 曝光很少，即使平均点击率暂时低，也可能因为不确定性高被选中继续探索。

4.10.3 例子 3: Thompson Sampling

Thompson Sampling 会为每个广告维护一个后验分布，然后从分布里采样。

如果广告 A 数据多，后验分布窄；广告 C 数据少，后验分布宽。C 偶尔采样出很高的潜在点击率，就会获得探索机会。

面试表达：

Epsilon-greedy 是固定比例随机探索；UCB 是乐观面对不确定性；Thompson Sampling 是按后验不确定性自然采样。推荐和广告场景里，探索机制直接影响冷启动和长期收益。

4.11 本章面试高频问题

4.11.1 为什么强化学习需要探索？

因为 agent 初始并不知道哪些动作长期收益最高。如果只选择当前估计最优的动作，可能因为早期估计错误而陷入局部最优。

4.11.2 epsilon-greedy 的缺点是什么？

探索是均匀随机的，没有利用不确定性信息，也不区分非 greedy 动作的潜在价值。在大动作空间中效率较低。

4.11.3 UCB 的核心思想是什么？

UCB 使用当前价值估计加上不确定性 bonus 选动作。被尝试较少的动作有更大 bonus，从而获得探索机会。

4.11.4 Thompson Sampling 和 UCB 的区别？

UCB 基于置信上界，显式构造 optimism bonus。Thompson Sampling 是 Bayesian 方法，从 posterior 采样参数，通过概率匹配自然平衡探索和利用。

4.11.5 Bandit 和完整 RL 的区别？

Bandit 没有复杂状态转移，动作只影响当前 reward。完整 RL 中动作会影响未来状态分布和长期 return。

4.12 本章练习

1. 写出 epsilon-greedy 的动作选择逻辑。
2. 解释 UCB 中 $\sqrt{\ln t / N(a)}$ 的直觉。
3. 用推荐系统举一个 bandit 的例子。
4. 比较 epsilon-greedy、UCB 和 Thompson Sampling。

▼ 参考答案（点击展开）

1. **Epsilon-greedy 逻辑**：生成 $u \sim \text{Uniform}(0,1)$ 。若 $u < \epsilon$ ，从全部动作中随机选一个；否则选择 $\arg \max_a Q(s,a)$ 。如果探索时也可能抽中 greedy action， K 个动作下 greedy action 的总概率是 $1 - \epsilon + \epsilon/K$ 。

2. **UCB bonus 直觉**: $N(a)$ 小表示动作尝试少、不确定性高, bonus 会变大; 同一动作被反复尝试后, $N(a)$ 增长, bonus 会衰减。 $\ln t$ 缓慢增长, 表示即使很久没选某个动作, 随着总轮数增加仍会重新检查它, 避免过早永久放弃。
3. **推荐系统例子**: 把候选广告当作 arms, context 是用户画像和场景, action 是展示某个广告, reward 是点击或转化。系统既要多展示当前点击率高的广告, 也要给新广告少量曝光来估计效果; 若展示不改变用户长期状态, 可先建模为 contextual bandit。
4. **三者比较**: epsilon-greedy 以固定概率做无差别随机探索, 实现最简单但不利用不确定性; UCB 选择“均值 + 置信 bonus”最大的动作, 属于确定性的 optimism in the face of uncertainty; Thompson Sampling 从每个动作的 posterior 采样后选最大值, 属于概率匹配。UCB 和 Thompson Sampling 通常比固定随机探索更有针对性, 但需要额外的不确定性估计或概率模型。

第 5 章：函数逼近、稳定性与 Deadly Triad

5.1 本章符号说明

符号/缩写	English	中文	含义
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值。
$Q(s,a)$	Action-value Function	动作价值函数	状态-动作对 (s,a) 的长期价值。
$\pi(a s)$	Policy	策略	状态 s 下选择动作 a 的概率。
θ / w	Parameters / Weights	参数 / 权重	策略网络或价值网络的可学习参数。
w^-	Target Network Weights	目标网络权重	从在线权重 w 延迟同步或软更新得到的参数。
y	Target	训练目标值	Bellman target，用来监督当前估计。
$L(w)$	Loss Function	损失函数	训练价值网络时最小化的目标。
(s,a,r,s',d)	Transition Tuple	转移样本五元组	当前状态、动作、即时奖励、下一状态和终止指示量；代码常把 d 写成 <code>done</code> 。
d / done	Terminal Indicator	终止指示量	$d=1$ 表示该转移到达真正终止状态， $d=0$ 表示仍可从 s' bootstrap。
γ	Discount Factor	折扣因子	控制 Bellman target 中未来价值的权重。
$d_\mu(s,a)$	Behavior Distribution	行为分布	行为策略 μ 与环境交互后在状态-动作对上的访问分布。
μ	Behavior Policy	行为策略	实际产生训练数据的策略；可与目标策略 π 不同。
τ_{soft}	Soft-update Rate	软更新系数	控制目标网络每次从在线网络吸收多少新参数，通常远小于 1。
behavior policy	Behavior Policy	行为策略	实际收集训练数据的策略。
target policy	Target Policy	目标策略	当前算法想学习或优化的策略。
on-policy	On-policy Learning	同策略学习	数据来自当前正在学习的策略。
off-policy	Off-policy Learning	异策略学习	用一个策略的数据学习另一个策略。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 $Q(s,a)$ 的 value-based 算法。
TD	Temporal Difference	时序差分	用一步 reward 和下一状态估计构造 target 的方法。
RL	Reinforcement Learning	强化学习	本章讨论 RL 从表格方法进入函数逼近后的稳定性问题。
LLM	Large Language Model	大语言模型	文中作为高维状态或复杂序列系统的例子。

5.2 本章目标

本章从表格型 RL 过渡到深度强化学习。你需要掌握：

- 为什么需要函数逼近。
- 线性函数逼近和神经网络逼近。
- value function approximation。
- bootstrapping 的影响。
- off-policy learning 的分布问题。
- Deadly Triad。

5.3 本章主线

前面几章默认可以为每个状态或状态-动作对存一个表格值。真实任务里状态空间通常巨大或连续，所以必须用参数化函数近似 $V(s)$ 、 $Q(s,a)$ 或 $\pi(a/s)$ 。本章先建立函数逼近的训练形式，再解释为什么 bootstrapping、off-policy learning 和 function approximation 同时出现时会带来稳定性问题。

5.4 本章新增概念说明

本章开始进入 deep RL，先把稳定性相关名词放在一起：

名词	中文	先记住什么
function approximation	函数逼近	用参数化模型近似 $V(s)$ 、 $Q(s,a)$ 或 $\pi(a/s)$ ，不再为每个状态单独建表。
value function approximation	价值函数逼近	用模型预测价值函数，例如 $Q(s,a;w)$ 。
bootstrapping	自举	target 里用了模型当前自己的估计，例如 $r+\gamma V(s')$ 。
off-policy distribution shift	异策略分布偏移	数据来自一个策略，学习目标却是另一个策略，导致训练分布和目标分布不一致。
Deadly Triad	致命三角	function approximation、bootstrapping、off-policy 同时出现时，训练容易不稳定。
replay buffer	经验回放池	存历史 transition，训练时随机采样，打破连续样本相关性。
target network	目标网络	滞后更新的网络，用来让 Bellman target 更稳定。
overestimation bias	过估计偏差	max 操作容易选择被噪声高估的动作，导致 Q 值偏高。
reward clipping / gradient clipping	奖励截断 / 梯度截断	工程稳定化技巧，用来限制训练信号或梯度过大。

5.5 函数逼近为什么出现

5.5.1 为什么需要函数逼近

表格型方法为每个状态或状态动作对存一个值：

$V(s)$
 $Q(s, a)$

但真实任务中状态空间可能极大，甚至连续：

- 图像输入。
- 机器人连续状态。
- 推荐系统用户特征。
- LLM 对话状态。

这时无法为每个状态单独建表，需要用参数化函数近似价值函数或策略。

例如：

$$\begin{aligned} V(s) &\approx V(s; w) \\ Q(s, a) &\approx Q(s, a; w) \\ \pi(a | s) &\approx \pi(a | s; \theta) \end{aligned}$$

5.5.2 Value Function Approximation

用神经网络近似 Q 函数：

$$Q(s, a; w)$$

训练目标通常来自 Bellman target。

例如 DQN 中：

$$y = r + \gamma \max_{a'} Q(s', a'; w)$$

损失：

$$L(w) = \mathbb{E}[(y - Q(s, a; w))^2]$$

其中 w^- 是 target network 参数。

5.5.3 函数逼近的好处

- 可以处理高维状态。
- 可以泛化到没见过的状态。
- 可以结合深度学习特征提取能力。
- 能用于图像、文本、连续控制等任务。

5.5.4 函数逼近的风险

表格型方法中，不同状态动作对的值相互独立。

函数逼近中，共享参数会导致一个样本更新影响许多状态动作对的估计。

这带来几个问题：

- 训练目标非平稳。
- 样本之间高度相关。
- 误差可能通过 bootstrap 扩散。
- 价值函数可能过估计或发散。

5.6 不稳定来源：Deadly Triad

5.6.1 Bootstrapping

Bootstrapping 指 target 中包含当前模型自己的估计。

例如 TD target:

$$r + \gamma V(s')$$

Q-learning target:

$$r + \gamma \max_{a'} Q(s', a')$$

它的优点:

- 可以在线更新。
- 不需要完整 episode。
- sample efficiency 较高。

它的风险:

- target 依赖当前估计。
- 估计错误会被继续传播。
- 函数逼近下更容易放大不稳定性。

5.6.2 Off-policy Distribution Shift

off-policy 学习中, 数据来自 behavior policy, 但学习目标是 target policy。

data distribution: $d_{\mu}(s, a)$

target policy: π

如果 behavior policy 产生的数据分布和 target policy 需要的数据分布差异很大, 模型可能会在没数据覆盖的状态动作上产生错误估计。

这种问题在 offline RL 中尤其严重。

5.6.3 Deadly Triad

Deadly Triad 指三个因素同时出现时, RL 训练可能不稳定甚至发散:

1. Function approximation。
2. Bootstrapping。
3. Off-policy learning。

为什么危险?

- 函数逼近会让误差泛化到其他状态。
- bootstrapping 会用当前估计构造 target, 错误会自我强化。
- off-policy 会带来数据分布和学习目标不一致。

DQN 同时包含这三个因素, 因此需要额外稳定技巧:

- replay buffer 打破样本相关性。
- target network 稳定 bootstrap target。
- reward clipping。
- gradient clipping。
- Double DQN 降低 overestimation bias。

5.7 DQN 稳定化思想

5.7.1 从 Deadly Triad 到 DQN 稳定化

下面的 replay buffer 和 target network 不应该理解成孤立技巧, 而是第 6 章 DQN 的铺垫。逻辑是:

```

Q-learning + neural network
-> function approximation
-> bootstrap target
-> off-policy replay data
-> Deadly Triad risk
-> replay buffer / target network / Double DQN

```

所以第 5 章先解释“不稳定从哪里来”，第 6 章再系统讲“DQN 如何把这些稳定化设计组合成算法”。

5.7.2 Experience Replay 的稳定作用

Replay buffer 存储历史 transition:

```
(s, a, r, s', done)
```

五个字段依次是当前状态 s 、已执行动作 a 、即时奖励 r 、下一状态 s' 和终止标记 $done$ 。下文用数学符号 $d \in \{0, 1\}$ 表示 $done$ ； $d=1$ 时没有可继续估计的未来回报。

训练时从 buffer 中随机采样 mini-batch。

好处：

- 打破连续样本相关性。
- 提高样本利用率。
- 让数据分布更平滑。

限制：

- 数据可能 stale。
- 对 on-policy 算法不天然适用。
- 分布偏移仍然存在。

5.7.3 Target Network 的稳定作用

如果 target 和 online prediction 使用同一个网络，带终止掩码的 target 是：

$$y = r + \gamma(1-d) \max_a Q(s', a'; w)$$

那么模型参数一更新，target 也立刻变化，训练目标非常不稳定。

Target network 使用滞后的参数 w^- ：

$$y = r + \gamma(1-d) \max_a Q(s', a'; w^-)$$

每隔一段时间同步：

$$w^- \leftarrow w$$

或者软更新：

$$w^- \leftarrow \tau_{soft} w + (1 - \tau_{soft}) w^-$$

5.8 本章例子：为什么表格法不够

5.8.1 例子 1：Atari 像素输入

Atari 游戏状态是一张图片。如果用表格法，每一种像素组合都对应一个状态，状态数量几乎无限。

函数逼近的做法是：

输入：游戏画面
神经网络：CNN / encoder
输出：每个动作的 Q 值

这样模型可以泛化。例如两个画面只差几个像素，但局面本质相似，网络可以给出相近价值。

5.8.2 例子 2：推荐系统高维状态

推荐系统状态可能包含：

- 用户画像。
- 最近点击序列。
- 商品特征。
- 时间、地点、设备。

这些状态无法枚举。函数逼近可以把高维特征映射到价值或策略：

$$Q(s, a; \theta)$$

5.8.3 例子 3：Deadly Triad 为什么危险

假设用 DQN 训练一个推荐策略：

- function approximation：用神经网络估计 Q。
- bootstrapping：target 里有 $\max_a Q(s', a)$ 。
- off-policy：数据来自旧策略或 replay buffer。

如果新策略常推荐训练数据里很少出现的内容，Q 网络可能对这些动作过度乐观；bootstrap 又会把这种高估继续传播，训练就可能发散。

面试表达：

Deadly Triad 的危险不是三个词各自危险，而是它们叠加后，错误估计会通过 bootstrap 在 off-policy 分布外被不断放大。

5.9 本章面试高频问题

5.9.1 为什么表格型 RL 不够用？

因为真实任务的状态空间通常很大或连续，无法为每个状态动作对单独存一个值。函数逼近可以泛化到未见状态，并处理高维输入。

5.9.2 什么是 Deadly Triad？

Function approximation、bootstrapping 和 off-policy learning 三者同时出现时，强化学习训练容易不稳定甚至发散。

5.9.3 为什么神经网络加 Q-learning 会不稳定？

Q-learning 使用 bootstrap target，target 又依赖当前估计。神经网络共享参数，一个样本更新会影响大量状态动作估计。再加上 off-policy 数据分布偏移，误差可能被放大。

5.9.4 replay buffer 解决什么问题？

它通过随机采样历史 transition，打破连续样本相关性，提高样本利用率，并让训练数据分布更平滑。

5.9.5 target network 解决什么问题？

它让 Bellman target 在一段时间内保持相对稳定，避免 online network 更新时 target 也同步剧烈变化。

5.10 本章练习

1. 用自己的话解释 bootstrapping。
2. 画出 DQN 中 online network 和 target network 的关系。
3. 解释为什么 off-policy + function approximation 容易出问题。
4. 总结 DQN 针对 Deadly Triad 做了哪些稳定化设计。

▼ 参考答案（点击展开）

1. **Bootstrapping**: 更新 target 中含有当前价值估计本身，例如 TD target $r + \gamma V(s')$ 或 Q-learning target $r + \gamma \max_a Q(s', a)$ 。它能在 episode 结束前更新，但错误估计也可能被反复传递和放大。

2. **Online/target network 关系**:

```

transition batch
  |
  v
online Q(s, a; theta) <--- optimize loss ---+--- target Q(s', a'; theta-)
  |                                     |
  +----- current prediction          +--- Bellman target y

```

梯度只更新 online network 的 θ ；target network 的 θ^- 通过周期性 hard update 或缓慢 soft update 获得。

3. **为什么容易不稳定**: off-policy 数据分布和当前 target policy 想访问的分布不一致；function approximation 又让一次参数更新同时改变许多状态的预测；如果 target 还通过 bootstrapping 依赖这些预测，分布外误差就可能被传播，甚至形成发散反馈。
4. **DQN 的稳定化设计**: replay buffer 随机化并复用历史样本，缓解连续样本相关性；target network 让 bootstrap target 在一段时间内固定；Huber loss、reward clipping、gradient clipping 可限制异常更新；Double DQN 缓解 max 导致的过估计。它们降低风险，但不代表从理论上彻底消除了 Deadly Triad。

第 6 章：DQN（Deep Q-Network，深度 Q 网络）与经典改进

6.1 本章符号说明

符号/缩写	English	中文	含义
RL	Reinforcement Learning	强化学习	agent 通过与环境交互、接收 reward 并优化长期 return 的学习范式。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
(s,a,r,s',d)	Transition Tuple	转移样本五元组	当前状态、执行动作、即时奖励、下一状态和终止指示量。
d / <code>done</code>	Terminal Indicator	终止指示量	$d=1$ 表示 transition 后任务终止，没有后续价值； $d=0$ 表示可以继续估计未来价值。代码常把这个布尔字段命名为 <code>done</code> 。
$\mathcal{A}$ / $ \mathcal{A} $	Action Set / Number of Actions	动作集合 / 动作数量	$\mathcal{A}$ 是离散动作集合， $ \mathcal{A} $ 是其中的动作数。
$Q(s,a;\theta)$	Online Action-value Function	在线动作价值函数	参数为 θ 的在线 Q 网络对 (s,a) 的估计。
$Q(s,a;\theta')$	Target Action-value Function	目标动作价值函数	参数为 θ' 的目标 Q 网络，用于构造较稳定的 TD target。
θ / θ'	Online / Target Parameters	在线 / 目标网络参数	θ 每次训练更新； θ' 按较慢频率从 θ 同步。
γ	Discount Factor	折扣因子	控制下一状态价值在 target 中的权重。
α	Learning Rate / Step Size	学习率 / 步长	表格 Q-learning 中吸收 TD error 的比例。
ϵ	Exploration Probability	探索概率	epsilon-greedy 中随机探索的概率。
y	TD Target	TD 目标	当前 Q 预测需要逼近的监督信号。
δ / δ_i	TD Error	TD 误差	target 与当前 Q 预测之差； δ_i 表示第 i 条样本的误差。
$L(\theta)$	Loss Function	损失函数	训练在线 Q 网络时最小化的回归损失。
$V(s)$	State-value Stream	状态价值分支	Dueling DQN 中表示“状态整体有多好”的标量输出。
$A(s,a)$	Advantage Stream	动作优势分支	Dueling DQN 中表示动作相对该状态共同基线好多少的输出；这里的斜体 A 不是动作集合 $\mathcal{A}$ 。
p_i / $P(i)$	Priority / Sampling Probability	优先级 / 采样概率	PER 中第 i 条样本的优先级及其被抽到的概率。
α_p	Priority Exponent	优先级指数	控制 PER 采样对优先级的偏向程度；与学习率 α 不同。
β_{IS}	IS Correction Exponent	重要性采样修正指数	控制 PER 对非均匀采样偏差的修正强度。
ϵ_p	Priority Floor	优先级下限常数	防止零 TD error 样本永远无法被采样的小正数；与探索率 ϵ 不同。

符号/缩写	English	中文	含义
w_i	Importance Sampling Weight	重要性采样权重	PER 中修正非均匀采样偏差的样本权重。
N	Replay-buffer Size	回放池样本数	PER 权重公式中的样本总数。
$G_t^{(n)}$	n-step Return	n 步回报	从时刻 t 开始累积 n 个真实 reward，再接入一次 bootstrap value。
$Z(s,a)$	Return Distribution	回报分布	Distributional RL 中状态动作对 (s,a) 的随机 return 分布；其期望才是 $Q(s,a)$ 。
$z_j / c_j(s,a)$	Atom Value / Atom Probability	原子取值 / 原子概率	C51 中第 j 个固定 return 原子及网络为其预测的概率。
C51	Categorical Distributional RL with 51 Atoms	使用 51 个原子的分类分布强化学习	用 51 个固定原子近似 return distribution 的经典 Distributional DQN。
$\mu_w / \sigma_w / \xi$	Mean Weight / Noise Scale / Random Noise	权重均值 / 噪声尺度 / 随机噪声	NoisyNet 中共同构造带噪参数 w ； μ_w, σ_w 可学习， ξ 每次采样但不学习。
TD	Temporal Difference	时序差分	用即时奖励和下一状态估计构造学习目标。
PER	Prioritized Experience Replay	优先经验回放	按学习价值而非完全均匀地抽取 replay 样本。
SGD	Stochastic Gradient Descent	随机梯度下降	用随机 mini-batch 更新神经网络参数的方法。
IS	Importance Sampling	重要性采样	用权重修正采样分布变化的方法。

6.2 本章目标

学完本章，你应该能沿着一条完整因果链解释：

1. 为什么表格 Q-learning 不能直接处理高维状态。
2. 为什么“把 Q 表换成神经网络”仍然不够稳定。
3. Experience Replay 和 Target Network 分别解决什么问题。
4. Double DQN、Dueling DQN、PER 各自针对哪个剩余缺陷。
5. 哪些改进改变 target，哪些改变网络结构，哪些改变采样分布。
6. Multi-step Learning、Distributional RL 与 NoisyNet 在 Rainbow DQN 中分别改变什么。

6.3 本章主线：每个组件都必须对应一个问题

先看整章的问题树：

表格 Q-learning 无法处理巨大状态空间

- > 用神经网络近似 $Q(s, a)$
- > 新问题 1: 连续交互样本高度相关
 - > Experience Replay
- > 新问题 2: 预测网络和训练 target 同时变化
 - > Target Network
- > 得到基础 DQN

基础 DQN 仍有三个典型缺陷

- > max 容易放大 Q 估计噪声
 - > Double DQN
- > 每个动作 Q 值包含大量重复的状态价值信息
 - > Dueling DQN
- > 均匀 replay 不区分样本学习价值
 - > Prioritized Experience Replay

进一步组合为 Rainbow DQN

- > 单步 target 传播奖励较慢
 - > Multi-step Learning
- > 单个 Q 均值丢失 return 的分布形状
 - > Distributional RL
- > epsilon-greedy 的随机探索缺少状态相关结构
 - > NoisyNet

用一张表先固定它们的边界：

方法	它解决的问题	它主要改变什么
Experience Replay	连续样本相关、单条经验只用一次	训练数据的组织方式
Target Network	TD target 随在线网络同步移动	target 的计算网络
Double DQN	max 操作放大过估计	target 中“选动作”和“估值”的分工
Dueling DQN	状态共同价值被每个动作输出重复学习	Q 网络的输出结构
PER	均匀采样浪费在已学好的样本上	replay buffer 的采样分布
Multi-step Learning	单步 TD 的奖励传播较慢	target 使用的真实 reward 步数
Distributional RL	单个均值无法描述 return 的随机性与分布形状	网络预测的价值对象
NoisyNet	epsilon-greedy 探索缺少状态相关结构	网络参数与行为策略

这张表是本章的脉络。后面每个公式都从对应问题出发。

6.4 从 Q-learning 推到 DQN

6.4.1 起点：表格 Q-learning

表格 Q-learning 对一个格子做增量更新：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [y - Q(s,a)]$$

其中 target 是：

$$y=r+\gamma(1-d)\max_a Q(s',a')$$

式中：

- r 是这一步真实获得的 reward。
- $\max_a Q(s',a')$ 是下一状态可达到的最大估计价值。
- $1-d$ 是终止掩码；终止时 $d=1$ ，未来价值自动变成 0。
- $y-Q(s,a)$ 就是 TD error。

表格方法的问题不是更新逻辑错误，而是无法存下所有状态。例如输入是一张游戏画面时，几乎不可能为每种像素组合建立一个 Q 表格行。

6.4.2 第一步：用神经网络替代 Q 表

DQN 用参数为 θ 的神经网络近似 Q 函数：

$$Q(s,a;\theta)$$

在离散动作空间中，网络通常只输入状态 s ，一次输出所有动作的 Q 值：

```
state s
-> Q network
-> [Q(s,a1), Q(s,a2), ..., Q(s,an)]
```

选择动作时：

- 以 ϵ 概率随机探索。
- 以 $1-\epsilon$ 概率选择 Q 值最大的动作。

6.4.3 第二步：把表格更新改写成回归

神经网络不能只修改某一个表格格子。它能做的是：让当前预测 $Q(s,a;\theta)$ 靠近 target y 。

因此把 Q-learning 改写为监督学习式回归：

```
input:      (s, a)
target:     y
prediction:  Q(s, a; theta)
error:      y - Q(s, a; theta)
```

平方损失可以写成：

$$L(\theta)=\mathbb{E}[(y-Q(s,a;\theta))^2]$$

实际 DQN 常用 Huber loss：小误差区域近似平方损失，大误差区域近似绝对值损失，从而降低异常 TD error 对梯度的冲击。

6.4.4 为什么“直接用神经网络在线训练”仍不稳定

如果每和环境交互一步，就立刻用最新 transition 训练同一个网络，会遇到两个核心问题。

6.4.4.1 问题一：样本高度相关

连续三帧游戏画面通常只发生很小变化：

s_t : 小车在中央
 s_{t+1} : 小车略微向右
 s_{t+2} : 小车继续向右

按时间顺序直接训练，连续 mini-batch 会被局部轨迹主导，不符合 SGD 希望数据较充分混合的条件。

6.4.4.2 问题二：target 一直移动

如果预测和 target 都使用当前网络：

$$y = r + \gamma(1-d)\max_a Q(s', a'; \theta)$$

每次更新 θ 时：

- 当前预测 $Q(s, a; \theta)$ 变了。
- 用来监督它的 target y 也立刻变了。

这像一边追目标，一边同步移动目标，容易产生震荡甚至发散。

基础 DQN 的两个核心组件正好分别处理这两个问题。

6.5 基础 DQN：Replay Buffer 加 Target Network

6.5.1 Experience Replay：先解决数据相关性

Replay buffer 保存历史 transition：

(s, a, r, s', d)

字段含义：

字段	含义
s	执行动作前的当前状态。
a	agent 实际执行的动作。
r	执行动作后得到的即时奖励。
s'	环境到达的下一状态。
d / done	是否已经终止；1 表示没有后续价值，0 表示仍可从 s' 估计未来价值。done 只是常见代码字段名。

训练时不只使用刚产生的 transition，而是从 replay buffer 随机抽取 mini-batch。

它带来两个作用：

1. **打散相关性**：同一个 batch 可以混合不同时间、不同 episode 的经验。
2. **复用经验**：一条环境交互数据可以被训练多次，提高样本效率。

它没有解决 target 移动问题，所以还需要 target network。

6.5.2 Target Network：再解决移动目标

DQN 维护两个结构相同的 Q 网络：

网络	参数	职责	更新方式
Online Network	θ	产生当前 Q 预测并接受梯度更新	每个训练 step 更新

网络	参数	职责	更新方式
Target Network	θ^-	只负责计算 TD target	每隔固定步数从 θ 同步

target 改成:

$$y = r + \gamma(1-d)\max_a Q(s', a'; \theta^-)$$

在两次同步之间, θ^- 保持不变, 所以在线网络面对的是相对稳定的监督目标。

注意: Target Network 不是第二个独立学习目标。它只是在线网络的滞后副本, 用时间上的延迟换取 target 稳定性。

6.5.3 完整 DQN 训练流程

```

initialize online network Q(theta)
initialize target network Q(theta-) <- Q(theta)
initialize replay buffer

repeat:
  1. use epsilon-greedy to choose action a
  2. execute a and observe (r, s', d)
  3. store (s, a, r, s', d) in replay buffer

  4. sample a random minibatch from replay buffer
  5. for each transition:
      y = r + gamma * (1-d) * max_a' Q(s', a'; theta-)
  6. update theta to reduce Huber(y - Q(s, a; theta))

  7. periodically copy theta to theta-
  8. move to s'

```

这就是基础 DQN。不要把它记成零散技巧, 应该记成:

```

Q-learning target
+ neural Q function
+ replay buffer
+ target network
= DQN

```

6.5.4 数值例子: target、TD error 和 loss 怎么算

假设一条非终止 transition 中:

```

r = 1
gamma = 0.9
d = 0
target network 在 s' 的输出 = [2, 4]
online network 当前 Q(s,a) = 3

```

先计算下一状态最大价值:

$$\max_a Q(s', a'; \theta^-) = 4$$

再计算 target:

$$y=1+0.9\times 4=4.6$$

TD error:

$$\delta=4.6-3=1.6$$

平方误差:

$$\delta^2=2.56$$

如果同一条 transition 是终止转移, 即 $d=1$:

$$y=1+0.9\times(1-1)\times 4=1$$

这就是 done 字段存在的全部意义: 告诉 target 终止之后不能再凭空接入未来价值。理解 DQN 本身不需要引入任何特定环境库。

6.5.5 基础 DQN 解决了什么, 还没解决什么

基础 DQN 已经解决:

- 高维状态下无法维护 Q 表。
- 连续样本高度相关。
- bootstrap target 同步快速移动。

但它仍然没有解决:

- max 操作带来的 Q 值过估计。
- 网络对“状态共同价值”和“动作相对差异”的学习效率。
- replay buffer 中不同样本的学习价值不同。

下面三个改进依次对应这三个问题。

6.6 Double DQN: 修正 max 带来的过估计

6.6.1 问题: max 为什么容易高估

假设每个动作的估计都等于:

$$\text{estimated } Q = \text{true } Q + \text{estimation noise}$$

即使每个动作的噪声平均为 0, 取最大值时也更容易选中“恰好被正噪声抬高”的动作。

基础 DQN 的 target network 同时完成两件事:

1. 从下一状态选择 Q 最大的动作。
2. 用这个最大 Q 值评价该动作。

同一份噪声既影响选择, 又影响估值, 因此正噪声容易被 max 放大。

6.6.2 思路: 选动作和估值分开

Double DQN 让在线网络负责选择动作:

$$a^* = \arg \max_a Q(s', a'; \theta)$$

再让目标网络只评价这个动作：

$$y=r+\gamma(1-d)Q(s',a^*;\theta)$$

一句话：

```
online network:  decide which action
target network:  evaluate that action
```

6.6.3 数值例子

假设在线网络和目标网络对下一状态给出：

网络	动作 1	动作 2
Online Network	9.9	10.4
Target Network	10.1	10.0

基础 DQN 只看 target network 的最大值：

$$\max(10.1, 10.0)=10.1$$

Double DQN 先由 online network 选择动作 2，再由 target network 评价动作 2：

$$Q(s',a_2;\theta)=10.0$$

这个例子中，Double DQN 没有重复选择 target network 中被噪声抬高的动作 1。

注意：Double DQN 不保证每一个样本的 target 都更小。它降低的是“同一估计噪声同时负责选择和估值”造成的系统性过估计。

6.6.4 Double DQN 改了什么

它只修改 target 的计算方式：

- replay buffer 不变。
- online/target 两个网络仍然存在。
- Q 网络结构可以完全不变。
- epsilon-greedy 行为策略不变。

因此 Double DQN 可以继续与 Dueling DQN、PER 组合。

6.7 Dueling DQN：拆开状态共同价值与动作相对差异

6.7.1 先看它要解决的问题

假设某个状态下有三个动作，Q 值分别是：

```
[5.1, 5.0, 4.9]
```

这里包含两类信息：

1. **共同信息**：这个状态整体价值大约是 5。
2. **差异信息**：动作之间只有 +0.1、0、-0.1 的相对差异。

普通 DQN 直接输出三个 Q 值，相当于让三个动作输出分别学习那一大块重复的“状态整体价值”。

在很多状态中，动作差异暂时并不重要。例如小车和障碍物还很远时，短期内向左或向右可能差别很小；但“当前局面整体安全还是危险”仍然值得先学准。

Dueling DQN 的动机就是：

让网络单独学习状态的共同价值，再单独学习动作之间的相对差异。

6.7.2 两个分支分别表示什么

Dueling DQN 把网络输出拆成两个分支：

- $V(s)$: **State-value Stream** (状态价值分支)，输出一个标量，表示状态 s 的共同价值基线。
- $A(s,a)$: **Advantage Stream** (动作优势分支)，为每个动作输出一个分数，表示动作相对共同基线的差异。

这里必须区分两个符号：

- 斜体 $A(s,a)$: 某个动作的 advantage 输出。
- 花体 $\mathcal{A}$: 全部离散动作组成的 action set。

6.7.3 网络结构发生了什么变化

```

state -> shared encoder
                        -> value head V(s): 1 scalar
                        -> advantage head A(s,.) : |A| scores

V stream + centered A stream -> Q(s,.)

```

其中：

- shared encoder 是两条分支共用的状态特征提取器。
- value head 是输出 $V(s)$ 的分支。
- advantage head 是输出每个动作 $A(s,a)$ 的分支。
- 最后的 aggregation 层把两条分支重新合成为每个动作的 Q 值。

Dueling DQN 仍然输出 Q 值，最终选动作的方式没有改变。

6.7.4 为什么不能直接写成 $Q=V+A$

最直观的写法是：

$$Q(s,a) = V(s) + A(s,a)$$

但这个分解不唯一。任意给定常数 c ：

$$[V(s)+c] + [A(s,a)-c] = V(s) + A(s,a)$$

也就是说：

- V 整体加 10。
- 所有动作的 A 整体减 10。
- 最终 Q 值完全不变。

网络只看最终 Q loss，无法判断这 10 应该属于 V 还是 A 。这就是不可辨识性：同一组 Q 值对应无数组 V/A 分解。

6.7.5 正确做法：把 advantage 中心化

假设离散动作共有 $n=|\mathcal{A}|$ 个。先把 advantage 分支的平均输出单独记为 $m_A(s)$ ：

$$m_A(s) = [A(s,a_1) + A(s,a_2) + \dots + A(s,a_n)]/n$$

再用这个均值把 raw advantage 中心化：

$$Q(s,a) = V(s) + A(s,a) - m_A(s)$$

其中 $a_1, \dots, a_n$ 是动作集中的全部动作。这个写法与“减去所有动作 advantage 的均值”完全等价，但更容易直接看出每一项在做什么：

```
当前动作的 raw advantage
- 所有动作 raw advantage 的平均值
= centered advantage
```

中心化后，所有动作 advantage 的平均值为 0，因此：

$$[Q(s,a_1) + Q(s,a_2) + \dots + Q(s,a_n)]/n = V(s)$$

这时 $V(s)$ 有了明确含义：它等于该状态下所有动作 Q 值的平均基线；每个动作的中心化 advantage 决定它在基线上方还是下方。

6.7.6 数值例子：完整算一遍

假设一个状态有三个动作，网络两条分支输出：

```
V(s) = 5
raw A(s,.) = [2, 1, 0]
```

第一步，计算 raw advantage 的平均值：

$$(2+1+0)/3=1$$

第二步，中心化 advantage：

```
[2, 1, 0] - 1 = [1, 0, -1]
```

第三步，与状态基线相加：

```
Q(s,.) = 5 + [1, 0, -1]
        = [6, 5, 4]
```

结果的含义：

- 状态整体基线是 5。
- 动作 1 比基线好 1，所以 Q 值是 6。
- 动作 2 与基线相同，所以 Q 值是 5。
- 动作 3 比基线差 1，所以 Q 值是 4。

这比一上来背 $Q=V+A$ 更重要：**Dueling** 的重点不是公式拆分，而是把重复的状态信息和动作差异信息交给不同分支学习。

6.7.7 Dueling 的梯度如何穿过均值

聚合操作是网络计算图的一部分，不是训练结束后的额外后处理。设有三个动作，当前 transition 执行的是动作 1：

$$Q_I = V + A_I - (A_1 + A_2 + A_3)/3$$

因此：

$$\partial Q_I / \partial V = 1$$

$$\partial Q_I / \partial A_1 = 2/3$$

$$\partial Q_I / \partial A_2 = \partial Q_I / \partial A_3 = -1/3$$

若 $\delta = Q_I - y$ ，那么 loss 对各输出的梯度就是上面的系数再乘 δ 。这意味着：

- value head 收到完整的公共误差信号。
- 当前动作的 advantage 收到 $2\delta/3$ 。
- 另外两个动作也会通过均值项分别收到 $-\delta/3$ 。
- 三个 advantage 输出方向上的梯度之和为 0，因此 loss 不会奖励所有 advantage 一起平移；每次 forward 仍会重新执行中心化。
- shared encoder 同时收到 value head 与 advantage head 两条路径的梯度。

在 PyTorch 中，`advantage.mean(dim=1, keepdim=True)` 是普通可微操作，自动求导会完成上述分配。

6.7.8 Dueling 可以理解为残差学习吗

可以把它理解为：

$Q(s, a)$
= 状态公共基线 $V(s)$
+ 动作相关残差 centered $A(s, a)$

这里的 centered advantage 确实是某个动作相对当前状态公共基线的残差。之所以叫 Advantage 而不是普通 Residual，是因为强化学习中 advantage 本来就表达“动作相对状态基线好多少”。

但它与 ResNet（Residual Network，残差网络）的残差连接不同：

- ResNet 的基线通常是输入 x ，形式是 $x + F(x)$ 。
- Dueling 的基线 $V(s)$ 和动作残差 $A(s, a)$ 都由网络学习。
- Dueling 还必须用中心化约束消除 V/A 之间任意搬移常数的问题。

这是一种有针对性的 inductive bias，而不是“学习残差必然更好”的定理。它在动作共享大量状态信息时更可能提高样本效率，但没有保证在每个任务上都优于普通 DQN。

6.7.9 Dueling DQN 改了些什么，没有改什么

项目	是否改变	说明
Q 网络内部结构	是	单一输出头改成 value head 和 advantage head。
最终输出	否	仍然输出每个离散动作的 Q 值。
DQN Bellman target	否	仍可使用基础 DQN target。
Double DQN target	可组合	可以用 Dueling 网络，同时使用 Double DQN 的 target。

项目	是否改变	说明
Replay Buffer	否	数据存储和采样逻辑不因 Dueling 改变。
行为策略	否	仍可使用 epsilon-greedy。
过估计问题	不直接解决	过估计主要由 Double DQN 处理。

所以要牢牢记住：

Double DQN 改 target；Dueling DQN 改网络结构。两者不是竞争关系，可以同时使用。

6.7.10 什么时候更可能有帮助

Dueling DQN 更适合：

- 离散动作空间。
- 很多状态下动作差异较小，但状态本身好坏很明显。
- 不同动作共享大量状态特征的任务。

它不是任何任务都必然提升，也不负责解决连续动作输出、探索不足或 replay 分布偏差。

6.7.11 面试回答模板

Dueling DQN 解决的是 Q 表示学习效率问题。普通 DQN 直接输出每个动作的 Q 值，而 Dueling 把网络拆成状态价值分支 $V(s)$ 和动作优势分支 $A(s,a)$ 。由于直接写 $Q=V+A$ 不可辨识，实际会减去所有动作 advantage 的均值再聚合。它只改变网络结构，不改变 replay 和 Bellman target，因此常与 Double DQN、PER 一起使用。

6.8 PER：让更有学习价值的 transition 更常被采样

6.8.1 问题：均匀 replay 一视同仁

普通 replay buffer 对所有 transition 均匀采样。

但不同样本的学习价值不同：

- TD error 很小：当前网络已经能较好预测。
- TD error 很大：当前预测与 target 差距大，通常更值得再次学习。

PER 的思路是让 TD error 大的样本更常被抽到。

6.8.2 从 TD error 构造优先级

第 i 条 transition 的 TD error：

$$\delta_i = y_i - Q(s_i, a_i; \theta)$$

优先级：

$$p_i = |\delta_i| + \epsilon_p$$

$\epsilon_p > 0$ 防止 TD error 暂时为 0 的样本永远失去采样机会。

采样概率：

$$z_p = p_1^{\alpha_p} + p_2^{\alpha_p} + \dots + p_N^{\alpha_p}$$

$$P(i) = p_i^{\alpha_p} / z_p$$

z_p 是归一化常数，即 replay buffer 中所有样本优先级幂次之和。

α_p 的作用：

- $\alpha_p = 0$ ：所有 $p_i^0 = 1$ ，退化为均匀采样。
- α_p 越大：越偏向高优先级样本。

6.8.3 为什么还需要重要性采样修正

非均匀采样改变了训练数据分布。高 TD error 样本被过度代表，直接训练会引入 sampling bias。

PER 给每条被采样的数据乘 importance sampling weight：

$$w_i = [N P(i)]^{-\beta_{IS}}$$

直觉：

- $P(i)$ 大的样本经常被抽到，因此单次更新权重应更小。
- $P(i)$ 小的样本很少被抽到，因此抽到时权重应更大。

β_{IS} 控制修正强度：

- $\beta_{IS} = 0$ ：不修正。
- $\beta_{IS} = 1$ ：完整使用该重要性权重形式。

实现中常把 batch 内 w_i 除以最大权重，避免整体梯度尺度突然变大。

6.8.4 数值直觉

假设两条样本：

```
sample 1: |TD error| = 0.2
sample 2: |TD error| = 1.8
```

忽略很小的 ϵ_p 时，第二条样本优先级远高于第一条，因此会更常被抽到。但因为第二条样本被重复看到的概率更高，它的 loss 还要乘较小的 w_i ，防止训练分布被它完全主导。

6.8.5 PER 的完整循环

```
1. new transition enters replay buffer
2. assign an initial priority
3. sample minibatch according to P(i)
4. compute weighted TD loss using w_i
5. update online network
6. recompute sampled transitions' TD errors
7. write new priorities back to replay buffer
```

PER 改的是采样分布，不改变 DQN 或 Double DQN 的 target 结构。

6.9 Rainbow DQN：这些改进如何组合

6.9.1 为什么这些组件可以组合

Double、Dueling、PER、Multi-step、Distributional RL 和 NoisyNet 作用在训练链路的不同位置，所以可以叠加。Rainbow DQN 的核心不是简单堆组件，而是让每个组件处理一个不同缺陷：

```
environment interaction
-> NoisyNet 决定如何探索
-> Multi-step collector 构造 n-step transition
-> PER 决定从 replay 中抽哪些样本
-> Double DQN 决定下一动作如何选择和估值
-> Distributional head 预测 return distribution
-> Dueling head 拆分状态公共信息和动作差异
```

6.9.2 Multi-step Learning：让 reward 更快向前传播

6.9.2.1 单步 TD 为什么传播慢

基础 DQN 的单步 target 是：

$$y_t = r_{t+1} + \gamma(1 - d_{t+1}) \max_a Q(s_{t+1}, a; \theta)$$

它只直接看到一个真实 reward，其余部分立即依赖网络 bootstrap。如果奖励只在轨迹末尾出现，这个信号需要经过许多轮 TD update 才能逐步传到更早状态。

Multi-step Learning（多步学习）先连续使用多个真实 reward，再接入一次 bootstrap。

6.9.2.2 n-step target 怎么构造

以 n 步为例：

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n (1 - d_{t+n}) \max_a Q(s_{t+n}, a; \theta)$$

这里：

- 前 n 项来自环境真实产生的 reward。
- 最后一项才使用目标网络 bootstrap。
- 如果中途已经终止，后续 reward 不再存在，bootstrap 项也必须清零。
- 当 $n=1$ 时，它退化为普通 DQN 的单步 target。

6.9.2.3 数值例子

假设：

```
n = 3
gamma = 0.9
连续 reward = [1, 1, 1]
第 3 步后尚未终止
target network 在 s_{t+3} 的最大 Q = 5
```

那么：

$$G_t^{(3)} = 1 + 0.9 + 0.81 + 0.729 \times 5 = 6.355$$

单步 target 只能直接使用第一个 reward；三步 target 一次就把后三步真实奖励带回当前状态，因此奖励传播更快。

6.9.2.4 它的代价是什么

- n 增大后，target 使用更多真实采样，bootstrap bias 通常减小。
- 但更长的随机 reward 序列也会提高 variance。

- replay buffer 需要保存或在线构造 n-step transition。
- off-policy 场景下, n 太大时还会更受行为策略与目标策略差异影响。

因此 Multi-step 不是“步数越多越好”, 而是在单步 TD 与完整 Monte Carlo return 之间取折中。

6.9.3 Distributional RL: 从预测均值改为预测分布

6.9.3.1 普通 DQN 丢掉了什么

普通 DQN 只预测期望:

$$Q(s,a) = \mathbb{E}[Z(s,a)]$$

其中 $Z(s,a)$ 是 Return Distribution (回报分布)。下面两个动作可能拥有相同的期望 Q 值:

```
action 1: 必然获得 return 5
action 2: 50% 获得 0, 50% 获得 10
```

它们的平均值都是 5, 但不确定性和分布形状完全不同。普通 DQN 会把它们压缩成同一个标量。

6.9.3.2 C51 怎么表示分布

C51 把可能的 return 范围离散成 51 个固定原子:

```
z_1, z_2, ..., z_51
```

网络不再为每个动作只输出一个 Q 值, 而是输出该动作落在 51 个原子上的概率:

```
c_1(s,a), c_2(s,a), ..., c_51(s,a)
```

概率加权后仍可得到用于选动作的期望 Q 值:

$$Q(s,a) = z_1 c_1(s,a) + z_2 c_2(s,a) + \dots + z_{51} c_{51}(s,a)$$

训练 target 对应分布形式的 Bellman update:

$$Z_{target} = r + \gamma(1-d)Z(s',a^*)$$

因为平移并折扣后的原子通常不再落在原来的固定位置上, C51 还要把 target distribution 投影回那 51 个固定原子, 再使用分类分布损失训练。

6.9.3.3 它带来什么, 不自动带来什么

- 网络获得了比单个均值更丰富的价值学习信号。
- 在值函数估计中, 分布形式常能改善表示与优化效果。
- 它可以区分“稳定获得 5”和“0/10 各一半”这类同均值分布。
- 但标准 C51 最终仍按分布的期望值选动作, 因此它不会自动变成风险规避策略; 要利用风险偏好, 还需要改变决策准则。

6.9.4 NoisyNet: 把探索放进网络参数

6.9.4.1 epsilon-greedy 的局限

epsilon-greedy 每一步通常独立地做一次选择:

- 以 ϵ 概率随机选动作。
- 否则选择当前 Q 值最大的动作。

这种探索简单, 但随机动作之间缺少结构和持续性。例如一个任务需要连续多步向同一方向探索时, 每一步独立随机可能很难形成完整行为。

6.9.4.2 NoisyNet 的参数化

NoisyNet 把普通网络参数 w 改写为：

$$w = \mu_w + \sigma_w \odot \xi$$

其中：

- μ_w 是可学习的参数均值。
- σ_w 是可学习的噪声尺度，控制这一参数需要多强的探索。
- ξ 是按一定频率重新采样的随机噪声，不参与学习。
- $\odot$ 表示逐元素相乘。

一次 forward/backward 内使用同一份 ξ 。梯度会更新 μ_w 和 σ_w ：

$$\partial L / \partial \mu_w = \partial L / \partial w$$

$$\partial L / \partial \sigma_w = (\partial L / \partial w) \odot \xi$$

因此网络可以自己学出“哪些参数需要较大噪声、哪些参数应该更稳定”。

6.9.4.3 它如何产生探索

当重新采样一组参数噪声后，整个 Q 函数会发生一致的轻微变化：

原来的 Q: [5.0, 4.9]

某次噪声后的 Q: [4.8, 5.2]

agent 于是会得到一套由状态决定、且在所有动作输出之间相互协调的探索偏好，而不是执行一个与状态无关的随机动作。如果同一份噪声保持多个环境 step，这种偏好还会具有时间持续性；如果每一步都重采样，主要收益则是状态相关的结构化探索，而不是持续性本身。

Rainbow 中通常用 NoisyNet 代替显式 epsilon-greedy schedule。评估时通常关闭随机噪声，只使用 μ_w 对应的确定性参数。

6.9.4.4 它不保证什么

- 参数噪声不保证一定发现稀疏奖励。
- 噪声采样频率和初始化会影响探索效果。
- 对非常简单的任务，epsilon-greedy 可能已经足够。
- NoisyNet 改的是探索机制，不负责修正过估计或 replay bias。

6.9.5 六类改进放在一起比较

组件	一句话定义	主要针对的问题
Double DQN	online 选动作，target 估值	Q 值过估计
Dueling DQN	状态价值和动作优势分支	Q 表示学习效率
PER	按 TD error 调整 replay 概率	均匀采样效率
Multi-step Learning (多步学习)	用连续多步真实 reward 构造 target	奖励传播过慢
Distributional RL (分布强化学习)	预测 return 的概率分布，而不只预测均值	价值分布表达不足
NoisyNet (噪声网络)	在网络参数中加入可学习噪声进行探索	epsilon-greedy 探索过于简单

本章面试重点不是背组件名字，而是能说明它们分别落在训练链路的哪里：

这些组件没有重复做同一件事；它们分别修改 target、网络结构、采样方式、回报形式、价值表示和探索机制。

6.10 DQN 的适用范围与完整例子

6.10.1 适用范围

DQN 适合：

- 离散动作空间。
- 可以一次输出并比较所有动作 Q 值。
- 状态维度较高，需要函数逼近。

DQN 不适合：

- 高维连续动作空间，因为无法枚举所有动作取 max。
- 极端稀疏奖励且简单 epsilon-greedy 很难探索到有效行为的任务。
- 环境数据分布变化极端、普通 replay 无法覆盖关键状态的任务。

6.10.2 CartPole 例子：从建模到训练

CartPole（小车倒立摆）是经典离散控制任务。目标是左右推动小车，让杆保持直立。

状态：

状态维度	含义
cart position	小车位置
cart velocity	小车速度
pole angle	杆角度
pole angular velocity	杆角速度

动作：

```
0: push left
1: push right
```

DQN 网络：

```
input: 4-dimensional state
output: [Q(s,left), Q(s,right)]
```

一轮交互和训练：

```
1. epsilon-greedy chooses left or right
2. environment returns reward, next state, and done
3. transition enters replay buffer
4. random minibatch is sampled
5. target network computes y
6. online network minimizes TD loss
7. target network is periodically synchronized
```

如果基础 DQN 有明显过估计，可以加入 Double DQN；如果许多状态下左右动作差异小，可以尝试 Dueling DQN；如果少数失败或关键转移很重要，可以尝试 PER。

这最后一句就是算法选型逻辑：先观察具体缺陷，再选择对应改进。

6.11 高频对比与面试问题

6.11.1 Replay Buffer 和 Target Network 分别解决什么

- Replay Buffer 处理训练数据问题：打散连续样本相关性并复用历史经验。
- Target Network 处理监督目标问题：让 bootstrap target 在一段时间内保持稳定。

6.11.2 Double DQN 和 Dueling DQN 有什么区别

- Double DQN 修改 target 计算，缓解 max 导致的过估计。
- Dueling DQN 修改网络结构，拆开状态共同价值与动作相对差异。
- 两者可以同时使用。

6.11.3 Dueling 为什么要减去 advantage 均值

直接写 $Q=V+A$ 时， V 加任意常数、所有 A 减同一常数都不会改变 Q ，因此分解不唯一。减去动作 advantage 的均值后，中心化 advantage 平均为 0， $V(s)$ 就对应所有动作 Q 值的平均基线。

6.11.4 PER 为什么需要 IS weight

PER 非均匀采样会改变训练分布。IS weight 降低高概率样本的单个影响、提高低概率样本被抽到时的影响，用于修正 sampling bias。

6.11.5 DQN 为什么不适合连续动作

DQN 需要比较所有动作并计算 $\max_a Q(s,a)$ 。连续动作有无限多个候选，无法像离散动作一样直接枚举。

6.12 本章练习

1. 从表格 Q-learning 更新推导 DQN target 和 loss。
2. 分别解释 Replay Buffer 与 Target Network 解决的问题。
3. 已知 $r=2$ 、 $\gamma=0.9$ 、 $d=0$ 、target network 在 s' 输出 $[3,5]$ ，计算 DQN target。
4. 解释 Double DQN 为什么把动作选择和动作估值拆开。
5. 某 Dueling 网络输出 $V(s)=4$ 、raw $A(s,\cdot)=[3,1,2]$ ，计算三个动作的 Q 值。
6. 说明 Double DQN 与 Dueling DQN 为什么可以组合。
7. 解释 PER 中 α_p 、 β_{IS} 和 ε_p 的作用。
8. 三动作 Dueling 网络训练动作 1 时，说明 advantage 均值为什么会让另外两个动作也收到梯度。
9. 已知 $n=3$ 、 $\gamma=0.9$ 、连续 reward 为 $[1,1,1]$ 、未终止、三步后的最大目标 Q 值为 5，计算三步 target。
10. 两个动作的 return 分别是“必然为 5”和“0/10 各 50%”。普通 DQN 与 Distributional RL 分别能看见什么？
11. NoisyNet 中 μ_w 、 σ_w 和 ζ 哪些参与学习？它为什么可能比逐步独立随机动作更有持续性？

▼ 参考答案（点击展开）

1. 从 Q-learning 到 DQN：表格更新是 $Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)]$ ，其中 $y = r + \gamma(1-d)\max_{a'} Q(s',a')$ 。用神经网络 $Q(s,a;\theta)$ 替代表格后，把 y 当作固定监督目标，最小化 $L(\theta) = \mathbb{E}[(y - Q(s,a;\theta))^2]$ 或 Huber loss。DQN 使用 target network 计算 $y = r + \gamma(1-d)\max_{a'} Q(s',a';\theta')$ 。
2. **Replay 与 Target Network**：Replay Buffer 随机混合不同时间的 transition，降低连续样本相关性并复用经验；Target Network 是 online 网络的滞后副本，让计算 target 的参数 θ' 在一段时间内保持不变。前者处理数据分布，后者处理移动目标。
3. **DQN target**：下一状态最大 Q 值是 5，因此 $y = 2 + 0.9 \times (1-0) \times 5 = 6.5$ 。如果 $d=1$ ，则 $y=2$ 。
4. **Double DQN**：对多个带噪声 Q 估计取 max 时，容易选中被正噪声高估的动作。基础 DQN 用同一个 target network 同时选动作和估值；Double DQN 用 online network 选择 a ，再用 target network 评价 $Q(s',a;\theta')$ ，降低同一噪声被选择和估值重复放大的程度。
5. **Dueling 数值题**：raw advantage 均值为 $(3+1+2)/3=2$ ，中心化后是 $[1,-1,0]$ 。加上 $V(s)=4$ ，得到 $Q(s,\cdot)=[5,3,4]$ 。
6. **为什么可以组合**：Double DQN 改的是 Bellman target 中动作选择与动作估值的分工；Dueling DQN 改的是 Q 网络内部的 value/advantage 输出结构。二者作用位置不同，因此可以用 Dueling 网络作为 online/target network，同时使用 Double DQN target。

7. **PER 参数**: α_p 控制采样对优先级的偏向, 0 表示均匀采样; β_{IS} 控制重要性采样偏差修正强度; ϵ_p 给所有样本保留正优先级, 避免 TD error 为 0 的样本永远无法再次被采样。
8. **Dueling 梯度**: $Q_I = V + A_I - (A_I + A_2 + A_3)/3$, 所以 Q 对三个 advantage 的导数分别为 $2/3, -1/3, -1/3$ 。均值项依赖所有动作输出, 因此即使当前 transition 执行的是动作 1, 动作 2 和动作 3 也会收到梯度; 三个 advantage 输出方向上的梯度之和为 0, 而每次 forward 仍会重新执行中心化。
9. **三步 target**: $G_t^{(3)} = I + 0.9 + 0.8I + 0.729 \times 5 = 6.355$ 。与单步 target 相比, 它一次直接使用了三个真实 reward。
10. **均值与分布**: 两个动作的期望 return 都是 5, 所以普通 DQN 只看到相同的 $Q=5$ 。Distributional RL 还能表示第一个动作集中在 5、第二个动作分别集中在 0 和 10; 不过标准 C51 若仍按期望选动作, 不会自动产生风险偏好。
11. **NoisyNet 参数**: μ_w 和 σ_w 通过梯度学习, ξ 只随机采样。一次参数噪声会一致地改变整个 Q 函数, 因此探索偏好会依赖状态并协调不同动作; 若同一噪声保持多个环境 step, 还会形成时间上更持续的探索。

第 7 章：Policy Gradient、REINFORCE（经典 Monte Carlo Policy Gradient 算法）与 Advantage

7.1 本章符号说明

符号/缩写	English	中文	含义
$\pi_\theta(a \mid s)$	Parameterized Policy	参数化策略	参数为 θ 的策略分布。
θ	Policy Parameters	策略参数	policy network 的可学习参数。
τ	Trajectory	轨迹	一整段 state-action-reward 序列。
$p_\theta(\tau)$	Trajectory Probability	轨迹概率	策略参数为 θ 时采样到整条轨迹 τ 的概率密度或概率质量。
$p(s_0)$	Initial-state Distribution	初始状态分布	环境开始一条轨迹时产生初始状态 s_0 的概率。
$P(s_{t+1} \mid s_t, a_t)$	Environment Transition	环境转移概率	执行动作后环境从 s_t 转移到 s_{t+1} 的概率，不依赖策略参数 θ 。
$J(\theta)$	Objective Function	目标函数	policy gradient 中要最大化的期望 return。
$R(\tau)$	Trajectory Return	轨迹回报	整条轨迹的累计 reward。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$b(s)$	Baseline	基线函数	用来降低 policy gradient 方差的 action-independent 函数。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_\pi(s, a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s, a) 的长期价值。
$A_\pi(s, a)$	Advantage Function	优势函数	$Q_\pi(s, a) - V_\pi(s)$ ，表示动作比平均水平好多少。
$f_\theta(s)$	Policy Logits	策略 logits	离散动作策略网络在 softmax 之前输出的未归一化分数。
$\mu_\theta(s) \mid \sigma_\theta(s)$	Gaussian Mean / Standard Deviation	高斯均值 / 标准差	连续动作策略在状态 s 输出的分布参数。
$\mathcal{N}(\mu, \sigma)$	Gaussian Distribution	高斯分布	连续动作策略常用的概率分布记号。
α_π	Policy Learning Rate	策略学习率	控制一次 policy gradient 更新的步长。
β_H	Entropy Coefficient	熵系数	控制 entropy regularization 在策略目标中的权重。
$H(\pi)$	Entropy	熵	衡量策略随机性，常用于鼓励探索。
RL	Reinforcement Learning	强化学习	Policy Gradient 是 RL 中直接优化策略的一类方法。
SAC	Soft Actor-Critic	软 Actor-Critic	后续最大熵 actor-critic 算法，本章先介绍 entropy 思想。

7.2 本章目标

本章学习 policy-based RL 的基础。你需要掌握：

- 为什么需要直接优化 policy。
- stochastic policy。
- policy gradient theorem。
- log-derivative trick。
- REINFORCE。
- baseline。
- advantage function。

7.3 本章主线

Value-based 方法先学 $Q(s,a)$ ，再通过 $\arg \max_a Q(s,a)$ 选动作。这个思路在连续动作、随机策略或需要直接控制动作分布时不够自然。本章转向 policy-based RL：直接参数化 $\pi_\theta(a/s)$ ，推导如何对期望 return 求梯度，并用 baseline 和 advantage 降低方差。

7.4 本章新增概念说明

本章从 value-based 转到 policy-based，先定义这些词：

名词	中文	先记住什么
value-based	基于价值的方法	先学 $Q(s,a)$ 或 $V(s)$ ，再用价值选动作。
policy-based	基于策略的方法	直接学习 $\pi_\theta(a/s)$ ，让策略本身输出动作或动作分布。
stochastic policy	随机策略	输出动作概率分布，采样动作；天然带探索。
deterministic policy	确定性策略	给定状态直接输出一个动作。
categorical distribution	类别分布	离散动作空间常用的动作概率分布。
Gaussian policy	高斯策略	连续动作空间常用的动作分布，输出均值和方差。
trajectory	轨迹	一整段 state-action-reward 序列。
log-derivative trick	对数导数技巧	把 ∇p 改写成 $p \nabla \log p$ ，用于对采样分布求梯度。
policy gradient theorem	策略梯度定理	说明如何对期望 return 关于策略参数求梯度。
REINFORCE	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法。
reward-to-go	从当前步开始的回报	只用当前动作之后的 reward 更新当前动作，降低方差。
baseline	基线	从 return 中减掉的 action-independent 参考值，用来降方差。
advantage	优势函数	动作比该状态平均水平好多少，即 $Q(s,a)-V(s)$ 。
entropy regularization	熵正则	鼓励策略保持随机性，避免过早确定。

7.5 Policy Gradient 的动机和策略形式

7.5.1 为什么需要 Policy Gradient

Value-based 方法学习 $Q(s,a)$ ，再通过 $\arg \max_a Q(s,a)$ 选动作。它在离散动作空间中很自然，但在以下场景中不够方便：

- 动作空间连续。
- 策略本身需要随机性。
- 希望直接优化策略分布。
- $\arg \max_a Q(s,a)$ 很难计算。

Policy Gradient 直接参数化策略：

$$\pi_{\theta}(a | s)$$

目标是最大化期望 return：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

其中 trajectory：

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

7.5.2 随机策略

离散动作空间中，策略可以是 categorical distribution：

$$\pi_{\theta}(a | s) = \text{softmax}(f_{\theta}(s))$$

连续动作空间中，策略常用 Gaussian：

$$\pi_{\theta}(a | s) = \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s))$$

随机策略的好处：

- 自带探索。
- 可以表达多模态或不确定行为。
- 适合 policy gradient 的概率形式推导。

7.6 Policy Gradient 推导

7.6.1 Log-derivative Trick

核心恒等式：

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

因为：

$$\nabla_{\theta} \log p_{\theta}(x) = \nabla_{\theta} p_{\theta}(x) / p_{\theta}(x)$$

这个技巧让我们可以对采样分布依赖参数的期望求梯度。

目标：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)]$$

梯度：

$$\begin{aligned} \nabla J(\theta) &= \nabla \int p_{\theta}(\tau) R(\tau) d\tau \\ &= \int \nabla p_{\theta}(\tau) R(\tau) d\tau \\ &= \int p_{\theta}(\tau) \nabla \log p_{\theta}(\tau) R(\tau) d\tau \\ &= \mathbb{E}[\nabla \log p_{\theta}(\tau) R(\tau)] \end{aligned}$$

trajectory 概率：

$$\begin{aligned} p_{\theta}(\tau) &= p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t) \end{aligned}$$

环境转移 P 不依赖 θ ，所以：

$$\begin{aligned} \nabla \log p_{\theta}(\tau) &= \sum_t \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

得到 policy gradient：

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{\tau} [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)] \end{aligned}$$

7.6.2 Policy Gradient 推导阅读线

这段推导最容易觉得“公式突然变形”。可以按下面逻辑理解。

第一步，目标函数是期望：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)]$$

难点在于：被求期望的轨迹分布 $p_{\theta}(\tau)$ 本身依赖策略参数 θ 。参数变了，采样出来的轨迹分布也变了。

第二步，把期望写成积分或求和：

$$J(\theta) = \int p_{\theta}(\tau) R(\tau) d\tau$$

由于 reward 函数本身通常不直接对策略参数求导，梯度主要作用在轨迹概率上：

$$\nabla J(\theta) = \int \nabla p_{\theta}(\tau) R(\tau) d\tau$$

第三步，用 log-derivative trick 把 ∇p 改写成 $p \nabla \log p$ ：

$$\nabla p_{\theta}(\tau) = p_{\theta}(\tau) \nabla \log p_{\theta}(\tau)$$

这样积分又变回期望：

$$\nabla J(\theta) = \mathbb{E}_{\tau} [\nabla \log p_{\theta}(\tau) R(\tau)]$$

第四步，展开轨迹概率：

$$p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

这里环境初始分布和转移概率不由策略参数控制，所以对 θ 求梯度时只剩策略项：

$$\nabla \log p_{\theta}(\tau) = \sum_t \nabla \log \pi_{\theta}(a_t | s_t)$$

最后得到：

$$\nabla J(\theta) = \mathbb{E}_t [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)]$$

一句话记忆：

因为采样动作本身不可直接反传，所以 policy gradient 不对 action 求导，而是对“采到这个 action 的 log probability”求导。高 return 的轨迹会提高其中动作的 log probability，低 return 的轨迹会降低它们。

7.7 REINFORCE 和方差控制

7.7.1 REINFORCE

REINFORCE 是最基础的 Monte Carlo policy gradient 算法。

更新方向：

$$\theta \leftarrow \theta + \alpha_{\pi} \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

直觉：

- 如果某个 action 后续 return 高，就增加该 action 在该 state 下的概率。
- 如果 return 低，就降低该 action 的概率。

算法流程：

```
initialize policy pi_theta
repeat:
    sample trajectories using pi_theta
    compute returns G_t
    update theta by sum_t grad log pi_theta(a_t|s_t) G_t
```

REINFORCE 的优点：

- 概念简单。
- 不需要 value function。
- 可以直接处理随机策略。

缺点：

- 必须等 episode 结束。
- Monte Carlo return 方差很大。
- sample efficiency 较低。

7.7.2 Reward-to-go：为什么 REINFORCE 可以用 G_t

基础推导中，每个时间步都乘整条轨迹回报 $R(\tau)$ 。但动作 a_t 不可能影响 t 之前已经发生的 reward，所以更合理的学习信号是从当前时间开始的 reward-to-go：

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

于是实际常用：

$$\nabla J(\theta) = \mathbb{E}[\sum_t \nabla \log \pi_{\theta}(a_t | s_t) G_t]$$

这样做不会丢掉由 a_t 影响的未来回报，反而去掉了和 a_t 无关的过去 reward，因此方差更低。

顺序上可以这样记：

```
full trajectory return R(tau)
-> reward-to-go G_t
-> G_t - baseline
-> advantage
-> actor-critic / GAE
```

7.7.3 Baseline

为了降低方差，可以从 return 中减去 baseline：

$$\begin{aligned} \nabla J(\theta) \\ = \mathbb{E}[\nabla \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))] \end{aligned}$$

常用 baseline：

$$b(s_t) = V(s_t)$$

为什么 baseline 不引入 bias？

因为：

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a | s) b(s)] = 0$$

推导：

$$\begin{aligned} & \sum_a \pi(a | s) \nabla \log \pi(a | s) b(s) \\ &= b(s) \sum_a \pi(a | s) \nabla \log \pi(a | s) \\ &= b(s) \sum_a \nabla \pi(a | s) \\ &= b(s) \nabla \sum_a \pi(a | s) \\ &= b(s) \nabla 1 \\ &= 0 \end{aligned}$$

只要 baseline 不依赖 action，就不会改变 policy gradient 的期望，只会改变方差。

更直观地说，baseline 只是把“绝对分数”改成“相对分数”：

- 原来：这次 return 是 100，就增加动作概率。
- 加 baseline 后：如果这个状态平均就能拿 120，那么 100 其实低于预期，应该降低动作概率。

这就是 advantage 的来源。

7.7.4 Advantage Function

Advantage 定义：

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

含义：

在状态 s 下，动作 a 比当前策略的平均动作好多少。

使用 advantage 的 policy gradient：

$$\begin{aligned} \nabla J(\theta) \\ = \mathbb{E}[\nabla \log \pi_{\theta}(a_t | s_t) A_{\pi}(s_t, a_t)] \end{aligned}$$

直觉：

- 如果 $A(s, a) > 0$ ，说明动作比平均水平好，增加其概率。
- 如果 $A(s, a) < 0$ ，说明动作比平均水平差，降低其概率。

相比直接用 return，advantage 通常方差更低，学习信号更明确。

7.7.5 从 Baseline 到 Actor-Critic

如果 baseline 取 $V_{\pi}(s)$ ，那么：

$$G_t - V_{\pi}(s_t)$$

就是一个 advantage 的 Monte Carlo 估计。问题是 G_t 仍然要等 episode 结束且方差大。

Actor-Critic 的下一步就是：训练一个 critic 来估计 $V(s)$ 或 $Q(s, a)$ ，用 TD target 更快、更低方差地构造 advantage。也就是说：

REINFORCE：用完整 return 做策略更新
 REINFORCE+b：用 return - baseline 降低方差
 Actor-Critic：用 critic 估计 baseline / advantage
 GAE/PPO：用更稳定的 advantage 和受限策略更新

所以第 7 章和第 8/9 章不是割裂的：后面的 actor-critic、GAE、PPO 都是在让 policy gradient 的学习信号更稳。

7.7.6 Entropy Regularization

为了鼓励探索，常在目标中加入 entropy：

$$J(\theta) = \mathbb{E}[\text{return}] + \beta_H \mathbb{E}[H(\pi_{\theta}(\cdot | s))]$$

策略熵：

$$H(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s)$$

作用：

- 防止策略过早变得确定。
- 增强探索。
- 改善训练稳定性。

SAC 中 entropy regularization 是核心组成部分。

7.8 本章例子：为什么需要直接优化策略

7.8.1 例子 1：连续动作机器人

机器人手臂要输出关节力矩：

```
a = [torque_1, torque_2, torque_3]
```

如果用 DQN，要对连续动作求 $\max_a Q(s,a)$ ，这个优化很难。Policy Gradient 直接学习：

$$\pi_{\theta}(a|s)$$

或确定性策略：

$$a=\mu_{\theta}(s)$$

这样动作可以直接由策略网络输出。

7.8.2 例子 2：Softmax 策略更新

假设在某个状态有两个动作：

动作	当前概率	采样后 return
left	0.4	+5
right	0.6	-1

Policy Gradient 会提高产生高 return 的动作概率，降低低 return 动作概率。

直觉：

采样到 left 且结果好 $\rightarrow$ 增大 $\log \pi(\text{left}|s)$
 采样到 right 且结果差 $\rightarrow$ 减小 $\log \pi(\text{right}|s)$

7.8.3 例子 3：Baseline 的作用

如果一个游戏平均每局 return 是 100：

- 某次动作后 return 是 105，不算特别好，只比平均好 5。
- 另一动作后 return 是 20，明显差。

使用 baseline 后，更新看的是：

$$G_t - V(s)$$

这比直接用 G_t 更能判断动作相对好坏，方差更低。

面试表达：

Policy Gradient 的核心例子是连续控制和随机策略。它不需要枚举动作，也不需要为 Q 做 $\arg\max$ ，而是让高回报轨迹上的动作概率上升。

7.9 本章面试高频问题

7.9.1 Policy Gradient 适合什么场景？

适合连续动作空间、随机策略建模、需要直接优化策略分布的场景。相比 value-based 方法，不需要对动作做 $\arg\max$ 。

7.9.2 REINFORCE 的核心更新是什么？

$$\theta \leftarrow \theta + \alpha_{\pi} \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

如果某个动作带来高 return，就提升该动作概率；如果 return 低，就降低该动作概率。

7.9.3 为什么 baseline 不改变梯度期望？

因为 baseline 不依赖 action，而 $\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s)] = 0$ 。所以减去 baseline 不引入 bias，只降低 variance。

7.9.4 Advantage 的意义是什么？

Advantage 衡量某个动作相对于该状态平均动作的好坏。它比直接用 return 更能反映动作的相对贡献，通常降低方差并稳定训练。

7.9.5 Policy Gradient 的主要缺点？

方差较大、sample efficiency 较低、训练对学习率敏感。REINFORCE 尤其需要完整 episode，训练慢且不稳定。

7.10 本章练习

1. 手写 log-derivative trick 推导。
2. 证明 action-independent baseline 不引入 bias。
3. 解释 $Q_{\pi}(s,a)$ 、 $V_{\pi}(s)$ 、 $A_{\pi}(s,a)$ 的关系。
4. 对比 value-based 和 policy-based 方法。

▼ 参考答案（点击展开）

1. **Log-derivative trick**: 由 $\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$ ，可得 $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)] = \nabla_{\theta} \int p_{\theta}(x) f(x) dx = \mathbb{E}_{x \sim p_{\theta}}[\nabla_{\theta} \log p_{\theta}(x) f(x)]$ 。把 x 换成 trajectory τ ，再利用环境转移不依赖 policy 参数，可得到 policy gradient 中的 $\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 。
2. **Baseline 不引入 bias**: 若 $b(s)$ 不依赖 action，则 $\mathbb{E}_{a \sim \pi}[\nabla_{\theta} \log \pi_{\theta}(a|s) b(s)] = b(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) = b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a|s) = 0$ 。因此减去 baseline 不改变梯度期望，只改变 variance。
3. **三者关系**: $Q_{\pi}(s,a)$ 是在 s 先做 a 后的期望 return； $V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s,a)$ 是该状态下按 policy 行动的平均价值； $A_{\pi}(s,a) = Q_{\pi}(s,a) - V_{\pi}(s)$ 表示动作相对平均水平好多少。
4. **Value-based vs policy-based**: value-based 方法学习 Q 并通过 greedy/epsilon-greedy 导出 policy，离散动作下高效，但连续动作的 $\arg \max_a Q(s,a)$ 很难；policy-based 方法直接学习动作分布，天然支持随机策略和连续动作，但梯度 variance 较大、通常 sample efficiency 较低。Actor-Critic 用 critic 的价值估计帮助 policy-based 方法降方差。

第 8 章：Actor-Critic、A2C（Advantage Actor-Critic，优势 Actor-Critic）、A3C（Asynchronous Advantage Actor-Critic，异步优势 Actor-Critic）与 GAE（Generalized Advantage Estimation，广义优势估计）

8.1 本章符号说明

符号/缩写	English	中文	含义
A2C	Advantage Actor-Critic	优势 Actor-Critic	同步采样和更新的 actor-critic 方法。
A3C	Asynchronous Advantage Actor-Critic	异步优势 Actor-Critic	多 worker 异步更新的 actor-critic 方法。
GAE	Generalized Advantage Estimation	广义优势估计	用 λ 在 TD 和 MC advantage 之间折中。
$\pi_{\theta}(a s)$	Actor Policy	Actor 策略	actor 网络输出的动作分布。
$V_w(s)$	Critic Value Function	Critic 状态价值函数	critic 网络对状态价值的估计。
$Q_w(s, a)$	Critic Action-value Function	Critic 动作价值函数	critic 网络对状态-动作价值的估计。
$V_{\pi}(s) / Q_{\pi}(s, a)$	True Value Functions under π	策略 π 下的真实价值函数	推导 advantage 时使用的理论状态价值与动作价值。
$A_{\pi}(s, a)$	Advantage Function	优势函数	策略 π 下动作价值相对状态价值的差，即 $Q_{\pi}(s, a) - V_{\pi}(s)$ 。
$J(\theta)$	Policy Objective	策略目标函数	actor 要最大化的目标。
δ_t	TD Error	时序差分误差	可作为一步 advantage 估计。
A_t	Advantage Estimate	优势估计	策略更新时衡量动作好坏的信号。
L_{policy} / L_{value}	Policy Loss / Value Loss	策略损失 / 价值损失	actor 和 critic 的训练损失。
V_t^{target}	Value Target	价值目标	用 return 或 bootstrap 构造、用于训练 critic 的监督目标。上标 target 用来区别当前预测 $V_w(s_t)$ 。
$R_t^{(n)}$	n-step Return	n 步回报	n-step bootstrap target。
γ	Discount Factor	折扣因子	控制未来 reward 和 value 的权重。
λ	GAE Lambda	GAE 衰减参数	控制 bias 和 variance 的折中。
α_{π} / α_V	Actor / Critic Learning Rate	Actor / Critic 学习率	分别控制策略参数 θ 和价值参数 w 的更新步长。
β_H	Entropy Coefficient	熵系数	控制 entropy bonus 的强度。
c_v	Value-loss Coefficient	价值损失系数	控制 value loss 在总 loss 中的权重。
on-policy	On-policy Learning	同策略学习	数据来自当前正在更新的策略。
TD	Temporal Difference	时序差分	critic 常用 TD error 做估计。

符号/缩写	English	中文	含义
MC	Monte Carlo	蒙特卡洛	GAE 的另一端近似完整 return。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	Actor-Critic 要改进的高方差 policy gradient 基线。
PPO	Proximal Policy Optimization	近端策略优化	常结合 GAE 使用的策略优化算法。

8.2 本章目标

本章学习 Actor-Critic 框架。你需要掌握：

- actor 和 critic 的职责。
- Actor-Critic 如何结合 policy gradient 和 value learning。
- TD error 如何作为 advantage 估计。
- A2C 和 A3C。
- Generalized Advantage Estimation。

8.3 本章主线

第 7 章的纯 policy gradient 可以直接优化策略，但 Monte Carlo return 方差高。第 3 章的 TD value learning 方差低但主要解决估值。本章把两者合并：actor 负责更新策略，critic 负责估计 value 或 advantage。GAE 则进一步在 MC 和 TD 之间调节 bias/variance。

8.4 本章新增概念说明

Actor-Critic 的词容易混在一起，先按角色定义：

名词	中文	先记住什么
actor	行动者 / 策略网络	负责输出动作或动作分布。
critic	评价者 / 价值网络	负责估计 $V(s)$ 、 $Q(s,a)$ 或 advantage，给 actor 提供学习信号。
actor-critic	Actor-Critic 框架	actor 学策略，critic 学价值，用 critic 降低 policy gradient 方差。
TD error	时序差分误差	一步 target 和当前价值估计的差，可近似 advantage。
A2C	Advantage Actor-Critic	同步采样和更新的 actor-critic 方法。
A3C	Asynchronous Advantage Actor-Critic	多 worker 异步采样和更新的 actor-critic 方法。
n-step return	n 步回报	用未来 n 步真实 reward 加 bootstrap target 构造学习信号。
GAE	广义优势估计	把多个步数的 TD residual 加权平均，平衡 bias 和 variance。
shared network / separate network	共享网络 / 分离网络	actor 和 critic 共用一部分网络，或各自用独立网络。
stop gradient	停止梯度	计算 actor loss 时把 advantage 当作固定学习信号，不让 actor 的梯度反向修改 advantage 的构造过程。

8.5 Actor-Critic 基础

8.5.1 Actor-Critic 基本思想

Actor-Critic 有两个部分：

- Actor：策略网络 $\pi_\theta(a|s)$ ，负责选动作。
- Critic：价值网络 $V_w(s)$ 或 $Q_w(s,a)$ ，负责评估动作或状态。

Actor 用 critic 提供的学习信号更新策略：

$$\begin{aligned} & \nabla_\theta J(\theta) \\ &= \mathbb{E}[\nabla \log \pi_\theta(a|s) A_\pi(s,a)] \end{aligned}$$

Critic 学习价值函数：

$$V_w(s) \approx V_\pi(s)$$

直觉：

Actor 负责做决策，critic 负责评价决策质量。Critic 降低了纯 Monte Carlo policy gradient 的方差，让策略更新更稳定。

8.5.2 TD Error 作为 Advantage

TD error：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

它可以看成一步 advantage 的估计：

$$A(s_t, a_t) \approx \delta_t$$

直觉：

- 如果实际 reward 加下一状态价值高于当前价值估计，说明这个动作比预期好。
- 如果低于当前价值估计，说明这个动作比预期差。

Actor 更新：

$$\theta \leftarrow \theta + \alpha_\pi \nabla \log \pi_\theta(a_t | s_t) \delta_t$$

Critic 更新：

$$w \leftarrow w - \alpha_V \nabla (r_t + \gamma V_w(s_{t+1}) - V_w(s_t))^2$$

8.5.3 为什么 TD Error 可以近似 Advantage

Advantage 的定义是：

$$A_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t)$$

而动作价值可以拆成“一步 reward + 下一状态价值”：

$$Q_\pi(s_t, a_t) = \mathbb{E}[r_t + \gamma V_\pi(s_{t+1})]$$

把它代入 advantage:

$$A_{\pi}(s_p, a_p) = \mathbb{E}[r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_p)]$$

如果用一次采样近似这个期望，就得到：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_p)$$

所以 TD error 不是随便定义的误差，它正是“一步采样下的 advantage 估计”。

面试里可以这样讲：

如果执行动作后得到的 reward 加下一状态价值超过当前状态价值，说明这个动作比平均动作好，advantage 为正；反之说明动作低于预期，advantage 为负。

8.5.4 Actor-Critic vs REINFORCE

维度	REINFORCE	Actor-Critic
策略更新信号	Monte Carlo return	critic 估计的 advantage
是否 bootstrap	否	通常是
方差	高	较低
bias	较低	有 critic bias
是否能在在线更新	通常需要 episode 结束	可以在线或 n-step 更新
sample efficiency	较低	更高

面试表达：

Actor-Critic 用 critic 估计 value 或 advantage，降低 policy gradient 的方差，并允许 TD 式在线更新。但 critic 是函数逼近模型，因此会引入估计 bias。

8.6 A2C / A3C：Actor-Critic 的采样组织方式

8.6.1 A2C

A2C 是 Advantage Actor-Critic。

目标通常包含三部分：

1. Policy loss。
2. Value loss。
3. Entropy bonus。

Policy loss：

$$L_{policy} = -\log \pi_{\theta}(a_t | s_t) A_t$$

Value loss：

$$L_{value} = (V_t^{target} - V_w(s_t))^2$$

Entropy bonus:

$$L_{entropy} = -\beta_H H(\pi_{\theta}(\cdot | s_t))$$

整体最小化:

$$L = L_{policy} + c_v L_{value} + L_{entropy}$$

注意符号: 很多实现中是最小化 loss, 因此 policy gradient 的最大化目标会写成负号。

8.6.2 A3C

A3C 是 Asynchronous Advantage Actor-Critic。

核心思想:

- 多个 worker 并行和环境交互。
- 每个 worker 异步更新共享全局网络。
- 不同 worker 产生的数据降低样本相关性。

A3C 在 replay buffer 不常用的 on-policy 场景中, 通过并行采样改善数据多样性。这里 on-policy 指数据来自当前正在更新的策略, 因此不能像 DQN 那样随意复用很旧的数据。

A2C 可以看作 A3C 的同步版本:

- A3C: asynchronous。
- A2C: synchronous, 多个环境同步采样后统一更新。

8.7 多步回报与 GAE

8.7.1 N-step Return

一步 TD target:

$$R_t^{(1)} = r_t + \gamma V(s_{t+1})$$

n-step return:

$$R_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$$

n-step return 在 MC 和 TD 之间折中:

- n 小: bias 较大, variance 较小。
- n 大: bias 较小, variance 较大。

8.7.2 Generalized Advantage Estimation

GAE 用一个参数 λ 在不同步数的 advantage 估计之间折中。

TD residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE:

$$A_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

直觉：

- $\lambda = 0$ ：接近一步 TD，bias 大、variance 小。
- $\lambda = 1$ ：接近 Monte Carlo，bias 小、variance 大。
- 实践中常用 $\lambda = 0.95$ 。

PPO 中经常使用 GAE 估计 advantage。

8.7.3 GAE 的推理过程

一步 TD error 方差小，但依赖 critic，bias 可能大。完整 Monte Carlo return bias 小，但方差大。GAE 的想法是：不要只选一个步数，而是把不同步数的 TD residual 加权平均。

从一步 TD residual 开始：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

如果下一步也持续“比预期好”，那么 δ_{t+1} 也应该影响当前动作的评价，只是影响要打折：

$$\delta_t + \gamma \lambda \delta_{t+1}$$

继续往后累加，就得到：

$$A_t^{GAE} = \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots$$

也就是：

$$A_t^{GAE} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

这里 γ 是时间折扣， λ 是“愿意看多远的误差信号”：

- λ 小：更相信短期 bootstrap，更新稳但 bias 大。
- λ 大：更接近长回报，bias 小但 variance 大。

实践里 PPO 常用 $\lambda=0.95$ ，是因为它通常在稳定性和学习信号之间比较均衡。

8.8 Actor-Critic 网络结构选择

8.8.1 Shared Network vs Separate Network

Actor 和 critic 可以：

- 共享一部分 backbone，分别输出 policy head 和 value head。
- 完全分离成两个网络。

共享 backbone 的优点：

- 参数更少。
- 表征可以复用。

风险：

- policy loss 和 value loss 的梯度可能相互干扰。
- 需要调节 value loss coefficient。

8.9 本章例子：Actor 和 Critic 分工

8.9.1 例子 1：游戏 agent

在一个游戏中：

- Actor 负责选择动作，例如移动、攻击、防御。
- Critic 负责评价当前局面值多少钱。

如果 Actor 做了一个动作后，Critic 发现：

$$r + \gamma V(s') - V(s) > 0$$

说明这步结果比预期好，Actor 应该增加这个动作在该状态下的概率。

如果 TD error 小于 0，则说明这步比预期差，Actor 应该降低该动作概率。

8.9.2 例子 2：A2C 和 REINFORCE 对比

REINFORCE 要等一整局结束，用完整 G_t 更新策略。A2C 用 critic 提供一步或多步 target，可以更早更新。

```
REINFORCE target:  $G_t$ 
A2C target:  $r + \gamma V(s')$ 
```

所以 A2C 通常方差更低，但 critic 不准时会带来 bias。

8.9.3 例子 3：GAE 的直觉

如果只用一步 TD error，更新稳定但可能短视。

如果用完整 return，信息完整但方差大。

GAE 用 λ 在中间插值：

- λ 接近 0：更像一步 TD。
- λ 接近 1：更像 Monte Carlo。

面试表达：

Actor-Critic 可以理解成 policy gradient 加一个会估值的助手。Critic 不直接决定动作，而是告诉 Actor 这次动作比预期好还是差。

8.10 本章面试高频问题

8.10.1 Actor-Critic 中 actor 和 critic 分别做什么？

Actor 学习策略并负责选动作。Critic 学习价值函数，用来评价 actor 的动作，并为 policy gradient 提供低方差的 advantage 估计。

8.10.2 Actor-Critic 相比 REINFORCE 的优势？

Actor-Critic 用 critic 的 value estimate 替代完整 Monte Carlo return，降低方差，并支持在线或 n-step 更新，sample efficiency 更高。

8.10.3 TD error 为什么可以作为 advantage？

TD error 衡量执行动作后的一步结果是否比当前价值估计更好。如果 $r + \gamma V(s') > V(s)$ ，说明动作带来的结果超出预期，可以增加其概率。

8.10.4 A2C 和 A3C 的区别？

A3C 使用多个 worker 异步更新全局网络。A2C 是同步版本，多个环境同步采样后统一更新，工程上更稳定、易并行。

8.10.5 GAE 解决什么问题？

GAE 在 bias 和 variance 之间做平滑折中，通过 λ 控制 advantage 估计更接近一步 TD 还是 Monte Carlo return。

8.11 本章练习

1. 写出 Actor-Critic 的 actor loss 和 critic loss。
2. 解释 TD error 和 advantage 的关系。
3. 推导 n-step return。
4. 说明 GAE 中 λ 的作用。

▼ 参考答案 (点击展开)

1. **Actor 和 critic loss**: 一种常见写法是 $L_{actor} = -\mathbb{E}[\log \pi_{\theta}(a_t | s_t, A_t)]$, $L_{critic} = \mathbb{E}[(V_{\pi}(s_t) - V_t^{target})^2]$ 。工程上常组合为 $L = L_{actor} + c_v L_{critic} - \beta_H H(\pi_{\theta}(\cdot | s_t))$; 计算 actor loss 时通常对 A_t stop gradient。
2. **TD error 与 advantage**: $\delta_t = r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 。给定 (s_t, a_t) 取条件期望后, $\mathbb{E}[\delta_t | s_t, a_t] = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t) = A_{\pi}(s_t, a_t)$ 。所以单步 TD error 是 advantage 的有噪声样本估计。
3. **N-step return**: 连续展开 return 递归 n 次, 得到 $G_t^{(n)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$ 。若在 n 步内终止, 则终止后的 bootstrap value 为 0。
4. λ 的作用: GAE 使用 $A_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$ 。 λ 小时更接近一步 TD, bias 较大但 variance 较小; λ 接近 1 时更接近长回报, bias 较小但 variance 较大。它是 bias-variance tradeoff 的旋钮。

第 9 章：TRPO（Trust Region Policy Optimization，信赖域策略优化）、PPO（Proximal Policy Optimization，近端策略优化）与策略更新约束

9.1 本章符号说明

符号/缩写	English	中文	含义
TRPO	Trust Region Policy Optimization	信赖域策略优化	用 KL 约束限制策略更新幅度。
PPO	Proximal Policy Optimization	近端策略优化	用 clipping objective 近似限制策略更新幅度。
KL	Kullback-Leibler Divergence	KL 散度	衡量新旧策略分布差异。
θ / θ_{old}	New / Old Policy Parameters	新 / 旧策略参数	当前要优化的策略参数和采样数据时的旧参数。
$r_t(\theta)$	Probability Ratio	概率比	新策略和旧策略对同一动作的概率比。
A_t	Advantage Estimate	优势估计	判断当前动作应提高还是降低概率的信号。
$L^{CLIP}(\theta)$	Clipped Objective	截断目标函数	PPO 用来限制策略更新的 surrogate objective。
ϵ_{clip}	Clipping Range	截断范围	PPO 中控制 ratio 允许变化范围的超参数；不是 epsilon-greedy 的探索率。
η	Policy Learning Rate	策略学习率	控制一次策略参数更新的步长。
δ_{KL}	Trust-region Radius	信赖域半径	TRPO 对平均 KL divergence 设置的上界。
L_{policy} / L_{value}	Policy Loss / Value Loss	策略损失 / 价值损失	PPO 总 loss 的两个主要部分。
$V(s)$	Critic Value Estimate	Critic 价值估计	PPO 的 value head 对状态 s 的价值预测。
V_t^{target}	Value Target	价值目标	用 return 或 bootstrap 构造、用于训练 critic 的监督目标。上标 target 用来区别当前预测 $V(s_t)$ 。
c_v / c_e	Value / Entropy Coefficient	价值 / 熵系数	分别控制 value loss 和 entropy bonus 在总 loss 中的权重。
$H(\pi)$	Entropy	熵	鼓励策略保持探索的正则项。
behavior policy	Behavior Policy	行为策略	产生当前 rollout 数据的旧策略。
target policy	Target Policy	目标策略	当前正在优化的新策略。
on-policy	On-policy Learning	同策略学习	主要用当前策略或近似当前策略采样的数据更新。
off-policy	Off-policy Learning	异策略学习	用其他策略产生的数据学习目标策略。
CLIP	Clipped Objective Label	截断目标标记	L^{CLIP} 中的上标，表示 PPO 的 clipped surrogate objective。

符号/缩写	English	中文	含义
GAE	Generalized Advantage Estimation	广义优势估计	PPO 中常用的 advantage 估计方法。
DQN	Deep Q-Network	深度 Q 网络	PPO 对比中的 value-based deep RL 算法。
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	PPO 既用于 RL，也常出现在 RLHF 流程中。

9.2 本章目标

本章学习现代 on-policy deep RL 中最常用的一类算法。你需要掌握：

- 为什么 policy gradient 需要限制更新步长。
- importance sampling ratio。
- TRPO 的 trust region 思想。
- PPO clipping objective。
- PPO 的训练细节。
- PPO 的优缺点。

9.3 本章主线

Actor-Critic 解决了 policy gradient 高方差问题，但策略更新仍可能一步走太大，导致新策略和采样数据对应的旧策略差异过大。本章专门处理“更新幅度控制”：TRPO 用 trust region 和 KL 约束，PPO 用 clipping objective 做更易实现的近似。

9.4 本章新增概念说明

本章核心是“限制策略更新幅度”，先把 PPO/TRPO 相关名词定义清楚：

名词	中文	先记住什么
old policy	旧策略	采样 rollout 时使用的策略。
new policy	新策略	当前正在优化、准备替换旧策略的策略。
behavior policy	行为策略	在 PPO 里通常就是采样数据的 old policy。
target policy	目标策略	当前想优化得到的 new policy。
KL divergence	KL 散度	衡量新旧策略分布差异，越大说明策略变化越大。
trust region	信赖域	限制策略每次只能在可信范围内小步更新。
importance sampling ratio	重要性采样比率	新策略概率除以旧策略概率，用旧数据估计新策略目标。
surrogate objective	替代目标	用已有 rollout 近似优化真实策略目标的目标函数。
clipping	截断	限制 ratio 过大或过小，防止策略更新过猛。
entropy bonus	熵奖励	鼓励策略保持一定随机性，避免过早收敛。
advantage normalization	优势归一化	把 advantage 标准化，让训练尺度更稳定。

9.5 为什么要约束策略更新

9.5.1 为什么要限制策略更新

Policy Gradient 直接更新策略参数：

$$\theta \leftarrow \theta + \text{eta} \nabla J(\theta)$$

但策略优化很敏感。如果一次更新太大，新的 policy 和采样数据对应的旧 policy 差异过大，可能导致性能崩掉。

原因：

- on-policy 数据只对采样它的旧策略可靠。
- 策略变化太大时，旧数据不能准确估计新策略表现。
- neural policy 的参数小变化可能导致动作分布大变化。

因此 TRPO/PPO 都试图限制新旧策略距离。

9.5.2 Importance Sampling Ratio

PPO/TRPO 常用 ratio：

$$r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

它表示在同一个 (s_t, a_t) 上，新策略相对于旧策略的概率变化。

Policy objective 可写成：

$$L(\theta) = \mathbb{E}[r_t(\theta) A_t]$$

如果 $A_t > 0$ ，希望增加该动作概率，即增大 ratio。

如果 $A_t < 0$ ，希望降低该动作概率，即减小 ratio。

9.5.3 Ratio 从哪里来

PPO 的数据是旧策略 $\pi_{\theta_{old}}$ 采样出来的，但我们想评估新策略 π_θ 。如果完全重新采样，成本很高；如果直接把旧数据当新数据，又会有分布偏差。

importance sampling 的基本想法是：

$$\mathbb{E}_{a \sim \pi_\theta} [f(a)] = \mathbb{E}_{a \sim \pi_{old}} [\pi_\theta(a | s) / \pi_{old}(a | s) f(a)]$$

所以出现 ratio：

$$r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

它把“旧策略采到的动作”校正成“新策略下这个动作有多可能”。但 ratio 如果太大或太小，说明新旧策略差异太大，旧数据对新策略的估计就不可靠。这就是后面 clipping 的动机。

9.6 Trust Region 与 PPO 方法

9.6.1 TRPO

TRPO 是 Trust Region Policy Optimization。

它希望在提升 surrogate objective 的同时，限制新旧策略的 KL divergence：

$$\begin{aligned} & \max_{\theta} \mathbb{E}[r_t(\theta) A_t] \\ & \text{subject to } \mathbb{E}[KL(\pi_{old}(\cdot | s_t) \parallel \pi_\theta(\cdot | s_t))] \leq \delta_{KL} \end{aligned}$$

核心思想：

每次策略更新不能离旧策略太远，保证更新在 trust region 内。

优点：

- 理论上更稳定。
- 避免灾难性大步更新。

缺点：

- 实现复杂。
- 需要二阶近似、conjugate gradient、line search。
- 工程成本较高。

9.6.2 PPO

PPO 是 Proximal Policy Optimization。它保留了限制策略更新幅度的思想，但实现更简单。

PPO clipped objective：

$$\begin{aligned} L^{CLIP}(\theta) \\ = \mathbb{E}[\min(r_t(\theta) A_t, \\ \text{clip}(r_t(\theta), 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) A_t)] \end{aligned}$$

其中：

$$r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

ϵ_{clip} 常用 0.1 或 0.2。

9.6.3 PPO Clipping 直觉

当 $A_t > 0$ ：

- 增大动作概率是好事。
- 但 ratio 超过 $1 + \epsilon_{clip}$ 后，收益被截断。
- 防止过度增加该动作概率。

当 $A_t < 0$ ：

- 降低动作概率是好事。
- 但 ratio 低于 $1 - \epsilon_{clip}$ 后，收益被截断。
- 防止过度降低该动作概率。

一句话：

PPO clipping 允许策略朝 advantage 指示的方向更新，但限制更新幅度，避免新旧策略差异过大。

9.6.4 Clipping 分情况推理

PPO 的 min 不是为了让公式复杂，而是为了“好方向可以走，但不要走太猛”。

9.6.4.1 情况 1: $A_t > 0$

动作比平均水平好，应该提高它的概率。未截断目标是：

$$r_t A_t$$

当 r_t 从 1 增大时，目标变大，表示鼓励新策略更常选这个动作。但如果 $r_t > 1 + \epsilon_{clip}$ ，说明概率已经提高太多。此时 clipped 项固定在：

$$(1 + \epsilon_{clip}) A_t$$

再提高概率也不会得到更多收益，梯度被压住。

9.6.4.2 情况 2: $A_t < 0$

动作比平均水平差，应该降低它的概率。因为 A_t 是负数，减小 r_t 会让 $r_t A_t$ 变大，看起来是好事。但如果 $r_t < 1 - \epsilon_{clip}$ ，说明概率已经降低太多。clipped 项固定在：

$$(1 - \epsilon_{clip}) A_t$$

PPO 取 min，会阻止继续通过过度降低概率来“刷目标”。

因此 clipped objective 同时限制两种过度更新：

- 好动作概率不能一下子加太多。
- 坏动作概率不能一下子降太多。

这就是 PPO 比普通 policy gradient 更稳的核心。

9.6.5 PPO Loss 组成

常见 PPO 总 loss：

$$L = L_{policy} + c_v L_{value} - c_e Entropy$$

如果按最小化写：

$$L_{policy} = - \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) A_t)]$$

Value loss：

$$L_{value} = \mathbb{E}[(V(s_t) - V_t^{target})^2]$$

Entropy bonus：

$$Entropy = \mathbb{E}[H(\pi(\cdot | s_t))]$$

实际实现中还可能有的：

- advantage normalization。
- value clipping。
- gradient clipping。
- target KL early stopping。
- learning rate annealing。

9.6.6 PPO 训练流程

```

for each iteration:
    collect rollout using pi_old
    compute returns and advantages, often with GAE
    for several epochs:
        shuffle rollout data
        split into minibatches
        update policy with clipped objective
        update value function
    set pi_old <- current policy

```

注意：

PPO 是 on-policy，但它会对同一批 rollout 做多个 epoch 的 minibatch 更新。clipping 机制就是为了让这种有限复用不至于让策略偏离太远。

9.6.7 PPO 一轮训练的推理顺序

从实现角度，PPO 一轮迭代可以按下面理解：

1. 用当前策略采样 rollout，此时这份数据绑定的是 π_{old} 。
2. 用 reward 和 critic 计算 return。
3. 用 GAE 计算 advantage，得到每个动作“比平均水平好多少”。
4. 保存旧策略对已采样动作的 log probability。
5. 更新时重新计算新策略 log probability。
6. 用新旧 log probability 的差得到 ratio。
7. 用 clipped objective 更新 actor。
8. 用 return target 更新 critic。
9. 用 entropy bonus 防止策略过早塌缩。

这里最关键的是第 4 到第 7 步：PPO 不是只看当前策略输出，而是一直比较“新策略相对采样时旧策略改了多少”。

9.7 PPO 使用场景和对比

9.7.1 PPO 为什么常用

优点：

- 实现比 TRPO 简单。
- 稳定性好。
- 对超参数相对不那么敏感。
- 离散和连续动作都能用。
- 成为很多 RLHF 早期流程中的常见选择。

缺点：

- on-policy，sample efficiency 不如 off-policy 方法。
- 需要大量环境交互。
- 对 reward scaling、advantage normalization 仍然敏感。
- clipping 不是严格 trust region，仍可能出现不稳定。

9.7.2 PPO vs DQN

维度	DQN	PPO
类型	value-based	policy-based / actor-critic
动作空间	离散	离散和连续

维度	DQN	PPO
策略	由 Q argmax 导出	直接学习 policy
数据使用	off-policy	on-policy
样本效率	较高	较低
稳定性	需要 replay 和 target network	clipping + GAE 较稳定

9.8 本章例子：PPO clipping 怎么工作

假设某个动作在旧策略下概率是 0.20，新策略下概率是 0.26。

importance ratio 是：

$$r_t = 0.26 / 0.20 = 1.3$$

如果 $\epsilon_{clip} = 0.2$ ，允许范围是：

$$[0.8, 1.2]$$

9.8.1 Advantage 为正

如果 $A_t > 0$ ，说明这个动作比预期好，应该提高概率。但 ratio 已经到 1.3，超过 1.2，PPO 会把收益截断在 1.2，防止策略一步变太多。

9.8.2 Advantage 为负

如果 $A_t < 0$ ，说明这个动作不好，应该降低概率。但如果新策略把概率降得过猛，ratio 低于 0.8，也会被 clip。

9.8.3 RLHF 例子

在 RLHF 里，策略是语言模型，动作可以理解成生成 token。PPO 更新时：

- reward model 给回答打分。
- critic 估计 value。
- advantage 判断这段输出比预期好还是差。
- KL 或 clipping 防止模型偏离参考模型太远。

面试表达：

PPO 的例子要讲清楚 ratio。它不是禁止策略更新，而是限制同一批数据上新旧策略概率比变化过大，避免 policy gradient 一步把策略推崩。

9.9 本章面试高频问题

9.9.1 PPO 解决了什么问题？

PPO 解决普通 policy gradient 更新过大导致策略崩掉的问题。它通过 clipping objective 限制新旧策略概率比的变化，使策略更新更稳定。

9.9.2 PPO 的 ratio 是什么？

$$r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{old}(a_t | s_t)$$

它衡量新策略相对于采样数据的旧策略，对同一动作概率改变了多少。

9.9.3 PPO clipping objective 的直觉?

如果 advantage 为正, 允许提高动作概率, 但超过 $1 + \epsilon_{clip}$ 后不再给额外收益。如果 advantage 为负, 允许降低动作概率, 但低于 $1 - \epsilon_{clip}$ 后不再给额外收益。

9.9.4 PPO 和 TRPO 的区别?

TRPO 通过 KL constraint 显式限制策略更新, 理论更严谨但实现复杂。PPO 用 clipping 或 KL penalty 近似 trust region 思想, 实现简单, 工程上更常用。

9.9.5 PPO 是 on-policy 还是 off-policy?

PPO 通常是 on-policy。虽然它会对同一批 rollout 做多个 epoch 更新, 但数据仍来自当前旧策略, 且通过 ratio 和 clipping 控制偏移。

9.10 本章练习

1. 手写 PPO clipped objective。
2. 分别解释 $A_t > 0$ 和 $A_t < 0$ 时 clipping 的效果。
3. 对比 TRPO 和 PPO。
4. 说明 PPO 中 GAE、entropy bonus、advantage normalization 的作用。

▼ 参考答案 (点击展开)

1. **PPO clipped objective:** $L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon_{clip}, 1 + \epsilon_{clip})A_t)]$, 其中 $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{old}(a_t | s_t)$ 。实现成最小化 loss 时通常在该目标前加负号。
2. **Clipping 的两种情况:** 当 $A_t > 0$, 该动作比平均好, 算法想提高其概率; 若 ratio 超过 $1 + \epsilon_{clip}$, 收益被截住, 避免提高过多。当 $A_t < 0$, 算法想降低该动作概率; 若 ratio 低于 $1 - \epsilon_{clip}$, 目标被截住, 避免一次把概率压得过低。 $\min$ 与 advantage 的符号共同形成这个效果。
3. **TRPO vs PPO:** TRPO 显式约束平均 KL divergence, 并用二阶近似和共轭梯度求解, 理论动机清晰但实现复杂。PPO 用 probability ratio clipping 或 KL penalty 近似限制更新, 可用普通 minibatch SGD, 工程实现更简单。二者通常都是 on-policy。
4. **三个组件的作用:** GAE 在 advantage 估计的 bias 和 variance 间折中; entropy bonus 防止 policy 过早变得确定、维持探索; advantage normalization 让一个 batch 内的更新尺度更稳定, 降低学习率对 reward 量纲的敏感性, 但它不改变 advantage 的相对排序。

第 10 章：连续控制、DDPG (Deep Deterministic Policy Gradient, 深度确定性策略梯度)、TD3 (Twin Delayed DDPG, 双延迟 DDPG) 与 SAC (Soft Actor-Critic, 软 Actor-Critic)

10.1 本章符号说明

符号/缩写	English	中文	含义
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	用双 Q、延迟 actor 更新和 target smoothing 改进 DDPG。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 off-policy actor-critic 算法。
$Q_w(s, a)$	Critic Action-value Function	Critic 动作价值函数	参数 w 下的 Q 估计。
$\mu_\theta(s)$	Deterministic Actor	确定性 actor	DDPG/TD3 中 actor 输出的连续动作。
$\pi_\theta(a s)$	Stochastic Actor	随机 actor	SAC 中 actor 输出的动作分布。
w^- / θ^-	Target Critic / Actor Parameters	目标 critic / actor 参数	从在线参数滞后更新得到、用于构造稳定 target 的参数；上标 - 表示 target network。
Q_i^- / μ^-	Target Critic / Target Actor	目标 Q 网络 / 目标 actor	TD3/SAC 中的 target 网络简写； i 是 twin critic 的编号。
(s, a, r, s', d)	Transition Tuple	转移样本五元组	replay buffer 中的当前状态、动作、奖励、下一状态和终止指示量。
d / done	Terminal Indicator	终止指示量	$d=1$ 时没有后续价值， $d=0$ 时可以从 s' bootstrap。
D	Replay-buffer Distribution	回放数据分布	公式 $s \sim D$ 表示从 replay buffer 的样本状态中取期望。
γ	Discount Factor	折扣因子	控制下一状态 soft value 或 Q value 的权重。
y	Bellman Target	Bellman 目标	critic 训练时的目标值。
$J(\theta)$	Actor Objective	Actor 目标函数	actor 要最大化或等价最小化的目标。
Q_1 / Q_2	Twin Q Networks	双 Q 网络	TD3/SAC 中用于缓解过估计的两个 critic。
$H(\pi)$	Entropy	熵	SAC 目标中鼓励策略随机性的项。
α	Temperature	温度系数	SAC 中 reward 和 entropy 的权衡参数。
ϵ_t	Exploration Noise	探索噪声	DDPG/TD3 与环境交互时加到确定性动作上的噪声。

符号/缩写	English	中文	含义
ϵ_{clip}	Clipped Target Noise	截断目标噪声	TD3 构造 target action 时加入并限制幅度的平滑噪声；不是 PPO 的 clipping range。
off-policy	Off-policy Learning	异策略学习	可以用 replay buffer 中旧策略产生的数据更新当前策略。
target policy smoothing	Target Policy Smoothing	目标策略平滑	TD3 中给 target action 加噪声，避免 critic 学到尖锐错误峰值。
DQN	Deep Q-Network	深度 Q 网络	离散动作空间的 value-based 算法，本章用它引出连续动作困难。

10.2 本章目标

本章学习连续动作空间中的经典 off-policy actor-critic 算法。你需要掌握：

- 连续动作为什么不适合普通 DQN。
- DDPG。
- TD3。
- SAC。
- entropy regularization。
- deterministic policy vs stochastic policy。

10.3 本章主线

第 6 章的 DQN 依赖离散动作枚举，第 9 章的 PPO 是 on-policy，样本效率相对低。连续控制通常既需要 actor 直接输出连续动作，又希望 off-policy 复用 replay buffer。本章围绕这个目标讲 DDPG、TD3、SAC：它们都是连续动作下的 off-policy actor-critic，只是稳定性和探索处理不同。

10.4 本章新增概念说明

连续控制章节的核心是“动作不能枚举”，相关名词先放这里：

名词	中文	先记住什么
continuous action	连续动作	动作是连续数值，例如力矩、速度、角度，不能像 DQN 那样枚举。
deterministic actor	确定性 actor	输入状态后直接输出一个连续动作。
stochastic actor	随机 actor	输出动作分布，再从分布中采样动作。
critic	评价网络	估计 $Q(s,a)$ ，告诉 actor 哪些动作价值高。
DDPG	深度确定性策略梯度	用 deterministic actor 处理连续动作的 off-policy actor-critic。
TD3	双延迟 DDPG	通过双 Q、延迟 actor 更新、target policy smoothing 稳定 DDPG。
SAC	软 Actor-Critic	用最大熵目标训练 stochastic actor，探索更稳定。
exploration noise	探索噪声	DDPG/TD3 中加到动作上的噪声，用来探索连续动作空间。

名词	中文	先记住什么
target actor / target critic	目标 actor / 目标 critic	滞后更新的 actor/critic，用来构造更稳定的 target。
clipped double Q	截断双 Q	两个 critic 取较小值，缓解过估计。
delayed policy update	延迟策略更新	critic 多更新几次后再更新 actor，让评价更可靠。
target policy smoothing	目标策略平滑	给 target action 加小噪声，让 critic 不过拟合尖锐 Q 峰值。
maximum entropy objective	最大熵目标	同时最大化 reward 和策略熵，让策略保持探索。

10.5 连续动作问题和 Actor 思路

10.5.1 连续动作空间的挑战

DQN 需要计算：

$$\max_a Q(s', a)$$

如果动作空间是离散且规模不大，可以枚举所有动作。
但连续动作空间无法枚举，例如机器人控制中的力矩、速度、角度。

因此需要直接学习一个 actor 输出动作：

$$a = \mu_{\theta}(s)$$

或者输出动作分布：

$$a \sim \pi_{\theta}(\cdot | s)$$

10.5.2 从 DQN 到 Actor 的推理过程

DQN 在离散动作中可以做：

$$a^* = \arg \max_a Q(s, a)$$

连续动作下，这个 maximization 本身就是一个困难优化问题。如果每次决策都在动作空间里搜索最大 Q，代价很高，也不稳定。

DDPG/TD3/SAC 的思路是额外训练一个 actor，让 actor 学会直接输出“critic 认为高价值”的动作：

```
critic: 评价 Q(s, a)
actor: 产生让 Q(s, a) 尽量大的动作
```

所以连续控制里的 actor 不是装饰，它是在替代 DQN 中对动作的显式枚举或 argmax。

10.6 DDPG：确定性 Actor-Critic

10.6.1 DDPG

DDPG 是 Deep Deterministic Policy Gradient。

它适用于连续动作空间，是 off-policy actor-critic。

组件：

- Actor: 确定性策略 $\mu_\theta(s)$ 。
- Critic: 动作价值函数 $Q_w(s,a)$ 。
- Replay buffer。
- Target actor。
- Target critic。

Critic target:

$$y = r + \gamma(1-d) Q_w(s', \mu_\theta(s'))$$

这里 d 是 replay transition 的终止指示量: $d=1$ 时 target 只剩即时奖励 r ; $d=0$ 时才加入下一状态的 bootstrap value。

Critic loss:

$$L(w) = \mathbb{E}[(Q_w(s,a) - y)^2]$$

Actor objective:

$$J(\theta) = \mathbb{E}[Q_w(s, \mu_\theta(s))]$$

Actor 更新:

$$\begin{aligned} & \nabla_\theta J \\ &= \mathbb{E}[\nabla_a Q_w(s,a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)] \end{aligned}$$

这个梯度来自 chain rule。目标是让:

$$J(\theta) = Q_w(s, \mu_\theta(s))$$

变大。参数 θ 先影响 actor 输出的动作 $\mu_\theta(s)$, 动作再影响 critic 给出的 $Q_w(s,a)$ 。所以更新方向分两段:

- $\nabla_a Q_w(s,a)$: critic 告诉 actor, 动作往哪个方向改能提高 Q 。
- $\nabla_\theta \mu_\theta(s)$: actor 告诉参数, 怎么改参数能让动作往那个方向移动。

面试表达:

DDPG 用 critic 对连续动作求梯度, 再通过 actor 反传到策略参数, 相当于用 actor 学一个可微的 argmax。

探索:

DDPG 的策略是 deterministic, 因此训练时通常给动作加噪声:

$$a_t = \mu_\theta(s_t) + \varepsilon_t$$

10.6.2 DDPG 的问题

DDPG 比较脆弱, 常见问题:

- Q 值过估计。
- 对超参数敏感。
- exploration noise 难调。
- actor 可能利用 critic 的错误高估。
- 训练容易不稳定。

TD3 就是针对 DDPG 的几个问题做改进。

10.7 TD3：修正 DDPG 的过估计和不稳定

10.7.1 TD3

TD3 是 Twin Delayed DDPG。

核心改进：

1. Clipped Double Q-learning。
2. Delayed policy update。
3. Target policy smoothing。

10.7.1.1 Clipped Double Q-learning

TD3 学两个 critic：

$$Q_1(s, a), Q_2(s, a)$$

target 使用两者较小值：

$$y = r + \gamma(1-d) \min_i Q_i^-(s', a')$$

这样可以缓解 overestimation bias。

10.7.1.2 Delayed Policy Update

Critic 更新多次后，actor 再更新一次。

直觉：

critic 估计更可靠后，再用它指导 actor，减少 actor 被错误 Q 值带偏。

10.7.1.3 Target Policy Smoothing

target action 加入 clipped noise：

$$a' = \mu^-(s') + \epsilon_{clip}$$

这里 ϵ_{clip} 通常由零均值高斯噪声截断得到， a' 还会再限制到环境合法动作范围；然后用 a' 计算 target。

直觉：

让 critic 不要对动作空间中的尖锐错误峰值过拟合，使 Q 函数更平滑。

10.7.2 TD3 三个改进的共同逻辑

TD3 的三个 trick 都围绕同一个问题：actor 很容易利用 critic 的错误。

改进	解决的问题	推理
双 Q 取最小	Q 过估计	如果一个 critic 偶然高估，取 min 可以更保守。
延迟 actor 更新	actor 跟着错误 critic 跑	先多训几步 critic，让评价器更靠谱，再更新 actor。
target smoothing	critic 对尖锐动作峰值过拟合	target 动作附近加噪声，让 Q 在局部更平滑。

一句话：

TD3 不是三个孤立技巧，而是在防止 deterministic actor 钻 critic 估计误差的空子。

10.8 SAC：最大熵 Actor-Critic

10.8.1 SAC

SAC 是 Soft Actor-Critic，是现代连续控制中非常常用的算法。

核心思想：

最大化 reward 的同时最大化策略 entropy。

目标：

$$\mathbb{E}[\sum_t \gamma^t (r_t + \alpha H(\pi(\cdot|s_t)))]$$

或者写成：

$$\mathbb{E}[\sum_t \gamma^t (r_t - \alpha \log \pi(a_t|s_t))]$$

其中 α 是 temperature，控制 entropy 权重。

10.8.2 SAC 的组件

SAC 通常包含：

- Stochastic actor: $\pi_\theta(a|s)$ 。
- 两个 Q network: Q_1, Q_2 。
- target Q networks。
- replay buffer。
- temperature parameter α 。

Soft Q target：

$$y = r + \gamma(1-d) [\min_i Q_i^-(s', a') - \alpha \log \pi(a'|s')]$$

其中：

$$a' \sim \pi_\theta(\cdot|s')$$

Actor objective：

$$J_\pi(\theta) = \mathbb{E}[\alpha \log \pi_\theta(a|s) - \min_i Q_i(s,a)]$$

最小化这个 loss 等价于选择高 Q 且高 entropy 的动作分布。

10.8.3 SAC Soft Target 的推理过程

普通 Bellman target 是：

$$y=r+\gamma Q(s',a')$$

SAC 的目标不是只最大化 reward，而是最大化 reward 加 entropy。对某个动作来说，entropy 项可以写成：

$$-\alpha \log \pi(a'|s')$$

所以 soft value 可以理解为：

$$Q(s',a')-\alpha \log \pi(a'|s')$$

代回 Bellman target，就得到：

$$y=r+\gamma(1-d)[\min_i Q_i^-(s',a')-a\log\pi(a'|s')]$$

为什么 actor loss 是：

$$J_{\pi}(\theta)=\mathbb{E}[a\log\pi_{\theta}(a|s)-\min_i Q_i^-(s,a)]$$

因为实现里通常最小化 loss。最小化这个式子等价于：

- 最大化 Q：让动作回报高。
- 最大化 entropy：因为最小化 $\log\pi$ 倾向避免策略过度确定。

面试里可以说：

SAC 把探索写进目标函数，而不是只靠外加噪声。Soft target 中的 $-a\log\pi$ 会奖励更高熵的动作分布，因此策略既追求高 Q，也保持足够随机性。

10.8.4 Entropy Regularization 的意义

SAC 不只是把 entropy 当作训练 trick，而是把它放进目标函数。

好处：

- 鼓励探索。
- 避免策略过早确定。
- 对多模态最优策略更友好。
- 训练通常比 DDPG/TD3 更稳定。

α 控制探索程度：

- α 大：更重视 entropy，策略更随机。
- α 小：更重视 reward，策略更确定。

现代 SAC 常自动调节 α ，让策略 entropy 接近 target entropy。

10.9 连续控制算法对比和选型

10.9.1 DDPG vs TD3 vs SAC

维度	DDPG	TD3	SAC
策略	deterministic	deterministic	stochastic
数据	off-policy	off-policy	off-policy
critic	单 Q	双 Q	双 Q
探索	外加噪声	外加噪声	entropy 驱动
稳定性	较弱	比 DDPG 强	通常更强
适用	连续控制	连续控制	连续控制、鲁棒训练

10.9.2 本章例子：连续控制怎么选算法

10.9.2.1 例子 1：机械臂控制

机械臂每个关节输出连续力矩：

```
a = [torque_shoulder, torque_elbow, torque_wrist]
```

DQN 不适合，因为动作不是离散的。DDPG、TD3、SAC 可以直接输出连续动作。

10.9.2.2 例子 2：DDPG 的问题

如果 critic 对某个连续动作附近估值过高，actor 会被 critic 引导到这个错误高值区域。

这就是 DDPG 容易不稳定的原因：

```
critic 估错 -> actor 追错 -> 数据分布变差 -> critic 更难学准
```

10.9.2.3 例子 3：TD3 怎么修

TD3 用两个 critic：

$$\min(Q_1, Q_2)$$

如果一个 critic 过高估计，另一个没那么高，取 min 可以降低过估计传播。

10.9.2.4 例子 4：SAC 为什么适合探索

如果机器人需要学会走路，早期策略不能太确定，否则很容易卡在局部动作模式。SAC 的 entropy 项鼓励策略保持随机性：

```
既要拿高 reward，也要保持足够探索
```

面试表达：

DDPG、TD3、SAC 都处理连续动作。DDPG 是基础 actor-critic，TD3 主要修过估计和不稳定，SAC 用最大熵目标把探索写进优化目标。

10.10 本章面试高频问题

10.10.1 为什么 DQN 不适合连续动作？

DQN 需要对动作枚举并取 $\max_a Q(s, a)$ 。连续动作空间无法枚举，直接求 $\arg\max$ 也通常很难，因此需要 actor 直接输出连续动作。

10.10.2 DDPG 的核心是什么？

DDPG 是连续动作空间下的 off-policy actor-critic。Actor 输出确定性动作，critic 评估 $Q(s, a)$ ，actor 通过最大化 critic 给出的 Q 值来更新。

10.10.3 TD3 相比 DDPG 做了哪些改进？

TD3 使用双 critic 取最小值缓解过估计，延迟 actor 更新提高稳定性，并使用 target policy smoothing 平滑 Q 函数。

10.10.4 SAC 为什么稳定？

SAC 使用最大熵目标，鼓励高 reward 和高 entropy 的策略；同时使用双 Q 网络、target network 和 replay buffer。Entropy regularization 带来更好的探索和鲁棒性。

10.10.5 SAC 中 α 是什么？

α 是 entropy temperature，控制 reward 和 entropy 的权衡。 α 越大，策略越随机、探索越强； α 越小，策略越偏向最大化 reward。

10.11 本章练习

1. 写出 DDPG critic target 和 actor objective。
2. 解释 TD3 的三个 trick。
3. 写出 SAC soft Q target。
4. 对比 deterministic policy 和 stochastic policy。

▼ 参考答案 (点击展开)

1. **DDPG 目标**: critic target 为 $y=r+\gamma(1-d)Q_w(s',\mu_{\theta}(s'))$, critic 最小化 $\mathbb{E}[(Q_w(s,a)-y)^2]$; actor 最大化 $J(\theta)=\mathbb{E}_{s\sim D}[Q_w(s,\mu_{\theta}(s))]$, 等价地最小化其负值。 w, θ 是 target network 参数, d 是 terminal indicator。
2. **TD3 三个 trick**: Clipped Double Q-learning 取两个 target critic 的较小值, 缓解过估计; Delayed Policy Update 降低 actor 和 target network 的更新频率, 让 critic 先学稳; Target Policy Smoothing 在 target action 上加截断噪声, 抑制 policy 利用狭窄的 Q 函数错误峰值。
3. **SAC soft Q target**: $y=r+\gamma(1-d)[\min_{i=1,2} Q_i^-(s',a')-a\log\pi_{\theta}(a'|s')]$, 其中 $a'\sim\pi_{\theta}(\cdot|s')$ 。减去 $a\log\pi$ 等价于加入 entropy reward。
4. **确定性 vs 随机策略**: deterministic policy 对每个 state 输出单一 action, 执行高效, DDPG/TD3 通常另外加 exploration noise; stochastic policy 输出 action distribution, 自带探索并能表示多峰或不确定行为, SAC/PPO 属于这一类, 但需要处理采样和 log-probability。确定性策略适合动作输出稳定的控制, 随机策略在探索和鲁棒性上通常更自然。

第 11 章：Model-based RL（Model-based Reinforcement Learning，基于模型的强化学习）、Offline RL（离线强化学习）、Imitation Learning 与 RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）

11.1 本章符号说明

符号/缩写	English	中文	含义
RL	Reinforcement Learning	强化学习	通过交互或数据学习策略以最大化长期 reward。
Offline RL	Offline Reinforcement Learning	离线强化学习	只用固定数据集学习策略，不继续和环境交互。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model，再优化 policy。
$P(s' s, a)$	Transition Probability	状态转移概率	环境 dynamics 的概率形式。
$R(s, a)$	Reward Function	奖励函数	环境或 reward model 给出的奖励。
D	Dataset	数据集	offline RL 或 imitation learning 使用的固定样本集合。
$f_{\varphi}(s, a)$	Dynamics Model	动力学模型	参数 φ 下预测下一状态的模型。
$P_{\varphi}(s' s, a)$	Learned Transition Model	学得转移模型	参数 φ 下对真实环境转移概率 $P(s' s, a)$ 的近似。
$\mu(a s)$	Behavior Policy	行为策略	产生离线数据集 D 的策略，可来自旧系统、人类或多个策略的混合。
d / <code>done</code>	Terminal Indicator	终止指示量	离线 transition 是否到达真正终止状态； $d=1$ 时后续 bootstrap value 为 0。
a_{expert}	Expert Action	专家动作	模仿学习数据中专家在状态 s 给出的动作标签。
θ	Policy Parameters	策略参数	Behavior Cloning 或 RL 优化中当前 policy 的可学习参数。
$\sigma(x)$	Logistic Sigmoid	Logistic Sigmoid 函数	将实数差值映射到 $(0, 1)$ ，在 pairwise reward loss 中表示偏好概率。
$r_{\text{chosen}} / r_{\text{rejected}}$	Chosen / Rejected Reward Score	被选 / 被拒回答的奖励分数	reward model 给一对回答输出的标量分数。
L_{RM}	Reward-model Loss	奖励模型损失	用 chosen/rejected 分数差训练 pairwise preference 的损失。
π / π_{ref}	Current / Reference Policy	当前 / 参考策略	RLHF 中正在优化的 policy，以及用于约束它不要偏离太远的固定参考模型。
$r_{RM} / r_{\text{total}}$	RM / Total Reward	奖励模型分数 / 总奖励	PPO 优化时的原始 reward model 分数，以及加入 KL 惩罚后的训练奖励。

符号/缩写	English	中文	含义
β_{KL}	KL-penalty Coefficient	KL 惩罚系数	控制偏离参考策略的惩罚强度。
BC	Behavior Cloning	行为克隆	把专家示范当监督学习来模仿。
IRL	Inverse Reinforcement Learning	逆强化学习	从专家行为反推出 reward function。
SFT	Supervised Fine-Tuning	监督微调	RLHF 中用人工示范数据微调语言模型。
RM	Reward Model	奖励模型	根据偏好数据预测 response 质量的模型。
DPO	Direct Preference Optimization	直接偏好优化	不显式跑 PPO 的偏好优化方法。
KL	Kullback-Leibler Divergence	KL 散度	RLHF 中常用来限制 policy 偏离 reference model。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	文中作为 model-free 或 RLHF 优化阶段的代表算法。
BCQ	Batch-Constrained deep Q-learning	批约束深度 Q 学习	offline RL 代表算法之一。
BEAR	Bootstrapping Error Accumulation Reduction	bootstrap 误差累积降低	offline RL 代表算法之一。
CQL	Conservative Q-Learning	保守 Q 学习	通过保守估计 Q 值处理 OOD action 的 offline RL 方法。
IQL	Implicit Q-Learning	隐式 Q 学习	不显式查询 OOD action 的 offline RL 方法。
OOD	Out-of-Distribution	分布外	数据集中缺少覆盖的状态或动作。

11.2 本章目标

本章补充面试中越来越常见的进阶主题。你需要掌握：

- model-free vs model-based RL。
- offline RL 的核心难点。
- imitation learning 和 behavior cloning。
- inverse reinforcement learning。
- RLHF 的基本流程。

11.3 本章主线

前 10 章主要讨论在线 model-free 学习。本章换成四类现实约束：能不能学或使用环境模型，能不能继续交互，能不能从专家示范学，能不能用人类偏好定义 reward。Model-based RL、Offline RL、Imitation Learning、RLHF 分别对应这些约束，是面试里从经典 RL 扩展到真实系统和大模型对齐的关键桥梁。

11.4 本章新增概念说明

这一章主题最多，先把容易混的名词集中定义：

名词	中文	先记住什么
model-free RL	无模型强化学习	不显式学习环境模型，直接学 value 或 policy。

名词	中文	先记住什么
model-based RL	基于模型的强化学习	学习或使用环境 dynamics/reward model，再用模型 planning 或 rollout。
dynamics model	动力学模型	预测执行动作后的下一状态，例如 $s_{t+l} \approx f_{\phi}(s_t, a_t)$ 。
planning	规划	在模型里搜索或评估未来动作序列。
model bias	模型偏差	学到的模型不准，规划时会放大误差。
offline RL	离线强化学习	只用固定历史数据集学习策略，不继续探索收集新数据。
behavior policy	行为策略	产生离线数据的旧策略、人类策略或混合策略。
OOD action	分布外动作	历史数据中很少出现或没出现过的动作。
extrapolation error	外推误差	critic 对没数据覆盖的动作估值不可靠。
imitation learning	模仿学习	从专家数据中学习行为。
behavior cloning	行为克隆	把专家动作当监督学习标签来模仿。
compounding error	误差累积	模型犯小错后进入未见状态，后续错误继续放大。
DAgger	数据集聚合	让专家给 learner 实际访问到的状态继续标注，缓解分布偏移。
IRL	逆强化学习	不直接模仿动作，而是从专家行为反推 reward。
reward model	奖励模型	根据偏好或标注预测 response/action 质量的模型。
preference data	偏好数据	人类对两个或多个 response 的好坏比较。
reference model	参考模型	RLHF 中用来约束新 policy 不要偏离太远的原始模型。
reward hacking	奖励黑客	policy 利用 reward model 漏洞拿高分，但真实质量变差。
DPO	直接偏好优化	直接用偏好数据优化 policy，不显式跑 PPO。

11.5 进阶主题总览

11.5.1 四类进阶主题的边界

这一章主题多，先用边界表把它们分开：

主题	核心问题	数据来源	主要风险
Model-based RL	能不能学一个环境模型来 planning	在线交互数据或模拟器	model bias 和长 horizon 误差累积
Offline RL	能不能只用历史数据学新策略	固定日志数据集	OOD action 和 extrapolation error
Imitation Learning	能不能不设计 reward，直接模仿专家	专家状态-动作数据	compounding error
RLHF	能不能用人类偏好构造 reward	prompt-response 偏好比较	reward hacking、偏好噪声、KL 约束

它们共同点是：真实交互昂贵、危险或不充分，因此不能只靠“在线试错 + model-free 更新”解决。

11.5.2 Model-free vs Model-based

Model-free RL 不显式学习环境模型，直接学习 value 或 policy：

- Q-learning。
- DQN。
- PPO。
- SAC。

Model-based RL 显式学习或使用环境模型：

$$P(s' | s, a), R(s, a)$$

然后基于模型进行 planning 或生成模拟数据。

11.6 Model-based RL

11.6.1 Model-based RL 的流程

典型流程：

1. 收集环境交互数据。
2. 学习 dynamics model：

$$s_{t+1} \approx f_{\phi}(s_t, a_t)$$

3. 用模型进行 planning 或 rollout。
4. 根据真实环境反馈继续修正模型。

优点：

- sample efficiency 可能更高。
- 可以通过模型进行规划。
- 在真实交互昂贵的场景中有吸引力。

缺点：

- 模型误差会累积。
- 长 horizon rollout 容易偏离真实环境。
- 高维复杂环境中 dynamics 很难学准。

面试表达：

Model-based RL 的核心优势是样本效率，但主要问题是 model bias。学到的 dynamics model 有误差，规划或 rollout 会放大这些误差，尤其在长时序任务中更明显。

11.7 Offline RL

11.7.1 Offline RL

Offline RL，也叫 batch RL，目标是在不继续和环境交互的情况下，只用固定数据集学习策略。

数据集：

$$D = \{(s, a, r, s', d)\}$$

每条记录包含当前状态 s 、动作 a 、奖励 r 、下一状态 s' 和终止标记 d 。这些数据来自某个 behavior policy $\mu(a|s)$ ，可能是人类、旧系统或混合策略。

与 online RL 的区别：

- online RL 可以探索并收集新数据。
- offline RL 只能使用已有数据。

11.7.2 Offline RL 的难点

核心问题：distribution shift。

学习到的新策略可能选择数据集中很少出现甚至没出现过的动作。此时 Q 函数对这些 out-of-distribution actions 的估计不可靠。

如果 Q 函数错误高估了某些 OOD 动作，policy improvement 会进一步偏向这些动作，导致性能崩溃。

这也叫 extrapolation error。

面试表达：

Offline RL 难在不能通过真实环境纠错。策略可能选择数据分布之外的动作，而 critic 对这些动作的 Q 值没有可靠监督，容易产生过估计并被 policy 利用。

11.7.3 Offline RL 的常见思路

常见方法会让策略不要偏离数据分布太远，或让 Q 值对 OOD 动作更保守。

几类思路：

- Behavior regularization：限制 learned policy 接近 behavior policy。
- Conservative value estimation：压低 OOD action 的 Q 值。
- Importance weighting：修正策略分布差异。
- Sequence modeling：把决策问题转成条件序列建模，例如 Decision Transformer。

代表算法了解即可：

- BCQ。
- BEAR。
- CQL。
- IQL。
- Decision Transformer。

11.8 Imitation Learning

11.8.1 Imitation Learning

Imitation Learning 从专家数据中学习策略。

专家数据：

$$(s, a_{expert})$$

目标：让 agent 模仿专家动作。

11.8.2 Behavior Cloning

Behavior Cloning，简称 BC，是最简单的 imitation learning。

把专家数据当监督学习：

$$\max_{\theta} \log \pi_{\theta}(a_{expert} | s)$$

或者最小化：

$$L(\theta) = -\log \pi_{\theta}(a_{\text{expert}} / s)$$

优点：

- 简单。
- 稳定。
- 不需要 reward。
- 训练像监督学习。

缺点：

- distribution shift。
- compounding error。
- 遇到专家数据中没覆盖的状态时容易出错。

为什么会 compounding error?

训练时模型只见过专家状态分布。部署时一旦模型犯小错，进入专家没覆盖的状态，就更容易继续犯错，误差逐步累积。

11.8.3 DAgger

DAgger 是 Dataset Aggregation。

核心思路：

1. 用当前策略和环境交互。
2. 让专家给当前策略访问到的状态标注正确动作。
3. 把新数据加入数据集。
4. 重新训练策略。

DAgger 通过收集 learner 自己会遇到的状态，缓解 behavior cloning 的分布偏移。

11.8.4 Inverse Reinforcement Learning

IRL 的目标不是直接模仿动作，而是从专家行为中反推 reward function。

直觉：

专家之所以这么行动，是因为它在优化某个隐藏 reward。IRL 希望恢复这个 reward，然后再用 RL 学策略。

适合场景：

- 专家动作可观察。
- reward 很难手工设计。
- 希望学到可迁移的偏好或目标。

缺点：

- 问题不适定：多个 reward 都可能解释同一行为。
- 计算复杂。
- 实践中常被更直接的 imitation 或 preference learning 替代。

11.9 RLHF

11.9.1 RLHF

RLHF 是 Reinforcement Learning from Human Feedback，常用于大语言模型对齐。

典型流程：

1. Pretraining：在大规模语料上预训练基础模型。

2. SFT：用人工示范数据做 supervised fine-tuning。
3. Reward Model：收集人类偏好比较，训练 reward model。
4. RL Optimization：用 PPO 等算法优化 policy，使其获得更高 reward model 分数。

偏好数据形式：

$$(prompt, response_{chosen}, response_{rejected})$$

Reward model 学习：

$$r_{chosen} > r_{rejected}$$

常见 pairwise loss：

$$L_{RM} = -\log \sigma(r_{chosen} - r_{rejected})$$

11.9.2 RLHF 中的 PPO

在 RLHF 中，policy 是语言模型。动作是 token，trajectory 是生成序列。

优化目标通常包含：

- reward model 分数。
- KL penalty，限制新模型不要偏离 reference model 太远。

形式：

$$r_{total} = r_{RM} - \beta_{KL} KL(\pi \parallel \pi_{ref})$$

为什么需要 KL penalty？

- 防止模型为了骗 reward model 而偏离语言质量。
- 保持和 SFT/reference model 的行为接近。
- 提高训练稳定性。

11.9.3 RLHF 的常见问题

- Reward hacking：模型找到 reward model 漏洞。
- Preference data 噪声。
- Reward model 泛化问题。
- PPO 训练不稳定。
- 对齐目标难以完全由标注偏好表达。

近年也常见不用 RL 的偏好优化方法，例如 DPO。面试中可以简单提到：

DPO 直接从偏好数据推导监督式目标，避免显式训练 reward model 后再跑 PPO，工程上更简单。

11.10 本章例子：四类高级 RL 场景

11.10.1 例子 1：Model-based RL 做机器人规划

机器人先学习一个 dynamics model：

$$P_{\phi}(s'|s,a)$$

然后在模型里模拟不同动作序列：

动作序列 1 -> 预测会撞墙
 动作序列 2 -> 预测能到目标点
 动作序列 3 -> 预测路径更短但风险高

再选择预测回报最高的动作序列执行。

11.10.2 例子 2：Offline RL 做历史推荐日志

公司已经有大量推荐日志：

用户状态、展示内容、点击/停留/购买

Offline RL 希望只用这些历史数据学习新策略。但难点是新策略可能推荐历史日志里很少出现的内容，价值估计容易外推错误。

所以 offline RL 常需要保守约束：不要让策略偏离数据分布太远。

11.10.3 例子 3：Imitation Learning 做驾驶

Behavior Cloning 可以用人类驾驶数据训练：

输入：摄像头、速度、导航
 输出：方向盘角度、油门、刹车

问题是模型一旦偏离人类数据分布，就可能进入训练集中没见过的状态。DAgger 的思路是让专家在模型实际访问到的状态上继续标注，补齐分布偏移。

11.10.4 例子 4：RLHF 做聊天模型

RLHF pipeline：

1. 用 SFT 让模型学会基本回答格式。
2. 收集同一个 prompt 下多个回答的人类偏好。
3. 训练 reward model。
4. 用 PPO 等方法优化策略，同时用 KL 限制模型偏离参考模型。

面试表达：

Model-based、Offline、Imitation、RLHF 的共同点是都在解决“真实交互成本高或风险大”的问题：要么学模型模拟，要么复用历史数据，要么模仿专家，要么用人类偏好构造 reward。

11.11 本章面试高频问题

11.11.1 Model-free 和 model-based RL 区别？

Model-free 不显式学习环境模型，直接学习 policy 或 value。Model-based 学习或使用环境 dynamics 和 reward model，并基于模型 planning 或生成数据。

11.11.2 Model-based RL 的主要问题是什么？

模型误差。学到的 dynamics model 不可能完全准确，长 horizon planning 会累积误差，导致策略利用模型漏洞而在真实环境中表现差。

11.11.3 Offline RL 为什么难？

因为只能使用固定数据集，不能通过环境探索纠错。策略可能选择数据分布之外的动作，而 Q 函数对这些动作估计不可靠，容易产生 extrapolation error。

11.11.4 Behavior Cloning 的问题?

BC 把模仿学习当监督学习，简单稳定，但有 distribution shift 和 compounding error。模型部署后遇到专家数据没覆盖的状态，错误会逐步累积。

11.11.5 RLHF 的基本流程?

先预训练语言模型，再用示范数据 SFT，然后用人类偏好训练 reward model，最后用 PPO 等 RL 算法在 KL 约束下优化模型输出。

11.12 本章练习

1. 用一句话区分 online RL 和 offline RL。
2. 解释 offline RL 中的 extrapolation error。
3. 对比 Behavior Cloning 和 RL。
4. 画出 RLHF 的四阶段流程。

▼ 参考答案 (点击展开)

1. **Online RL vs Offline RL**: Online RL 可以继续与环境交互并通过探索改变数据分布; Offline RL 只能使用固定数据集学习, 不能在线试错, 因此必须控制对数据分布外动作的错误估值。
2. **Extrapolation error**: 数据集中很少或从未出现的 (s,a) 上, function approximator 的 Q 估计缺少监督。Policy improvement 又会偏向选择被偶然高估的分布外动作, 随后 bootstrap 把错误继续传播, 造成性能崩溃。常见缓解思路是保守 Q 估计、行为约束或只在数据支持区域内优化。
3. **Behavior Cloning vs RL**: Behavior Cloning 把专家 (s,a) 当监督数据, 直接最大化专家 action 的 likelihood, 简单稳定但会有 covariate shift, 也无法主动超过示范; RL 根据 reward 和环境交互优化长期 return, 需要探索和 credit assignment, 但可以发现示范中没有的更优行为。Imitation Learning 还包括 DAgger、IRL 等, 不等同于只做 BC。
4. 经典 RLHF 四阶段:

```
pretrained language model
-> supervised fine-tuning on demonstrations
-> preference data + reward model training
-> policy optimization with reward and KL constraint
```

第四阶段经典做法是 PPO; 实际系统也可能采用 DPO 等直接偏好优化方法, 此时不显式训练 reward model 并运行在线 PPO。

第 12 章：强化学习面试速查与项目准备

12.1 本章符号说明

本章是速查表，会集中复现前面章节的核心符号：

符号/缩写	English	中文	含义
MDP	Markov Decision Process	马尔可夫决策过程	RL 标准建模。
$S / A / P / R$	State / Action / Transition / Reward	状态 / 动作 / 转移 / 奖励	MDP 的状态空间、动作空间、环境转移和奖励函数。
γ	Discount Factor	折扣因子	控制未来 reward 在 return 或 Bellman target 中的权重。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s,a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s,a) 的长期价值。
$V(s) / Q(s,a)$	Optimal Value Functions	最优价值函数	在所有策略中可达到的最优状态价值和动作价值。
$A_{\pi}(s,a)$	Advantage Function	优势函数	动作相对状态平均价值的优势；这里的 A 不是 action space。
A_t	Advantage Estimate	优势估计	PPO 等算法在样本时刻 t 使用的 advantage 估计值。
δ_t	TD Error	时序差分误差	一步 TD target 与当前价值估计的差。
$\rho_{\theta}(\theta)$	PPO Probability Ratio	PPO 概率比	新策略与旧策略对样本动作的概率之比；特意用 ρ 避免和 reward r_t 混淆。
ϵ_{clip}	PPO Clipping Range	PPO 截断范围	限制 PPO probability ratio 的变化幅度；不是 epsilon-greedy 的探索率。
y	Bellman Target	Bellman 目标	DQN 或 SAC 中用于训练 critic/Q 网络的目标值。
Q_i^{-}	Target Twin Q Network	目标双 Q 网络	SAC target 中第 i 个滞后更新的 critic。
α_{SAC}	SAC Temperature	SAC 温度系数	控制 reward 和 entropy 在 SAC 目标中的权衡。
d	Terminal Indicator	终止指示量	$d=1$ 时 transition 真正终止，target 不再 bootstrap。
$J(\theta)$	Objective Function	目标函数	policy optimization 中要最大化的目标。
$L(\theta)$	Loss Function	损失函数	神经网络训练中要最小化的目标。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个策略分布的差异。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	面试中最高频的三类 deep RL 算法。

符号/缩写	English	中文	含义
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	本套课程和对齐流程中的核心概念。
DP / MC / TD	Dynamic Programming / Monte Carlo / Temporal Difference	动态规划 / 蒙特卡洛 / 时序差分	价值估计和控制的基础方法。
behavior policy / target policy	Behavior Policy / Target Policy	行为策略 / 目标策略	前者负责采样数据，后者是算法真正想学习的策略。
on-policy / off-policy	On-policy / Off-policy Learning	同策略 / 异策略学习	同策略表示采样策略和学习目标一致；异策略表示用一个策略的数据学习另一个策略。
$\epsilon_{explore}$	Exploration Probability	探索概率	epsilon-greedy 中随机探索的概率。
epsilon-greedy	Epsilon-greedy Policy	epsilon-greedy 策略	以 $\epsilon_{explore}$ 概率随机探索，否则选当前最优动作。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	使用实际下一动作更新的 TD control 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	Monte Carlo policy gradient 基线算法。
A2C / A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic	优势 Actor-Critic / 异步优势 Actor-Critic	actor-critic 的同步和异步版本。
GAE	Generalized Advantage Estimation	广义优势估计	PPO 常用的 advantage 估计方法。
DDPG / TD3	Deep Deterministic Policy Gradient / Twin Delayed DDPG	深度确定性策略梯度 / 双延迟 DDPG	连续动作控制算法。
UCB	Upper Confidence Bound	置信上界	bandit 中基于不确定性 bonus 的探索方法。
MLP	Multi-Layer Perceptron	多层感知机	CartPole DQN 项目中常用的小型神经网络。
PG	Policy Gradient	策略梯度	直接优化策略参数的一类方法。

12.2 本章目标

本章用于面试前快速复习。你需要整理：

- 核心概念速查。
- 常见算法横向对比。
- 高频公式。
- 高频问答。
- 项目表达模板。

12.3 本章主线

本章不是引入新算法，而是把前面所有内容压缩成面试表达结构。复习顺序是：先用一张图串起 MDP、Bellman、DP、MC/TD、DQN、Policy Gradient、Actor-Critic、PPO/SAC；再横向比较算法；最后把项目按“问题建模、状态动作奖励、算法选择、训练稳定性、指标和失败案例”讲清楚。

12.4 本章使用方式

本章原则上不引入新算法名词，只做复习压缩。读本章时如果遇到陌生词，先回到对应章节：

陌生词类型	回看章节
MDP、Bellman、return、value	第 1 章
DP、Policy Iteration、Value Iteration	第 2 章
MC、TD、SARSA、Q-learning、on-policy/off-policy	第 3 章
epsilon-greedy、UCB、Thompson Sampling、regret	第 4 章
function approximation、Deadly Triad、replay、target network	第 5-6 章
Policy Gradient、REINFORCE、baseline、advantage	第 7 章
Actor-Critic、A2C/A3C、GAE	第 8 章
TRPO、PPO、ratio、clipping、KL	第 9 章
DDPG、TD3、SAC、entropy	第 10 章
Offline RL、Imitation、RLHF、DPO	第 11 章

12.5 速查和高频问答

12.5.1 核心概念速查

MDP：

$$(S, A, P, R, \gamma)$$

Return：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Value：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Q value：

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Advantage：

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

Bellman expectation：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

Bellman optimality：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') | S_t = s, A_t = a]$$

TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Policy Gradient:

$$\nabla J(\theta) = \mathbb{E}[\nabla \log \pi_{\theta}(a|s) A_{\pi}(s,a)]$$

PPO ratio。这里用 ρ_{θ} 表示 probability ratio, 避免和即时 reward r_t 使用同一个字母:

$$\rho_{\theta}(a_t) = \pi_{\theta}(a_t|s_t) / \pi_{old}(a_t|s_t)$$

PPO clipped objective:

$$\mathbb{E}[\min(\rho_{\theta} A_t, \text{clip}(\rho_{\theta}, 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) A_t)]$$

SAC soft target:

$$y = r + \gamma(1-d) [\min_i Q_i(s',a') - \alpha_{SAC} \log \pi(a'|s')]$$

12.5.2 算法横向对比

算法	类型	on/off-policy	关键点	适用
Policy Iteration	DP	模型已知	evaluation + improvement	小规模 MDP
Value Iteration	DP	模型已知	Bellman optimality backup	小规模 MDP
Monte Carlo	value estimation	on-policy 常见	完整 return	episodic task
TD(0)	value estimation	on-policy 常见	bootstrap	在线估值
SARSA	value-based	on-policy	target 用实际 a'	表格控制
Q-learning	value-based	off-policy	target 用 max	表格控制
DQN	value-based	off-policy	replay + target network	离散动作
REINFORCE	policy-based	on-policy	Monte Carlo PG	教学基础
A2C/A3C	actor-critic	on-policy	critic 降方差	离散/连续
PPO	actor-critic	on-policy	clipped objective	通用稳定基线
DDPG	actor-critic	off-policy	deterministic actor	连续动作
TD3	actor-critic	off-policy	double Q + delayed update	连续动作
SAC	actor-critic	off-policy	maximum entropy	连续动作

12.5.3 高频概念问答

12.5.3.1 RL 为什么难?

因为 RL 同时面对 delayed reward、探索与利用、非 i.i.d. 数据、动作影响未来数据分布、credit assignment 和训练稳定性问题。

12.5.3.2 Bellman 方程为什么重要?

Bellman 方程描述了价值函数的递归自治关系: 当前价值等于即时 reward 加未来价值。绝大多数 value-based 和 actor-critic 方法都直接或间接基于 Bellman backup。

12.5.3.3 On-policy 和 off-policy 区别?

On-policy 学习当前采样策略的价值或改进当前策略。Off-policy 用 behavior policy 的数据学习另一个 target policy，样本效率更高，但有分布偏移风险。

12.5.3.4 MC 和 TD 区别?

MC 用完整 episode 的真实 return，不 bootstrap，bias 小但 variance 大。TD 用一步 reward 加下一状态价值估计，能在线更新，variance 小但有 bootstrap bias。

12.5.3.5 SARSA 和 Q-learning 区别?

SARSA 是 on-policy，target 使用实际采样的下一个动作。Q-learning 是 off-policy，target 使用下一状态最大 Q 值。

12.5.3.6 DQN 的两个核心稳定技巧?

Experience replay 和 target network。前者打破样本相关性并复用样本，后者稳定 bootstrap target。

12.5.3.7 Double DQN 解决什么?

缓解 DQN 中 $\max$ 操作造成的 Q 值过估计。它用 online network 选动作，用 target network 评估动作。

12.5.3.8 Policy Gradient 怎么理解?

如果某个动作带来正 advantage，就增加该动作在该状态下的概率；如果 advantage 为负，就降低概率。公式是：

$$\nabla J = \mathbb{E}[\nabla \log \pi(a|s) A_{\pi}(s,a)]$$

12.5.3.9 Baseline 为什么不引入 bias?

因为 baseline 不依赖 action，而：

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s)] = 0$$

因此减去 baseline 只改变方差，不改变梯度期望。

12.5.3.10 PPO 为什么常用?

PPO 用 clipped objective 限制新旧策略差异，稳定性好、实现简单、适用面广，是 on-policy actor-critic 中非常常用的工程基线。

12.5.3.11 SAC 为什么适合连续控制?

SAC 是 off-policy，样本效率较高；最大熵目标鼓励探索并提升稳定性；双 Q 网络缓解过估计，因此在连续控制任务中表现稳定。

12.5.3.12 Offline RL 为什么难?

因为不能继续探索收集数据。学习策略可能选择数据集中没覆盖的动作，critic 对这些动作估计不可靠，容易出现 extrapolation error。

12.6 面试表达模板和项目准备

12.6.1 面试中算法讲解模板

回答某个算法时，按这个顺序讲：

1. 它解决什么问题?
2. 是 value-based、policy-based 还是 actor-critic?
3. 是 on-policy 还是 off-policy?
4. 核心目标函数或更新公式是什么?
5. 关键 trick 是什么?
6. 相比前一个算法改进在哪里?
7. 局限是什么?

例如讲 DQN：

DQN 是把 Q-learning 扩展到高维状态输入的 deep RL 算法。它用神经网络近似 $Q(s,a)$ ，适合离散动作空间。核心 target 是 $r+\gamma \max_a Q_{target}(s',a')$ 。为了稳定训练，它使用 replay buffer 打破样本相关性、复用历史经验，并使用 target network 稳定 Bellman target。它的局限是难以处理连续动作空间，并且存在 Q 值过估计问题，后续 Double DQN 对此做了改进。

12.6.2 项目准备建议

面试前建议至少准备一个可讲清楚的 RL 小项目。

优先级：

- 1. CartPole with DQN。
- 2. LunarLander with PPO。
- 3. Pendulum or HalfCheetah with SAC。
- 4. 简化推荐系统 bandit。
- 5. RLHF toy example，例如偏好数据训练 reward model。

12.6.3 面试例子索引

准备面试时，建议把算法和例子绑定记忆：

算法/概念	推荐例子	为什么适合
MDP / Bellman	GridWorld	状态、动作、转移、奖励最直观。
DP	已知地图的 GridWorld	可以明确说明“已知环境模型”。
MC	Blackjack	episode 短，完整 return 容易得到。
TD	在线推荐或持续控制	不想等 episode 结束，每步都能更新。
SARSA / Q-learning	Cliff Walking	on-policy 与 off-policy 差异明显。
Bandit	广告推荐冷启动	探索和利用最直接。
Function Approximation	Atari 或推荐系统	状态空间无法枚举。
DQN	CartPole / Atari	离散动作 + Q 网络。
Policy Gradient	连续控制机器人	动作连续，难以枚举 max。
Actor-Critic / GAE	游戏或机器人控制	Actor 选动作，Critic 降低方差。
PPO	RLHF 或 LunarLander	策略更新稳定，工程常用。
DDPG / TD3 / SAC	机械臂、MuJoCo	连续动作控制。
Offline RL	历史推荐日志	只能用离线数据，不能随便探索。
Imitation Learning	自动驾驶示范数据	从专家轨迹学习行为。
RLHF	聊天模型对齐	用人类偏好构造 reward。

回答时不要只说“我用 DQN 做 CartPole”。更好的结构是：

CartPole 是离散动作控制任务，状态低维连续，动作是 left/right，reward 是存活步数。DQN 用 Q 网络输出两个动作价值，配合 replay buffer 和 target network 稳定训练。

12.6.4 项目表达模板

项目背景：
 任务建模：
 状态空间：
 动作空间：
 奖励设计：
 算法选择：
 训练流程：
 关键工程细节：
 实验指标：
 遇到的问题：
 改进方向：

12.7 项目讲法

12.7.1 CartPole DQN 项目讲法

任务建模：

- State：小车位置、速度、杆角度、角速度。
- Action：向左或向右施加力。
- Reward：每保持一步平衡得到 1。
- Done：杆倾斜过大或小车越界。

算法选择：

- 动作空间离散，适合 DQN。
- 用 MLP 近似 Q 函数。
- epsilon-greedy 探索。
- replay buffer 存储 transition。
- target network 稳定 target。

可以强调的问题：

- epsilon decay 如何设置。
- replay buffer 大小。
- target network 更新频率。
- loss 曲线不稳定是否正常。
- reward 是否能达到环境上限。

12.7.2 LunarLander PPO 项目讲法

任务建模：

- State：位置、速度、角度、角速度、腿部接触信息。
- Action：离散喷气控制。
- Reward：接近着陆点、平稳降落、节省燃料等。

算法选择：

- PPO 稳定且适合 policy optimization。
- 使用 GAE 估计 advantage。
- 使用 entropy bonus 保持探索。
- 使用 clipped objective 限制策略更新。

可以强调：

- PPO 是 on-policy，采样效率不如 DQN/SAC。

- 但训练稳定，调参相对友好。
- advantage normalization 对训练很重要。

12.7.3 推荐系统 Bandit 项目讲法

任务建模：

- State/context：用户画像、上下文、历史行为。
- Action：推荐某个 item 或策略。
- Reward：点击、停留、转化、长期留存。

算法选择：

- 如果动作不影响长期状态，可先用 contextual bandit。
- 可比较 epsilon-greedy、UCB、Thompson Sampling。

注意点：

- 线上探索有风险，需要控制流量。
- reward 延迟和偏差要处理。
- 离线评估有 selection bias。

12.8 最终复习

12.8.1 面试前最终检查清单

- 能手写 Bellman expectation 和 optimality equation。
- 能解释 MC、TD、DP 的区别。
- 能写出 SARSA 和 Q-learning 更新公式。
- 能解释 DQN 两个核心 trick。
- 能解释 Double DQN 为什么缓解过估计。
- 能推导 policy gradient 的 log-derivative trick。
- 能证明 baseline 不引入 bias。
- 能解释 Actor-Critic 和 GAE。
- 能手写 PPO clipped objective。
- 能解释 SAC 的最大熵目标。
- 能说明 offline RL 的 extrapolation error。
- 能讲一个完整 RL 项目。

12.8.2 30 分钟口述复习路线

1. 5 分钟：MDP、return、V/Q、Bellman。
2. 5 分钟：DP、MC、TD、SARSA、Q-learning。
3. 5 分钟：DQN、Double DQN、Dueling、Replay。
4. 5 分钟：Policy Gradient、baseline、Actor-Critic、GAE。
5. 5 分钟：PPO、DDPG、TD3、SAC。
6. 5 分钟：offline RL、RLHF、项目总结。

12.9 本章练习

1. 不看笔记，用两分钟讲清从 MDP 到 PPO/SAC 的全课程主线。
2. 面对“离散动作在线交互”“连续动作在线交互”“固定离线数据”三个场景，分别给出算法候选并说明选择依据。
3. 用统一的 target、data、policy update 三个维度，对比 DQN、PPO 和 SAC。
4. 按项目表达模板，口述一个 CartPole DQN 项目，并至少说出两个可能失败的原因。

▼ 参考答案 (点击展开)

1. **两分钟主线**：MDP 定义 state、action、transition、reward 和 discount；return 的期望得到 V/Q ，Bellman 方程给出递归关系。模型已知时用 DP 做精确 backup；模型未知时 MC 用完整 return、TD 用 bootstrap target。SARSA/Q-learning 把 prediction 推到 control；状态太大时用函数逼近，DQN 通过 replay 和 target network 稳定 Q-learning。Policy Gradient 直接优化 policy，Actor-Critic 用 critic 降方差，GAE 平衡 bias/variance，PPO 限制 on-policy 更新幅度；连续动作下 DDPG/TD3/SAC 用 actor 输出 action，其中 SAC 用随机策略和最大熵目标增强探索。
2. **场景选型**：离散动作且可在线交互，可从 DQN 或 PPO 开始；DQN off-policy、样本复用高，PPO 更通用但采样成本高。连续动作且可在线交互，可优先 SAC，若需要确定性执行可比较 TD3；PPO 也能处理连续动作但为 on-policy。固定离线数据不能直接套普通 online SAC/DQN，应选择 Offline RL 方法或先做 Behavior Cloning，并重点处理 distribution shift。
3. **统一对比**：DQN 的 data 来自 epsilon-greedy behavior policy 和 replay buffer，target 是 $r + \gamma \max Q^*$ ，通过最小化 TD loss 更新 Q，再由 argmax 导出 policy；PPO 的 data 来自旧 policy 的短期 rollout，target/学习信号是 return 或 GAE advantage，通过 clipped ratio 直接更新 stochastic policy；SAC 的 data 来自 replay buffer，soft Q target 含 entropy 项，同时更新双 critic 和随机 actor，属于 off-policy actor-critic。
4. **CartPole DQN 示例**：state 是位置、速度、杆角度和角速度；action 是向左或向右；每存活一步 reward 为 1。Q 网络输出两个 action value，使用 epsilon-greedy 收集 transition，replay buffer 随机采样，target network 构造稳定 target。指标可看 episode return、成功率和样本量。失败原因包括 epsilon 衰减过快导致探索不足、target 更新过频导致 target 不稳、没有正确处理 terminal mask、学习率过大或 replay warm-up 不足。